Table of Contents

- Java Data Structures
  - Array Vs List Vs Maps
    - Lists
    - Arrays
    - Maps
    - Collision Handling in Maps
      - 1. Separate Chaining
      - 2. Open Addressing
    - Hashmap
    - TreeMap
    - LinkedHashMap
      - Difference Between LinkedList and ArrayList in Java
    - Hashmap
    - TreeMap
  - Transversing a tree
    - Pre-order Traversal (Root, Left, Right):
    - In-order Traversal (Left, Root, Right):
    - Post-order Traversal (Left, Right, Root):
    - Level-order Traversal (Breadth-First Search):
    - LinkedHashMap
  - Collection vs Streams
- Basic OOP with Java
  - What is a Class?
  - Encapsulation
  - Polymorphism
  - Inheritance
  - Abstraction
    - transient Keyword
  - Serializable Interface
    - Marker Interface
    - Cloneable Interface
  - Static Keyword
    - Private Static Methods or Fields
    - Static Array Can't Take Size at Runtime
    - Access modifier for Methods in Interface / Class / Abstract Class
    - Access Modifiers: Default vs. Protected
    - Why Use Static Method in Interfaces vs. Default
    - Final Non-Access Modifier
      - Method Cannot be Overridden
      - Final Class Can't Be Inherited
    - Association vs. Composition
    - Member Variable vs. Class Variable
  - Wrapper vs Primitive

    - Enums
    - Class Vs Interface
    - Binding in Programming
        - Static Binding (Compile-Time Binding)
        - Dynamic Binding (Run-Time Binding)
- Data Storage
    - Cache Friendliness Explained
        - Cache Line
        - Spatial Locality
    - Various data Storage
        - Meaning of "Volatile"
        - Clock Cycle
    - Summary: Speed and Use Cases
- Memory Mangement
    - Heap Vs Stack
    - Static Memory Allocation
    - Dynamic Memory Allocation
    - Reference Types
        - Memory Diagram
- Exceptions
    - Multiple Catch Blocks
    - Exceptions
    - Error
    - Checked Exceptions
    - Unchecked Exceptions (Runtime Exceptions):
    - Errors
    - Types of Error Handling
    - Best Practices for Exception Handling
    - Why Use Custom Exceptions
    - Finally Block
    - Checked vs. Unchecked Exceptions
        - Checked Exceptions
        - Unchecked Exceptions
- HTTP Basics
    - APIs in Modern Software Development
    - HTTP Status Codes
    - HTTP Headers
        - OPTIONS Method in HTTP
        - Cross-Origin Resource Sharing (CORS)
            - How CORS Works
            - Preflight Requests
            - CORS Headers
    - PUT vs. PATCH vs. POST (Idempotent Methods)
        - PATCH
        - PUT
    - RESTful API Maturity Model

        - Level 0: The Swamp of POX
        - Level 1: Resources
        - Level 2: HTTP Verbs
        - Level 3: Hypermedia Controls (HATEOAS)
- Spring Core : IOC Vs Dependency Injection Vs @Autowire
    - Inversion of Control (IoC):
    - Spring's IoC container
    - Dependency Injection (DI):
        - Code Snippet Explanation
    - @Autowired Annotation:
    - Spring Configuration Annotations
        - @Component
        - @Service
        - @Repository
- Understanding of ORM
    - Entity Mapping
    - Association Mapping
        - One-to-One Association
        - One-to-Many Association and Many-to-One Association
        - Many-to-Many Association
    - JPQL (Java Persistence Query Language)
    - JPA (Java Persistence API)
    - JPA vs. Hibernate
    - When using @Repository with a JpaRepository
    - Fetching Strategies
        - Eager Fetching
        - Lazy Fetching
    - Spring Boot's Embedded Web Server (Spring Boot vs. Spring Framework)
    - Advice Controller
    - ApplicationContext (Spring)
    - Spring Container
    - Bean Lifecycle
    - Bean Scopes
    - Request Filter/Interceptor in Spring Boot
        - Request Filter
        - Interceptor
- Advanced Memory Management
    - Memory Leaks
        - Memory Leak stops when java application stops
        - Heap Dump
        - Thread Dump
        - Profiling
    - Garbage Collection
    - OutOfMemoryError Types
        - Java Heap Space Error
        - Java PermGen Space Error

- Testing
  - Difference between Unit Testing and Integration Testing
  - Using Mockito
    - @Mock, @Spy and @InjectMocks + Stubbing (when().thenReturn())
  - JUnit
  - Spring TextContext Framework
    - Example of Transaction Management in Spring
  - Hamcrest vs Mockito
  - @Rule vs @ExtendWith and @RunWith
    - @Rule
    - @RunWith
    - @ExtendWith
    - Differences Summary
  - Reflection
  - Mockito Doesn't Allow Stubbing of Final, Private, or Static Methods
  - ArgumentCaptor
- Parallel Programming
  - Concurrent Collections
    - ConcurrentHashMap
    - ConcurrentLinkedQueue
    - CopyOnWriteArrayList
    - Difference between HashMap and ConcurrentHashMap
      - Example: Issues with Normal HashMap in Multi-threaded Environment
  - CompletableFutures
    - Methods in CompletableFuture for Chaining and Combining
    - 3. Single Table
    - 4. Joined
    - Choosing a Strategy
  - Formulas
- Maven
  - Dependency Management
    - Key Tags for Maven Dependencies
      - 1. `<groupId>`
      - 2. `<artifactId>`
      - 3. `<version>`
      - 4. `<scope>`
      - 5. `<exclusions>`
      - 6. `<optional>`
      - 8. `<type>`
      - Example of a Maven Dependency
    - Remote Repositories
    - The Super POM.
    - Local Repositories
    - The `<dependencyManagement>` tag
    - BOM
    - Parent POM

        - BOM vs Parent POM
        - Dependency Tree
        - Exclusion
    - Build Lifecycle (validate, compile, test, package, install, deploy)
        - Predefined Goals
        - Maven Lifecycle Phases
    - Plugins
        - Compiler Plugin
        - Surefire Plugin
        - JAR Plugin
    - POM Packaging vs JAR
    - Module Export Packages
- Security
    - Securing REST APIs with Spring Security
        - Authentication
        - Authorization
        - Token (JWT - JSON Web Token)
        - Refresh Token
    - Outh2
        - Key Roles and Components
        - Detailed Flow
            - Authorization Request
            - Authorization Grant
        - Access Token Request
        - Access Token Response
            - Access Protected Resource
    - Servlet Filters
        - A naive SecurityFilter
        - FilterChains
    - Authentication with Spring Security
        - Having access to the user's password
        - AuthenticationProvider: Not having access to the user's password
    - Authorization with Spring Security
        - GrantedAuthorities nd SimpleGrantedAuthorities?
        - AuthenticationManager: Where to store and get authorities?
        - WebSecurityConfigurerAdapter for Authorities
    - Common Exploit Protections
        - Cache Control Attacks
        - Content Sniffing
        - Clickjacking
        - Cross-Site Scripting (XSS):
    - Cross-Site-Request-Forgery: CSRF
        - Typical CSRF Attack Flow
        - How CSRF Protection Works in Restless/stateless application
            - Why It's Secure
            - In a stateless REST API, CSRF protection is typically not necessary

        - Session-Centric Applications:
        - CSRF Protection with Spring MVC and Thymeleaf
    - Password encryption techniques
        - Four main types of Encryption
            - How Salt Works
        - 5 Common Password Encryption Methods
        - Common Password Encryption Formats
    - Method Security
        - What is the difference between @PreAuthorize, @Secured and @RolesAllowed?
        - PostAuthorize
        - Using @PreFilter and @PostFilter Annotations
        - Method Security Meta-Annotation(Creating a global interafce)
        - Security Annotation at the Class Level
        - Multiple Security Annotations on a Method
- Java Versions
    - Java 8 Features
    - Spring 1 Vs 2 Vs 3
        - Spring Framework 1
        - Spring Framework 2
        - Spring Framework 3 (Focus on simplification and ease of use)
        - Table
    - AOP
        - Proxy-based Implementation:
        - AspectJ-based Implementation:
    - Varags
        - Declarative transaction management
    - @Configuration annotation for Java-based configuration
    - Lombok
    - How spring handle multiple requests to endpoints concurrently
        - Number of threads can one laptop handle
    - Java Versions Comparision
        - Summary
    - Jdk vs Jre what are they are what are the differences
        - JDK (Java Development Kit):
        - JRE (Java Runtime Environment):
        - Differences
- Java features
    - Record
    - SpEL
        - Key Features of SpEL
        - Common Use Cases

Java Data Structures

Array Vs List Vs Maps

Feature	List	Array	Map
Definition	A collection of elements with dynamic size	A collection of elements with a fixed size	A collection of key-value pairs
Memory Allocation	Dynamic (can grow and shrink as needed)	Static (allocated at the time of declaration)	Dynamic, can grow as needed
Access Time	O(n) for general access (sequential access)	O(1) for direct access (random access)	O(1) average for key-based access (hashing)
Insertion	O(1) at the beginning or end; O(n) at the middle	O(n) for general insertion	O(1) average for key-based insertion
Deletion	O(1) at the beginning or end; O(n) at the middle	O(n) for general deletion	O(1) average for key-based deletion
Order Preservation	Maintains order of elements	Maintains order of elements	Does not maintain order (**unless using LinkedHashMap**)
Duplicate Keys	Not applicable	Not applicable	Keys must be unique
Use Cases	Preferred for dynamic lists where order matters	Preferred for fixed-size collections with frequent access	Preferred for fast lookups and retrieval by keys
Example in Java	`List<Integer> list = new ArrayList<>();`	`int[] array = new int[10];`	`Map<String, Integer> map = new HashMap<>();`
Flexibility	More flexible, can easily grow or shrink	Less flexible, fixed size once declared	Flexible, can grow with new key-value pairs
Memory Overhead	Moderate to high due to dynamic resizing and pointers	Lower as it only contains data elements	Higher due to hashing overhead
Search Time	O(n)	O(n) for unsorted, O(log n) for sorted	O(1) average
Cache Friendliness (works with the CPU cache)	Less cache-friendly due to potential fragmentation	More cache-friendly due to contiguous memory	Depends on hash table implementation

Feature	List	Array	Map
Ordered Retrieval	Maintains insertion order (e.g., ArrayList, LinkedList)	Maintains element order	Depends on implementation (e.g., LinkedHashMap)
Algorithmic Considerations	Easy to implement and flexible; good for frequent insertions and deletions	Efficient for fixed-size data with known indices; ideal for scenarios requiring fast access	Highly efficient for lookups, insertions, and deletions by key; hash function quality crucial for performance
Example Use Cases	Dynamic lists, playlists, queues, and stacks	Static datasets, matrix operations, fixed collections	Dictionaries, associative arrays, and databases

Lists

- Less cache-friendly due to potential fragmentation(store elements non-contiguously(adjacent) in memory where Each element contains a pointer to the next element) and non-contiguous storage (especially true for linked lists).
- When accessing a node in a linked list, the CPU loads the cache line containing that node. However, because the next node is not guaranteed to be nearby in memory, it is often not in the same or adjacent cache line.
- As a result, accessing the next node likely results in a cache miss, causing the CPU to load a new cache line from the main memory, which is a slower operation.

Arrays

Are Highly cache-friendly due to contiguous memory storage, meaning that elements are laid out sequentially in memory.

- When accessing array elements sequentially, once the first element is loaded into the cache, subsequent elements are likely to be in the same cache line or nearby lines., making sequential access very efficient.
- Choosing between arrays and ArrayLists in Java boils down to the specific requirements of the application. If the collection's size is fixed and performance is a key concern, arrays are the better choice.

Maps

Cache friendliness varies with the hash table implementation. Good hash functions and collision resolution strategies can improve cache performance, but linked structures for collisions can reduce cache efficiency.

Collision Handling in Maps

A collision occurs when two keys hash to the same index in the array. Different strategies are used to handle these collisions:

1. Separate Chaining

In separate chaining, each bucket (array index) contains a linked list (or another secondary data structure like a binary tree) of all elements that hash to the same index.

```
Advantages:

- Simple to implement.
- No need for resizing when the table is full.

Disadvantages:
- Additional memory overhead to store pointers for the linked lists.
- Poor cache performance due to non-contiguous memory access for linked list nodes.
```

2. Open Addressing

In open addressing, all elements are stored within the hash table itself. When a collision occurs, the algorithm searches for the next available slot according to a probing sequence.

```
Advantages:
- No additional memory overhead for linked lists.
- Potentially better cache performance due to contiguous memory storage.
```

- Disadvantages:
  - More complex to implement.
  - Performance degrades when the table is nearly full, requiring resizing and rehashing.

Hashmap

- HashMap uses the hashCode method of keys to determine their bucket location.

TreeMap

- Sorted Order: Maintains entries in **natural order** of keys or according to a comparator provided at map creation.
- Key Methods:
  - Uses compareTo() (or Comparator if provided) to order keys.
  - For containsKey() and get(), the compareTo() method is used to navigate the tree structure.
- TreeMap Structure
  - Nodes: Each node contains a key-value pair.
  - Tree Properties: The tree maintains properties that ensure balance, allowing for efficient operations like insertion, deletion, and lookups.
- Diagram of a TreeMap

```
TreeMap (Red-Black Tree):
       [30, 300]
       /       \
 [10, 100]    [40, 400]
    /            \
 [5, 50]         [35, 350]
```

- Root Node: The node with key 30 and value 300.

LinkedHashMap

- Maintains Insertion Order: Unlike HashMap, it maintains the ** order in which entries were inserted**.
- Key Methods:
  - Uses hashCode() and equals() like HashMap but maintains a linked list of entries to keep the insertion order.

Difference Between LinkedList and ArrayList in Java

- Use ArrayList when frequent access to elements by index is required.
- Use LinkedList when you need to add or remove elements frequently from the middle of the list.

Hashmap

- HashMap uses the hashCode method of keys to determine their bucket location.

TreeMap

- a TreeMap cannot contain null keys. The reason is that TreeMap uses natural ordering or a comparator for its keys, and null is not comparable.
- Sorted Order: Maintains entries in **natural order** of keys or according to a comparator provided at map creation.
- Key Methods:
  - Uses compareTo() (or Comparator if provided) to order keys.
  - For containsKey() and get(), the compareTo() method is used to navigate the tree structure.
- TreeMap Structure
  - Nodes: Each node contains a key-value pair.
  - Tree Properties: The tree maintains properties that ensure balance, allowing for efficient operations like insertion, deletion, and lookups.
- Diagram of a TreeMap

```
TreeMap (Red-Black Tree):
       [30, 300]
       /       \
```

```
  [10, 100]    [40, 400]
     /            \
  [5, 50]         [35, 350]
```

- Root Node: The node with key 30 and value 300.

Transversing a tree

Transversing a tree typically involves visiting all the nodes in the tree in a specific order. The most common tree traversal methods are:

- Pre-order Traversal (Root, Left, Right)
- In-order Traversal (Left, Root, Right)
- Post-order Traversal (Left, Right, Root)
- Level-order Traversal (Breadth-First(level by level) Search)

```
      1
    / \
   2   3
  / \ / \
 4  5 6  7
```

Pre-order Traversal (Root, Left, Right):

Visit the root node first, then recursively visit the left subtree, followed by the right subtree.

- Traversal Order: 1, 2, 4, 5, 3, 6, 7

```
public void preOrderTraversal(TreeNode node) {
    if (node == null) {
        return;
    }
    System.out.print(node.val + " ");
    preOrderTraversal(node.left);
    preOrderTraversal(node.right);
}
```

In-order Traversal (Left, Root, Right):

Visit the left subtree first, then the root node, and finally the right subtree.

- Traversal Order: 4, 2, 5, 1, 6, 3, 7

```
public void inOrderTraversal(TreeNode node) {
    if (node == null) {
        return;
```

```
    }
    inOrderTraversal(node.left);
    System.out.print(node.val + " ");
    inOrderTraversal(node.right);
}
```

Post-order Traversal (Left, Right, Root):

Visit the left subtree first, then the right subtree, and finally the root node.

- Traversal Order: 4, 5, 2, 6, 7, 3, 1

```
public void postOrderTraversal(TreeNode node) {
    if (node == null) {
        return;
    }
    postOrderTraversal(node.left);
    postOrderTraversal(node.right);
    System.out.print(node.val + " ");
}
```

Level-order Traversal (Breadth-First Search):

Visit nodes level by level, from top to bottom and left to right.

- Traversal Order: 1, 2, 3, 4, 5, 6, 7

LinkedHashMap

- Maintains Insertion Order: Unlike HashMap, it maintains the ** order in which entries were inserted**.
- Key Methods:
  - Uses hashCode() and equals() like HashMap but maintains a linked list of entries to keep the insertion order.
  -

Collection vs Streams

Feature	Collections	Streams
Definition	Data structures that store elements.	A sequence of elements supporting functional operations.
Usage	Primarily used for storing and managing groups of objects.	Primarily used for processing data with functional operations.
Data Storage	Stores data elements directly.	Does not store data; instead, it **processes** data pipelined from a source.

Feature	Collections	Streams
Mutability	Can be mutable or immutable.	Streams are immutable; operations produce new streams.
Iteration	Iteration is done using loops (e.g., for-each loop) or iterators.	Iteration is done using internal iteration via functional methods (e.g., `forEach`).
Eager vs. Lazy Evaluation	Operations on collections are eagerly evaluated.	Intermediate operations are lazily evaluated, only executed when a terminal operation is invoked.
Mutability of Elements	Elements can be modified directly within the collection.	Streams do not modify the underlying data; they produce new streams or results.
Parallel Processing	Requires explicit handling to process elements in parallel.	Built-in support for parallel processing via `parallelStream()`.
Filter/Map/Reduce Operations	Requires manual implementation using loops or custom methods.	Directly supports filter, map, and reduce operations using functional programming paradigms.
Reusability	Can be reused after iteration.	Streams cannot be reused; once a terminal operation is invoked, the stream is closed.
Null Handling	Collections can contain null elements.	Streams generally avoid nulls and encourage the use of `Optional` to handle absent values.
Complexity	More manual effort required for operations like filtering and mapping.	Simplifies complex data processing with built-in operations.
API Introduction	Available since Java 1.2.	Introduced in Java 8.
Example Usage	`List<String> list = new ArrayList<>();`	`Stream<String> stream = list.stream();`
Bulk Operations	Iteration-based (e.g., loops, `Collections` utility methods).	Declarative (e.g., `filter`, `map`, `reduce`).
Memory Consumption	Stores all elements in memory.	Processes elements on-demand, which can be more memory-efficient.

Basic OOP with Java

What is a Class?

A class is a blueprint for creating objects (a particular data structure), providing initial values for state (member variables/ attributes or fields), and implementations of behavior (member functions or methods). A class encapsulates data for the object and methods to manipulate that data.

Usage of Classes:

- Encapsulation of Data for protection (from modification and un authorized access)
- Inheritance (promoting a hierarchical classification and reuse of existing implementations)

Encapsulation

- Differences: Promotes **modularity** and maintainability, hides internal state and only exposes necessary parts via methods.

Polymorphism

- Concept: Polymorphism allows objects to be treated as instances of their parent class rather than their actual class. It supports method overriding and overloading.
- Differences: Enhances **flexibility** and integration, enabling a single interface to represent different underlying forms (data types).

Inheritance

Concept: Inheritance is a mechanism wherein a new class inherits properties and behavior (fields and methods) from an existing class.

Abstraction

- Concept: Abstraction is the concept of hiding the complex implementation details and showing only the essential features of the object.
- Differences: Reduces complexity, focuses on relevant data and operations, can be achieved using abstract classes and interfaces.

transient Keyword

Purpose: Used in serialization to indicate that a field should not be serialized. Usage: When an object is serialized, fields marked as transient are not included in the serialized form. This is useful for fields that are derived or not essential to the object's state.

```
import java.io.*;

class Person implements Serializable {
    private String name;
    private transient int age; // This field will not be serialized

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

Serializable Interface

Serialization is the process of converting an object's state (its fields and their values) into a format that can be easily stored or transmitted (e.g., a byte stream or JSON).

- Serializable Interface: **A marker interface (an interface with no methods)** used to indicate that a class can be serialized.
- Purpose: Allows an object to be converted into a byte stream for storage or transmission. Classes implementing Serializable must ensure all non-transient fields are serializable.
- Serialization: To save an object to a file or transmit it over a network.
- In Java, when a field is declared as `transient`, it means that this field should not be included in the serialization of the object. This is useful for fields that are either temporary, not necessary for object reconstruction, or sensitive information that should not be persisted.

```
// If you serialize an instance of Person, the name field will be included
in the serialized data, but the age field will not.
class Person implements Serializable {
    private String name;
    private transient int age; // This field will not be serialized

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getters and setters
    public String getName() { return name; }
    public int getAge() { return age; }
}


public class SerializationExample {
    public static void main(String[] args) throws IOException,
ClassNotFoundException {
        Person person = new Person("Alice", 30);

        // When the object is serialized, the age field is skipped.
        // Serialize
        ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("person.ser"));
        out.writeObject(person);
        out.close();

        // Deserialize
        ObjectInputStream in = new ObjectInputStream(new
FileInputStream("person.ser"));
        Person deserializedPerson = (Person) in.readObject();
        in.close();


        // This means that during deserialization (when converting the
```

```
byte stream back into a Person object),
        // the age field will be missing or reset to its default value (0
for an int).
        System.out.println("Name: " + deserializedPerson.getName()); //
Alice
        System.out.println("Age: " + deserializedPerson.getAge());   // 0
(default value for int)


        // Convert to JSON
        ObjectMapper mapper = new ObjectMapper();
        String json = mapper.writeValueAsString(person);
        // In the context of JSON (such as when sending data over a web
API), if a field is transient
        // it will not appear in the JSON representation of the object.
        System.out.println(json); // Output might be: {"name":"Alice"}

    }
}
```

Marker Interface

A marker interface in Java is an interface that has no methods or fields.

- It is used to signal or mark a class with a particular property or behavior.
- The absence of methods allows the interface to be used purely for **tagging** purposes, indicating that a class implementing this interface has a certain property or should be treated in a specific way.
- Example => Serializable: Marks a class as capable of serialization.

Cloneable Interface

- Cloneable is indeed a marker interface; it has no methods.
- It is used to indicate that the Object.clone() method can be used to create a copy of an object.
- `clone()` Method: The Cloneable interface itself does not declare a clone() method. Instead, it is the Object class that defines the clone() method, which should be overridden in classes that implement Cloneable to provide the cloning functionality.

Cloneable Interface: Has no methods. It's a marker interface. clone() Method: Defined in the Object class. When a class implements Cloneable, it overrides the clone() method to provide the actual cloning behavior.

```
class Person implements Cloneable {
    private String name;

    public Person(String name) {
        this.name = name;
    }

    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone(); // Calls Object's clone method
```

```
    }

    public String getName() {
        return name;
    }
}
```

Static Keyword

Static Methods or fields belong to the class rather than any specific instance of the class. They can be called on the class itself, not on instances of the class.

- Class-Level: They are associated with the class, not with any specific object of the class.

Private Static Methods or Fields

- Private static methods serve as utility methods within a class. Since they are not tied to any instance, they can be called using the class name without the need for an object. Making them private ensures that they are not accessible from outside the class, maintaining encapsulation.

Static Array Can't Take Size at Runtime

In Java, arrays are static in the sense that their size must be determined when they are created, and this size cannot be changed later. This means that you **cannot dynamically allocate** the size of an array **based on runtime information**.

```
int[] array = new int[10]; // size 10 is fixed and cannot change
```

Static Anything?: The term "static" here refers to the fixed nature of the array's size after its creation.

Access modifier for Methods in Interface / Class / Abstract Class

- Class and abstract class methods: **Default** to package-private if no modifier is specified.
- Interface methods: Default to **public** and abstract unless explicitly declared as private, default, or static.
- class access modifieries can only be default and public

Access Modifiers: Default vs. Protected

- Default (Package-Private) Access Modifier: In Java, if no access modifier is specified for a class, method, or variable, it is given package-private access by default. This means that it can only be accessed by other classes within the same package. It is less restrictive than private but more restrictive than protected or public.

```
class MyClass { // This class is package-private
}
```

- The protected access modifier allows access within the same package and also by subclasses in different packages.

Why Use Static Method in Interfaces vs. Default

- Static methods in interfaces **cannot be overridden** by implementing classes. They belong to the interface itself and are not inherited by the classes that implement the interface. This is useful when you want to provide **utility methods** related to the interface, which do not depend on instance data.
- Default Methods: These methods can be overridden by implementing classes. They allow you to add functionality to an interface without breaking the implementing classes. Default methods are useful when you want to provide **common functionality** to all implementing classes, with the option for them to override it.

Final Non-Access Modifier

Method Cannot be Overridden

- When a method is declared as final, it cannot be overridden by any subclass. This is used when you want to prevent alteration of the method's behavior by subclasses.

Final Class Can't Be Inherited

Explanation: When a class is declared as final, it cannot be extended by any other class. This is used to prevent the class from being subclassed, ensuring that its implementation remains unchanged.

Association vs. Composition

- Association: Represents a "has-a" relationship where two classes are related but can exist independently. For example, a Library and a Book have an association; a library can exist without a particular book.
- Composition: A strong form of association where the dependent object cannot exist without the parent. For example, a House and Room have a composition relationship; a room cannot exist without the house it belongs to.

Member Variable vs. Class Variable

Member Variable: Also known as instance variables, these are associated with an instance of a class. Each object of the class has its own copy of the instance variable. Class Variable: Also known as static variables, these are associated with the class itself rather than any particular instance. All instances of the class share the same copy of the static variable.

Wrapper vs Primitive

- Wrapper Classes: Integer, Double, Boolean, etc., have overridden equals() method. Can be null. Methods can throw NullPointerException if called on a null reference.
- Primitive Types: int, boolean, etc., cannot be null. Do not have equals() method; comparison is done using ==.

Enums

Define a fixed sets of named constants, and can have methods and fields.

```
public enum Day {
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY;

    public boolean isWeekend() {
        return this == SATURDAY || this == SUNDAY;
    }
}
```

Class Vs Interface

Feature	Class	Interface
Definition	A blueprint for creating objects with fields and methods	A contract specifying methods that a class must implement
Inheritance	Supports single inheritance (can extend one class)	Supports multiple inheritance (a class can implement multiple interfaces)
Implementation	Can provide both method implementations and declarations	Can only declare methods (default methods in modern languages can have implementation)
Fields	Can have fields (state)	Cannot have instance fields, only static final fields
Access Modifiers	Can use all access modifiers (public, private, protected)	Methods are implicitly public (default methods can be an exception)
Constructors	Can have constructors	Cannot have constructors
Purpose	To represent a real-world entity with state and behavior	To define a contract for classes without dictating how it should be implemented
Example in Java	`class MyClass { ... }`	`interface MyInterface { ... }`
Method Implementation	Methods can have code	Methods cannot have code (except default methods)
Usage	Used to create objects and define their behavior	Used to specify a set of methods that implementing classes must provide

Binding in Programming

Binding refers to the process of linking a procedure call to the code that executes the procedure. In the context of programming languages, there are two types of binding:

- Static Binding (Compile-Time Binding)
- Dynamic Binding (Run-Time Binding)

Static Binding (Compile-Time Binding)

- Static Binding, also known as early binding, occurs when the method to be invoked is determined at compile time.
- Resolved at Compile-Time: The compiler knows exactly which method to call during the compilation of the program.
- Speed: Since the binding is done at compile time, it is generally faster.

Dynamic Binding (Run-Time Binding)

- Dynamic Binding, also known as late binding, occurs when the method to be invoked is determined at runtime.
- Resolved at Run-Time: The method to be called is determined when the program is running, based on the actual object type.
- Flexibility: Allows for more flexible and dynamic behavior in programs, enabling polymorphism.

Data Storage

Cache Friendliness Explained

Cache friendliness refers to how well a data structure works with the CPU cache, which can significantly affect the performance of your program. CPU caches are small, fast memory locations closer to the CPU than the main memory (RAM), and accessing data from the cache is much faster than accessing it from RAM. Data structures that make good use of the cache can result in faster programs.

Cache Line

A cache line is a small block of memory (typically 64 bytes) that the CPU loads into the cache at once. When data is accessed, the CPU loads not just the specific data item, but a contiguous block of memory around it into the cache.

Spatial Locality

CPUs exploit spatial locality by loading contiguous blocks of memory into the cache, anticipating that if one memory location is accessed, nearby memory locations will likely be accessed soon.

Various data Storage

In computer hardware, data can be stored in various places, each with different characteristics in terms of speed, size, and purpose. Here's a breakdown of the main types of data storage, listed from fastest to slowest:

Storage Type	Speed	Description	Use	Capacity	Access Time

Storage Type	Speed	Description	Use	Capacity	Access Time
CPU Registers	Fastest	Small, fast storage locations within the CPU	To hold data that the CPU is currently processing (Temporary storage for operands and control information)	limited(Few bytes to a few dozen bytes)	A few clock cycles(faster than nanoseconds)
CPU Cache	Very fast (slower than registers but faster than RAM)	Small, high-speed memory located close to the CPU. It stores frequently accessed data and instructions to reduce the time the CPU needs to fetch data from the main memory.	To store copies of frequently accessed memory data to reduce latency	L1: Tens of KB L2: Hundreds of KB to a few MB L3: A few MB to tens of MB	A few nanoseconds
Random Access Memory (RAM)	Fast	Volatile memory used to store active data (sed or processed by the CPU) and instructions	To store the operating system, applications, and currently used data	Gigabytes (GB)	Tens of nanoseconds
Solid State Drive (SSD)	Faster than HDD, slower than RAM	Non-volatile storage using NAND flash memory	To store the operating system, applications, and user data for quick access.	Hundreds of GB to multiple TB	Microseconds to milliseconds
Hard Disk Drive (HDD)	Slower than SSD	Non-volatile storage that uses spinning magnetic disks to store data. It has mechanical parts that move to read/write data.	To store large amounts of data at a lower cost, such as media files, backups, and archives	Hundreds of GB to multiple TB	Milliseconds

Storage Type	Speed	Description	Use	Capacity	Access Time
Optical Drives (CD/DVD/Blu-ray)	Slower than HDD	Uses laser technology to read/write data on discs	Media distribution, backups, and archival storage	GB range (up to 100 GB for Blu-ray)	Milliseconds
USB Flash Drives	Variable (depends on USB version)	Portable, non-volatile storage using flash memory	Data transfer and portable storage	GB range	Variable (depends on USB version)
Network Attached Storage (NAS)	Variable (depends on network speed)	Storage devices connected to a network, providing data access to multiple users	Centralized data storage and backup for multiple devices	TB to PB range	Variable (depends on network speed)

Meaning of "Volatile"

Volatile memory refers to types of computer memory that require power to maintain the stored information. When the power is turned off, all data stored in volatile memory is lost. The primary example of volatile memory is RAM (Random Access Memory).

Clock Cycle

- The basic unit of time for CPU operations, determined by the CPU's clock speed.
- 1 clock cycle duration depends on the CPU clock speed (e.g., 0.33 nanoseconds for a 3 GHz CPU).

Summary: Speed and Use Cases

Use Case	Recommended Storage Types
Speed-critical applications	CPU Registers, CPU Cache
Running programs and active data	RAM
Operating system and applications	SSD
Large data storage and cost-effective solutions	HDD
Data backup and portability	Optical Drives, USB Flash Drives, NAS

Memory Mangement

Heap Vs Stack

Feature	Stack Memory	Heap Memory

Feature	Stack Memory	Heap Memory
Memory Allocation	Static memory allocation (done at compile time)	Dynamic (done at runtime)
Storage	Method call frames, local variables, and references	Objects and arrays
Access Speed	Faster	Slower
Management	LIFO (Last In, First Out) order	Managed by Garbage Collector
Lifetime	Lives until the method execution completes	Lives until no references remain (GC removes it)
Size Limit	Usually smaller and size is determined at runtime	Typically larger and size can grow as needed
Thread Safety	Each thread has its own stack	Shared among all threads, not thread-safe without synchronization
Common Uses	Primitive variables, function calls, references to heap objects	Objects, arrays, instance variables
Example Code	`int a = 10;`	`Integer a = new Integer(10);`

Static Memory Allocation

Static memory allocation is done at compile time. This means that the amount of memory required for variables and data structures is determined before the program runs and remains fixed throughout the program's execution.

- Done at compile time.
- Fixed memory size.
- Lifetime of the entire program.
- Automatically managed.

Dynamic Memory Allocation

- Dynamic memory allocation is done at runtime. Memory is allocated as needed during the execution of the program, allowing for more flexible and efficient use of memory.
- Done at runtime.
- Flexible memory size.
- Variable lifetime.
- Manually or automatically managed.

Reference Types

In Java, reference types are used to store references to objects. Unlike primitive types (int, char, etc.) which store actual values, reference types store the address (reference) of the object in memory.

Step-by-Step Explanation of Reference Types

1. When you declare a reference type variable, you are essentially creating a variable that can hold the address of an object of a specific type.

```
Person person;
```

- At this point, `person` is declared but not yet initialized. It does not point to any object.
- `person` is a reference variable stored in the stack.

2. To create an object, you use the new keyword. This allocates memory for the object on the heap.

```
person = new Person();
```

- Now, a new Person object is created on the heap. The person variable holds the reference to this object.
- `new Person()` allocates memory for a Person object on the heap.
- The `person` reference variable in the stack now points to the memory address of the `Person` object in the heap.

3. Using the reference variable, you can access members (fields and methods) of the object.

```
person.setName("Alice");
person.setAge(30);
```

- `setName` and `setAge` methods are called on the object referenced by person.
- `name` and `age` fields are set in the object on the heap.

Memory Diagram

- Stack Memory:

```
+------------------+
| Reference: person|
| Value: 0x1a2b3c4d|  // Memory address of the object in heap
+------------------+
```

- Heap Memory:

```
0x1a2b3c4d: Person Object
+--------------+
| name: "Alice"|
```

```
| age: 30      |
+--------------+
```

Exceptions

- Exception handling: This feature of Java enables developers to manage the runtime errors caused by the exceptions.

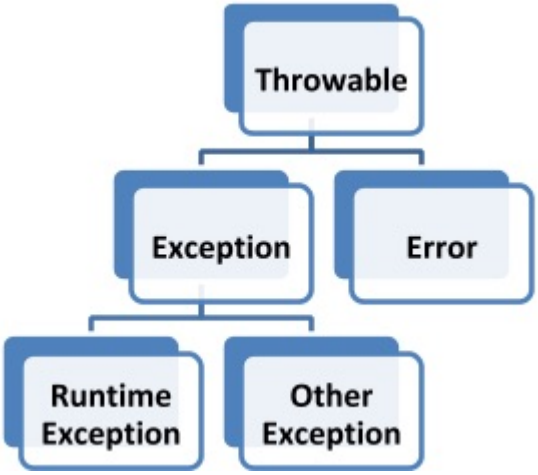
Multiple Catch Blocks

Multiple catch blocks are necessary when you need to handle different exceptions in different ways. The catch blocks are executed in the order they appear, and the first block that matches the exception type will be executed. The first matching block handles the exception, and the rest are skipped.

```
public class MultipleCatchExample {
    public static void main(String[] args) {
        try {
            int num = Integer.parseInt("XYZ"); // This will throw
NumberFormatException
            int result = 10 / 0; // This will throw ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero.");
        } catch (NumberFormatException e) {
            System.out.println("Number format exception: " +
e.getMessage());
        } catch (Exception e) {
            System.out.println("An unexpected error occurred: " +
e.getMessage());
        }
    }
}
```

Exceptions

- Exceptions are the unwanted errors or bugs or events that restrict the normal execution of a program. Each time an exception occurs, program execution gets disrupted. An error message is displayed on the screen.



Error

- Error can be defined as an abnormal condition that indicates something has gone wrong with the execution of the program. These are not handled by Java programs.

Checked Exceptions

Checked exceptions are also known as **compile-time** exceptions as these exceptions are checked by the compiler during the compilation process to **confirm whether the exception is handled** by the programmer or not. If not, then the system displays a compilation error. For example, SQLException, IOException, InvocationTargetException, and ClassNotFoundException.

When the program throws exceptions during the compilation process, There are two methods of resolving such issues.

1. You can declare the exception with the help of the **throw** keyword.The throws keyword is used in method signatures to declare that a method can throw exceptions.It propagates the exception to the calling method to handle it.

2. try-catch block

Unchecked Exceptions (Runtime Exceptions):

The unchecked exceptions are those exceptions that occur during the execution of the program. Hence they are also referred to as **Runtime exceptions**. These exceptions are generally ignored during the compilation process. They are not checked while compiling the program. For example, programming bugs like logical errors, and using incorrect APIs. These are subclasses of **RuntimeException**. Common unchecked exceptions include NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, etc.

Errors

Errors are serious issues that are not typically caught or handled by the application. They are subclasses of Error. Common errors include OutOfMemoryError, StackOverflowError, etc.

throws Keyword:

The throws keyword is used in method signatures to declare that a method can throw exceptions. Example:

Types of Error Handling

1. Catch specific exceptions to handle known error conditions.
2. Catching Multiple Exceptions in One Catch Block:

Java 7 introduced the ability to catch multiple exceptions in a single catch block using the pipe (|) symbol
`catch (FileNotFoundException | IOException e)`

1. Re-throwing Exceptions: Sometimes, it's necessary to catch an exception and then re-throw it to be handled at a higher level.

Best Practices for Exception Handling

- Use Specific Exceptions: Catch and handle specific exceptions rather than using a generic Exception catch block.
- Meaningful Exception Messages: Provide meaningful messages to help identify the issue.
- Resource Management: Use the **finally** block or **try-with-resources statement** to close resources.
- Avoid Silent Failures: Avoid empty catch blocks.
- Propagate Exceptions Appropriately: Use the throws keyword to propagate exceptions when necessary.

Why Use Custom Exceptions

Explanation: Custom exceptions are user-defined exceptions **(Domain-Specific Scenarios)** created to **handle specific scenarios** in your application that are **not** adequately **addressed** by the standard Java exceptions. Creating custom exceptions improves **code readability** and maintainability by clearly conveying what went wrong.

Finally Block

Explanation: The finally block in Java is used to execute code that must run regardless of whether an exception was thrown or not. It is primarily used for cleanup activities like closing files, releasing resources, etc.

Checked vs. Unchecked Exceptions

Checked Exceptions

Checked exceptions are exceptions that are checked at **compile-time**. These exceptions must be either caught in a try-catch block or **declared in the method's throws clause**.

- Examples: **IOException**, **SQLException**, **FileNotFoundException**.
- Checked exceptions are used for conditions that are recoverable and are expected to occur during normal program execution.
- Stack Overflow: StackOverflowError is an **error**, **not an exception**, and is unchecked. Errors are generally conditions that are not expected to be caught by applications.

Unchecked Exceptions

- Unchecked exceptions are exceptions that **are not checked at compile-time**. These include subclasses of **RuntimeException** and Error.
- Examples: **NullPointerException**, **ArrayIndexOutOfBoundsException**, **ArithmeticException**.
- Unchecked exceptions are usually programming errors, such as **logic errors**, and are not expected to be caught or declared.

HTTP Basics

APIs (Application Programming Interfaces) are used to allow different software systems to **communicate** with each other. They enable the integration of various services and functionalities, making it easier to build scalable and modular applications.

APIs in Modern Software Development

APIs (Application Programming Interfaces) are crucial because they enable different software systems to **communicate** with each other. In modern development, APIs are the **backbone of microservices, mobile apps, and web applications**.

HTTP Status Codes

Significance of HTTP Status Codes: HTTP status codes indicate the **result** of an HTTP request. They help clients understand whether a request was successful, if it failed, and why.

- **200 OK**: The request has succeeded.
- **201 Created**: The request was successful, and a new resource was created.
- **204 No Content**: The server successfully processed the request, but is not returning any content.
- **400 Bad Request**: The server could not understand the request due to invalid syntax.
- **401 Unauthorized**: The client must authenticate itself to get the requested response.
- **403 Forbidden**: The client does not have access rights to the content.
- **404 Not Found**: The server can not find the requested resource.
- **405 Method Not Allowed**: The request method is known by the server but is not supported by the target resource.
- **100 Continue**: The server has received the request headers, and the client should proceed to send the request body.
- **500 Internal Server Error**: The server encountered an unexpected condition.

HTTP Headers

HTTP headers are key-value pairs sent between the client and server in an HTTP request or response. They provide essential information about the request or response, such as content type, length, and encoding, and control the behavior of the client or server.

- Request Headers
  - `Content-Type`: Indicates the media type of the resource(in the HTTP request).
  - `Authorization`: Contains credentials for authenticating the client.
  - `Host`: The domain name of the server.
  - `User-Agent`: Information about the client software.

    - `Accept`: Media types the client can handle.
    -
- Response Headers
- `Content-Type`: Media type of the body sent to the client.
- `Content-Length`: Size of the body in bytes.
- `Set-Cookie`: Cookie data to be stored by the client.
- `Location`: URL to redirect a page.
- `Server`: Information about the server software.

OPTIONS Method in HTTP

- The OPTIONS HTTP method is used to describe the **communication options** for the target resource. It is typically used by clients to determine which **HTTP methods are supported** by a server or to check the **server's Cross-Origin Resource Sharing (CORS) policy**.
- CORS Preflight Requests: Browsers use the OPTIONS method to perform a "preflight" request to determine if the actual request is safe to send.
- API Documentation: Can be used to inform clients of the allowed methods on a specific endpoint.

Cross-Origin Resource Sharing (CORS)

- CORS is a security feature implemented by web browsers to prevent web pages from making requests to a domain different from the one that served the web page.
- This policy is in place to protect users from certain types of attacks, such as Cross-Site Request Forgery (CSRF).
- By default, web pages can only make requests to the same origin (same domain, protocol, and port) that served the web page. CORS is an extension to this policy, allowing controlled access to resources on a different origin.
- CORS is crucial for enabling secure cross-origin requests, allowing resources like APIs to be accessed from different origins while still protecting users from malicious activities. It allows developers to specify which domains are permitted to access server resources, making it a flexible but controlled way to share data between different origins.

How CORS Works

- When a web page attempts to make a request (such as an AJAX request) to a different domain, protocol, or port, the browser first checks if the server allows this cross-origin request.
- If the server allows it, it includes specific headers in its response to indicate that the request is permitted.

Preflight Requests

- For certain types of requests (e.g., POST, PUT, DELETE, or requests with custom headers), the browser sends an HTTP OPTIONS request to the server before the actual request. This is called a preflight request.
- The server responds with headers that indicate whether the actual request is allowed. If it is allowed, the browser proceeds with the request; otherwise, it is blocked.

CORS Headers

- Access-Control-Allow-Origin: Specifies which origin is allowed to access the resource. It can be set to a specific domain or * to allow any domain.

```
Access-Control-Allow-Origin: https://example.com
```

- Access-Control-Allow-Methods: Specifies which HTTP methods (e.g., GET, POST, PUT) are allowed for the resource.

```
Access-Control-Allow-Methods: GET, POST, PUT
```

- Access-Control-Allow-Headers: Specifies which headers can be used during the actual request.

```
Access-Control-Allow-Headers: Content-Type, Authorization
```

- Access-Control-Allow-Credentials: Indicates **whether the request can include user credentials** (e.g., cookies, HTTP authentication).

```
Access-Control-Allow-Credentials: true
```

- Access-Control-Max-Age: Indicates how long the results of the preflight request can be cached.

```
Access-Control-Max-Age: 86400
```

PUT vs. PATCH vs. POST (Idempotent Methods)

Idempotent: Yes. Calling PUT multiple times with the same data will always result in the same state on the server. Usage: Replaces the entire resource or creates it if it doesn't exist. PATCH:

Idempotent: No. Applying the same PATCH request multiple times could result in different states if the updates are cumulative. Usage: Partially updates a resource. POST:

Idempotent: No. Calling POST multiple times with the same data might result in different outcomes, such as creating multiple resources. Usage: Creates a new resource or submits data to be processed.

PATCH

- The PATCH method is used to apply partial updates to a resource. It allows you to update **just a specific field** of a resource.
- Body Convention: The body of a PATCH request typically includes only the fields that need to be updated, rather than the entire object.

```
{
  "email": "newemail@example.com"
}
```

PUT

- The PUT method is used to **update or create** a resource. It replaces the **entire resource** with the data provided in the body.
- The body of a PUT request typically contains the entire object that should replace the existing resource.

```
{
  "id": 123,
  "name": "John Doe",
  "email": "newemail@example.com"
}
```

RESTful API Maturity Model

- The RESTful API Maturity Model is a conceptual framework that categorizes REST APIs into different levels of maturity based on their adherence to REST principles.
- This model helps in understanding how well an API is designed according to RESTful principles and guides the improvement of APIs to make them more REST-compliant.

Level 0: The Swamp of POX

- Using plain XML or JSON over HTTP without leveraging the full capabilities of HTTP.
- At this level, the API is essentially a remote procedure call (RPC) style over HTTP. It typically uses a single endpoint (e.g., /api) and sends data in a single format, often XML or JSON, without leveraging HTTP's full capabilities.
- The API does not use HTTP methods (GET, POST, PUT, DELETE) in a meaningful way. All requests are sent to the same URL and are differentiated by **parameters** or **payload content**.

```
POST /api
<request>
    <operation>getUser</operation>
    <id>123</id>
</request>
```

Level 1: Resources

- Defining individual URIs for different resources.
- At this level, the API starts using distinct URIs (Uniform Resource Identifiers) to represent different resources. **Each resource has its own endpoint**. However, HTTP methods may still not be used

correctly, and the API may still behave like a traditional RPC system, with methods often hardcoded into the URL.

```
GET /users/123
```

Level 2: HTTP Verbs

- Making proper use of HTTP methods (GET, POST, PUT, DELETE, PATCH, OPTIONS).
- This level fully embraces the HTTP protocol, using methods to indicate the desired action (e.g., GET for retrieval, POST for creation).
- It also uses standard HTTP status codes to indicate the outcome of operations (e.g., 200 OK, 201 Created, 404 Not Found).

```
GET /users/123    // Retrieves user with ID 123
POST /users       // Creates a new user
PUT /users/123    // Updates user with ID 123
DELETE /users/123 // Deletes user with ID 123
```

Level 3: Hypermedia Controls (HATEOAS)

- Embedding links and actions within resources to guide the client through the API.
- This is the highest level of RESTful maturity, where the API not only uses resources and HTTP methods effectively but also provides **Hypermedia as the ### Engine of Application State (HATEOAS).**
- The API responses include hypermedia links that guide the client on what actions can be performed next and how to interact with other resources.
- At this level, clients can dynamically navigate the API using these hypermedia links without needing hardcoded knowledge of the API structure.

```
{
    "id": 123,
    "name": "John Doe",
    "links": [
        {
            "rel": "self",
            "href": "/users/123"
        },
        {
            "rel": "orders",
            "href": "/users/123/orders"
        },
        {
            "rel": "delete",
            "href": "/users/123",
            "method": "DELETE"
        }
```

```
    ]
}
```

Spring Core : IOC Vs Dependency Injection Vs @Autowire

Inversion of Control (IoC):

The framework manages the lifecycle of objects and their dependencies. IoC is a design principle in which the control of object creation and dependency management is inverted from the application code to a container or framework. In Spring, IoC is achieved through Dependency Injection (DI), where the framework injects the required dependencies into objects at runtime.

- allows loose coupling between components
- promotes easier testing and maintenance.

Spring's IoC container

Spring's IoC container is responsible for managing objects and their dependencies based on configuration.

Dependency Injection (DI):

Dependency Injection is a specific pattern within IoC where the dependencies of a component are "injected" into it rather than the component creating or managing its dependencies itself. There are several forms of dependency injection: constructor injection, setter injection, and field injection. Spring primarily uses constructor and setter injection.

```
// Service interface
public interface GreetingService {
    String greet();
}

// Service implementation
public class GreetingServiceImpl implements GreetingService {
    @Override
    public String greet() {
        return "Hello, world!";
    }
}

// Main application class using IoC
public class MyApp {
    public static void main(String[] args) {
        // Create the Spring IoC container
        ApplicationContext context = new
ClassPathXmlApplicationContext("applicationContext.xml");
```

```
        // Retrieve the bean from the container
        GreetingService service = context.getBean("greetingService",
GreetingService.class);

        // Use the bean
        String message = service.greet();
        System.out.println(message);
    }
}
```

```
<!-- applicationContext.xml -->
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans

http://www.springframework.org/schema/beans/spring-beans.xsd">

    <!-- Define a bean for GreetingService -->
    <bean id="greetingService" class="com.example.GreetingServiceImpl"/>

</beans>
```

Code Snippet Explanation

We're retrieving the GreetingService bean from the container using context.getBean("greetingService", GreetingService.class);. The container handles the instantiation of GreetingServiceImpl and injects it into the GreetingService reference in MyApp. This demonstrates dependency injection because the GreetingServiceImpl dependency is injected into the MyApp class.

@Autowired Annotation:

@Autowired is a Spring annotation used for automatic dependency injection. When Spring encounters @Autowired annotation, it looks for a bean of the corresponding type and injects it into the annotated field, method, or constructor. It simplifies the configuration by eliminating the need for explicit configuration of dependencies in XML or Java-based configuration.

```
@Component
public class MyComponent {

    private final MyService myService;

    @Autowired
    public MyComponent(MyService myService) {
        this.myService = myService;
    }

    // or for field injection
```

```
    @Autowired
    private MyRepository myRepository;

    // or for setter injection
    @Autowired
    public void setMyRepository(MyRepository myRepository) {
        this.myRepository = myRepository;
    }
}
```

Spring Configuration Annotations

@Component

- Used to declare a bean.

@Service

- Specialization of @Component.
- Additional Benefits: It can be used by Spring for additional processing
- Indicates that an annotated class is a "service".
- Used to denote business service layer.

@Repository

- Specialization of @Component.
- Indicates that an annotated class is a "repository".
- Used to indicate that the class provides the mechanism for storage, retrieval, search, update, and delete operation on objects.

Understanding of ORM

Entity Mapping

Mapping Java objects to database tables. Using Java Persistence API (JPA), involves defining how Java objects (entities) map to database tables. It allows for the `automatic and transparent` persistence of Java objects into database tables and retrieval from database tables into Java objects. Example:

```
@Entity
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    // getters and setters
}
```

Basic Annotations

- `@Entity`: Specifies that the class is an entity.
- `@Table`: Specifies the table name (optional if it matches the class name).
- `@Id`: Specifies the primary key.
- `@GeneratedValue`: Specifies the generation strategy for the primary key.
  - `IDENTITY`: Relies on the database's auto-increment feature.
  - `SEQUENCE`: Uses a database sequence object.
  - `TABLE`: Uses a separate table to generate primary keys.
  - `AUTO`: The persistence provider automatically chooses the appropriate strategy.
- `@Column`: Specifies the column details (optional if the field name matches the column name).

Association Mapping

Mapping relationships between entities (e.g., OneToOne, OneToMany, ManyToMany).

One-to-One Association

A one-to-one relationship is where one entity is related to exactly one other entity.

```
@Entity
public class User {

    @OneToOne
    @JoinColumn(name = "profile_id")
    private Profile profile;
}
```

One-to-Many Association and Many-to-One Association

- A one-to-many relationship is where one entity is related to multiple other entities.
- A many-to-one relationship is where multiple entities are related to one entity.

```
@Entity
public class User {

    @OneToMany(mappedBy = "user")
    private List<Order> orders;

}

@Entity
public class Order {

    @ManyToOne
    @JoinColumn(name = "user_id")
    private User user;

}
```

Many-to-Many Association

A many-to-many relationship is where multiple entities are related to multiple other entities.

```
@Entity
public class User {

    @ManyToMany
    @JoinTable(
        name = "user_role",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles;

}

@Entity
public class Role {

    @ManyToMany(mappedBy = "roles")
    private Set<User> users;
}
```

JPQL (Java Persistence Query Language)

A query language used to make queries against entities stored in a relational database. JPQL is similar to SQL but operates on the entity objects rather than directly on database tables.

```
List<User> users = entityManager.createQuery("SELECT u FROM User u",
User.class).getResultList();
```

JPA (Java Persistence API)

A specification for accessing, persisting, and managing data between Java objects and relational databases. JPA is a specification that defines a set of interfaces and annotations for managing relational data in Java applications. It provides a standardized way to interact with databases using Java objects, allowing developers to perform CRUD (Create, Read, Update, Delete) operations and more complex queries without writing SQL directly.

Key Components of JPA Entity: A lightweight, persistent domain object. Entities are managed by the entity manager in JPA.

JPA vs. Hibernate

JPA: A specification for persistence in Java, providing an API for managing relational data.

- Hibernate: A specific implementation of the JPA specification, providing additional features beyond the JPA standard.
  - offers more control and additional features beyond the JPA standard.
  - provides a built-in second-level cache.
  - allows for more complex SQL queries and native SQL support.
  - allows for custom interceptors and event listeners.

When using @Repository with a JpaRepository

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

you are using Spring Data JPA, which is a part of the Spring framework that makes it easier to implement JPA-based repositories. Spring Data JPA provides a repository abstraction over the JPA specification and uses a JPA provider (like Hibernate) under the hood.

Fetching Strategies

Eager Fetching

With eager fetching, related entities are fetched immediately with the main entity.

- Associated entities are loaded immediately with the primary entity.
- Pitfall: Can lead to performance issues due to loading unnecessary data.

```
@Entity
public class User {

    // When a User is fetched, all associated Order entities are also
fetched immediately.

    @OneToMany(fetch = FetchType.EAGER, mappedBy = "user")
    private List<Order> orders;

}
```

Lazy Fetching

With lazy fetching, related entities are fetched only when they are accessed for the first time.

- Associated entities are loaded on-demand when accessed.
- Pitfall: Can cause N+1 problem

```
@Entity
public class User {

    //When a User is fetched, associated Order entities are not fetched
until user.getOrders() is called.

    @OneToMany(fetch = FetchType.LAZY, mappedBy = "user")
    private List<Order> orders;

}
```

Note: Lazy fetching can lead to the "N+1 Selects" problem, where a large number of individual select statements are executed, which can be inefficient.

Spring Boot's Embedded Web Server (Spring Boot vs. Spring Framework)

- Spring Framework: Traditionally, when using the Spring Framework, you would need to manually deploy your web application to an external servlet container like Apache Tomcat, Jetty, or Undertow. This means packaging your application as a WAR file and deploying it to a server.
- Spring Boot: Spring Boot simplifies the process by providing **an embedded web server**. With Spring Boot, you can package your application as a JAR file that includes an embedded server. This allows you to run your application by simply executing the JAR file, eliminating the need for an external container.

Feature	Spring Framework	Spring Boot
Purpose	Comprehensive framework for enterprise Java development.	Simplifies Spring applications with convention-over-configuration.
Configuration	Requires extensive manual configuration (XML/Java-based).	Provides auto-configuration and starter dependencies.
Project Setup	More complex with multiple configurations and dependencies.	Simplified with starter POMs and minimal configuration.
Deployment	Typically deployed as WAR files on external servers.	Typically packaged as executable JARs with embedded servers.
Embedded Server	Not typically used; relies on external servers.	Comes with embedded servers like Tomcat, Jetty, or Undertow.
Development Speed	Slower due to extensive configuration.	Faster due to auto-configuration and embedded servers.
Flexibility	Offers fine-grained control and maximum flexibility.	Focuses on simplicity and ease of use, with less emphasis on customization.
Ecosystem	Rich ecosystem with many Spring projects (e.g., Spring Security, Spring Data).	Integrates Spring projects easily with starter dependencies.

Feature	Spring Framework	Spring Boot
Testing	Requires more setup for testing configurations and environments.	Provides built-in testing support with minimal configuration.
Initial Setup	Manual creation of configuration files, dependencies, and server setup.	Initial setup with `spring-boot-starter` dependencies and embedded server.
Learning Curve	Steeper due to the need to understand various configurations and components.	Gentler learning curve with opinionated defaults and less boilerplate code.
Microservices Support	Can be used for microservices but requires more configuration.	Built with microservices in mind, with features like Spring Boot Actuator.
Actuator	Not included by default; requires additional configuration.	Comes with Spring Boot Actuator for monitoring and management out of the box.

Advice Controller

Controller Advice in Spring is a way to handle exceptions and apply global behaviors to controllers. It is used with the @ControllerAdvice annotation to create a centralized error-handling mechanism for controllers.

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(value = {Exception.class})
    public ResponseEntity<Object> handleException(Exception e) {
        return new ResponseEntity<>(e.getMessage(),
HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

- In this example, GlobalExceptionHandler is a controller advice that handles all exceptions thrown by any controller in the application. It provides a uniform response in case of errors, improving code maintainability and readability.

ApplicationContext (Spring)

- The ApplicationContext is the central **interface** to the Spring IoC (Inversion of Control) container. It is responsible for **instantiating**, **configuring**, and **assembling** the beans in a Spring application.
- Relation to BeanFactory: ApplicationContext is an **extension** of the BeanFactory interface. While BeanFactory provides the basic mechanisms for **dependency injection and bean management**, ApplicationContext builds on top of that to provide additional enterprise-level functionality.
- Event propagation.

- Declarative mechanisms for creating a bean : allows you to define beans and their dependencies without needing to write complex code, through xml or annotations.

Spring Container

Spring Container is responsible for managing the lifecycle and configuration of application beans. It is essentially the core of the Spring Framework. The container creates beans, **wires them together, configures them, and manages their lifecycle**.

- The two primary container types in Spring are:
  - BeanFactory: The simplest container providing basic dependency injection.
  - ApplicationContext: A more advanced container that adds additional features, such as event propagation, declarative mechanisms, etc.

Bean Lifecycle

Bean Lifecycle in Spring refers to the different stages a bean goes through from creation to destruction within the Spring container.

- The typical lifecycle stages are:
  - Instantiation: The bean is instantiated by the container.
  - Populate Properties: The container injects dependencies into the bean.
  - ApplicationContextAware Interface: If the bean implements ApplicationContextAware, the container passes the ApplicationContext.
  - Custom Initialization Method: The container calls the `init-method` or the `@PostConstruct` annotated method.
  - Bean Ready to Use: The bean is fully initialized and ready for use.
  - Custom Destruction Method: The container calls the `destroy-method` or the `@PreDestroy` annotated method before the bean is destroyed.

Bean Scopes

Bean Scopes define the lifecycle of a bean within the container. The most commonly used scopes are:

- Singleton: The default scope, where a **single instance** of the bean is created and shared across the entire application context.
- Prototype: A new instance of the bean is created every time it is requested.
- Request: A new instance of the bean is created for each HTTP request (web-aware Spring applications).
- Session: A new instance of the bean is created for each HTTP session (web-aware Spring applications).

Request Filter/Interceptor in Spring Boot

Request Filter

Filters in Spring Boot are used to perform filtering tasks such as logging, authentication, or modifying requests/responses before they reach the controller.

```
@Component
public class MyFilter implements Filter {
    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
FilterChain chain)
            throws IOException, ServletException {
        // Pre-processing logic
        chain.doFilter(request, response); // Continue to the next filter
or controller
        // Post-processing logic
    }
}
```

Interceptor

- Interceptors in Spring Boot are used to intercept HTTP requests and responses before they reach the controller and after they are processed by the controller, but before the view is rendered.

```
// Implement the HandlerInterceptor interface and override methods like
preHandle, postHandle, and afterCompletion.
@Component
public class MyInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request,
HttpServletResponse response, Object handler)
            throws Exception {
        // Pre-processing logic
        return true; // Continue with the request
    }

    @Override
    public void postHandle(HttpServletRequest request, HttpServletResponse
response, Object handler,
            ModelAndView modelAndView) throws Exception {
        // Post-processing logic
    }

    @Override
    public void afterCompletion(HttpServletRequest request,
HttpServletResponse response, Object handler,
            Exception ex) throws Exception {
        // Cleanup logic
    }
}
```

Advanced Memory Management

Memory Leaks

A Memory Leak is a situation where there are objects present in the heap that are no longer used, but the garbage collector is unable to remove them from memory, and therefore, they're unnecessarily maintained.

- A memory leak is bad because it blocks memory resources and degrades system performance over time.
- Symptoms of a Memory Leak
  - Severe performance degradation when the application is continuously running for a long time
  - OutOfMemoryError heap error in the application
  - Spontaneous and strange application crashes
  - The application is occasionally running out of connection objects.

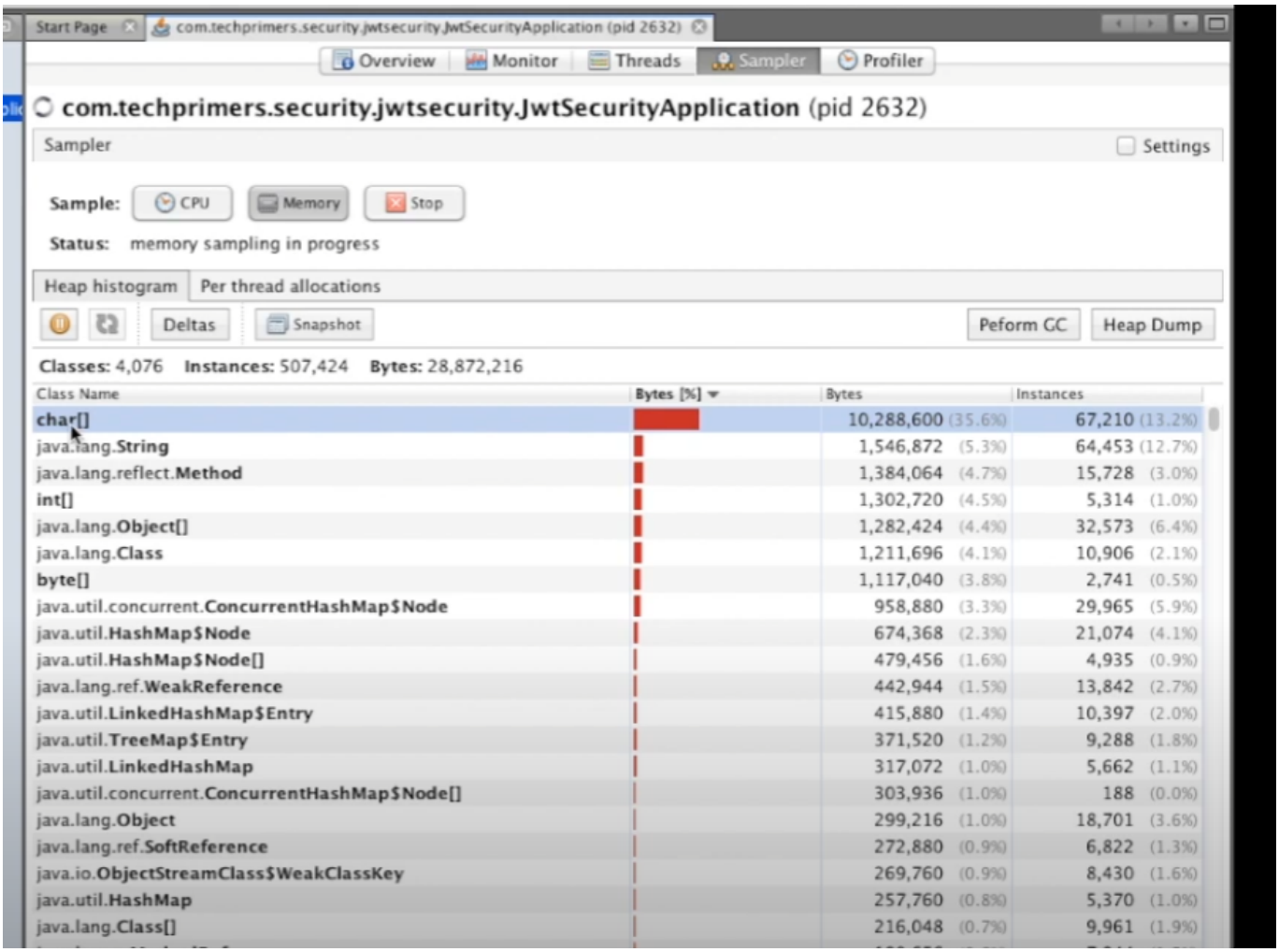
check link here

Memory Leak stops when java application stops

When a Java application stops, the Java Virtual Machine (JVM) that was running the application also terminates. This means that all the memory allocated by the JVM, including the heap memory, is released back to the operating system. Consequently, any memory leaks caused by static variables or other issues within the JVM will not persist after the application or the JVM terminates.

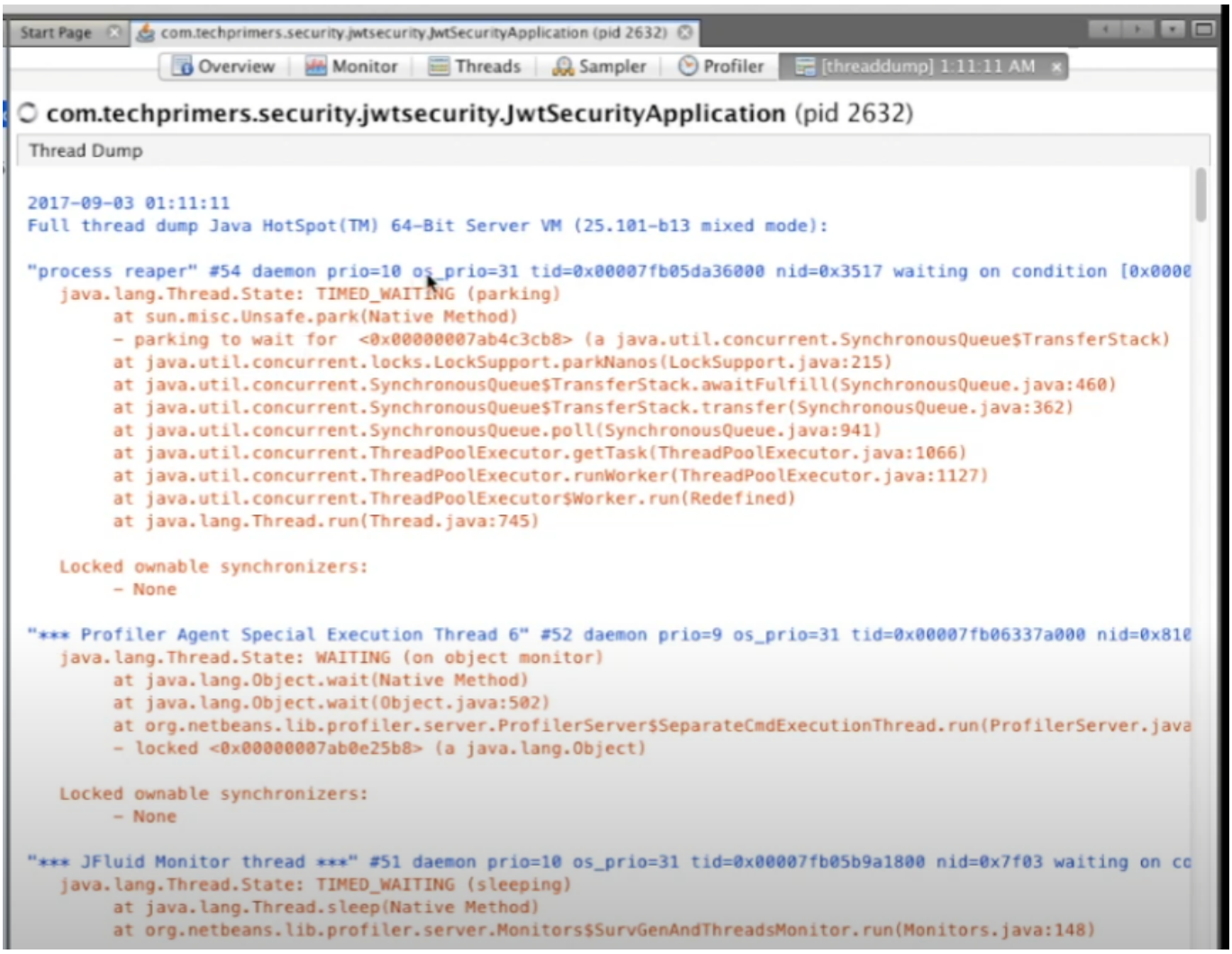
Heap Dump

A heap dump is a snapshot of the memory of a Java process at a given point in time. It contains information about the Java objects and classes in the heap. A heap dump is typically used for debugging memory issues, such as memory leaks.



Thread Dump

A thread dump is a snapshot of all the threads running in a Java process at a given point in time. It contains information about the threads, their states, and stack traces. Thread dumps are used to diagnose issues like deadlocks, performance bottlenecks, and thread contention.



Profiling

Profiling is the process of analyzing a program to identify performance bottlenecks, memory usage, and other runtime characteristics. Profiling tools can provide detailed information about CPU usage, memory allocation, and method execution times.

- Java profiling tools help developers monitor and analyze the performance of Java applications. These tools provide insights into CPU usage, memory consumption, thread activity, garbage collection, and more. They are essential for identifying bottlenecks and optimizing applications.
- Popular profiling tools:
    - **VisualVM**: A free profiling tool that provides insights into memory usage, CPU performance, and more.

Garbage Collection

Garbage collection (GC) is the process by which the Java Virtual Machine (JVM) automatically reclaims memory by identifying and disposing of objects that are no longer reachable from any live threads or static references.

OutOfMemoryError Types

Java Heap Space Error

Occurs when the JVM runs out of heap memory, which is used for dynamic memory (runtime) allocation.

Example:

```
public class HeapSpaceError {
    public static void main(String[] args) {
        List<int[]> list = new ArrayList<>();
        while (true) {
            list.add(new int[1024 * 1024]);
            // Since the loop is infinite, this process never stops,
leading to a continuous increase in memory consumption.  Eventually, the
heap memory is exhausted, and the JVM cannot allocate any more memory.
This results in an OutOfMemoryError. The program will terminate with a
java.lang.OutOfMemoryError: Java heap space error once the heap memory is
fully utilized.
        }
    }
}
```

Solution:

- Increase the heap size using JVM options
- Optimize memory usage and object creation.
- Use memory profilers to detect memory leaks.

Java PermGen Space Error

Occurs when the JVM runs out of memory in the Permanent Generation space, which is used to store metadata such as class definitions.

- PermGen space is a part of the heap that holds metadata such as class definitions and method objects
- PermGen space has been removed in Java 8 and replaced by Metaspace.
- Each new class definition consumes PermGen space
- In Java 8, the PermGen space is replaced by Metaspace, which dynamically resizes and is allocated in native memory, reducing the likelihood of such errors.
- Native memory refers to the memory directly managed by the operating system, outside the Java heap. Yet limited by the available system memory.
- Metaspace area as needed, limited by the available system memory.
- When classes are no longer needed, their metadata can be removed by garbage collector, freeing up Metaspace.

```
import javassist.ClassPool;

public class PermGenSpaceError {
    public static void main(String[] args) throws Exception {
        // Creates a default ClassPool instance, which is used to create
and manipulate classes
        ClassPool classPool = ClassPool.getDefault();
```

```
        for (int i = 0; ; i++) {
            // continuously creates new classes with unique names (Class0,
Class1, etc.) using the ClassPool and loads them into the JVM.
            Class<?> clazz = classPool.makeClass("Class" + i).toClass();
        }
    }
}
```

Solution:

- Increase PermGen size.
- Reduce the number of classes loaded.
- Migrate to Java 8 or later, where PermGen is replaced by Metaspace.

Testing

Difference between Unit Testing and Integration Testing

- Unit Testing: Testing individual components.
- Integration Testing: Testing the interaction between components.

Using Mockito

- Mockito is a mocking framework used for creating mock objects in unit tests.
- Mockito is typically used in conjunction with JUnit for writing unit tests.
- It is primarily used for `stubbing` and `verifying` interactions between objects during unit testing, rather than integration testing within the Spring environment.
- Purpose:
    - Isolation: Allows testing of a specific part of the system in isolation from its dependencies.
    - Control: Provides control over the behavior of dependencies, making it easier to test **edge cases** and **error conditions**.
    - Efficiency: Reduces the complexity and execution time of tests by avoiding the need to set up and execute the actual dependent components
- JMockit: Another framework that supports stubbing and mocking in Java.

@Mock, @Spy and @InjectMocks + Stubbing (when().thenReturn())

- The @Mock annotation is used to create and inject mock objects.
- The @Spy annotation is used to create partial mocks. It allows calling real methods unless they are stubbed.
- @InjectMocks creates an instance of the class and injects the mocks that are created with the @Mock (or @Spy) annotations into this instance.
- The when().thenReturn() method is used to define the behavior of mocked methods.

```

```

```
public class FullExample {

    @Mock
    private MyService myService;

    @Spy
    private MyHelper myHelper = new MyHelper();

    @InjectMocks
    private MyController myController;

    @Before
    public void init() {
        // you can also use spring annottaion to run this test class using
Mockito
        MockitoAnnotations.openMocks(this);
    }

    @Test
    public void testAll() {
        // Stubbing
        when(myService.someMethod()).thenReturn("stubbed result");

        // Verify behavior through controller
        String result = myController.callService();
        assertEquals("stubbed result", result);

        // Verify real method call on spy
        verify(myHelper).assist();
    }
}
```

```
class MyService {
    public String someMethod() {
        return "real result";
    }
}

class MyHelper {
    public void assist() {
        // Some assistance logic
    }
}

class MyController {
    private MyService myService;
    private MyHelper myHelper;

    public String callService() {
        myHelper.assist();
        return myService.someMethod();
```

```
    }
}
```

JUnit

- JUnit is a popular unit testing framework primarily used for writing and running unit tests in Java.
- It provides annotations like @Test, @Before, @After, etc., for writing test cases and setup/teardown methods.

```
public class ExampleUnitTest {

    private Calculator calculator;

    // This annotation marks a method that should be run before each test
method to set up any necessary preconditions (like initializing objects).
    @Before
    public void setUp() {
        // Initialize any resources needed before each test method
        calculator = new Calculator();
        System.out.println("Setup executed.");
    }

    // This annotation marks a method that should be run after each test
method to clean up resources used by the test (like releasing memory or
closing files).
    @After
    public void tearDown() {
        // Clean up any resources after each test method
        calculator = null;
        System.out.println("Teardown executed.");
    }

    @Test
    public void testAddition() {
        // Test case for addition
        int result = calculator.add(3, 5);
        // This method is from JUnit's Assert class and checks that the
actual value matches the expected value. If they don't match, it throws an
assertion error.
        assertEquals(8, result);
    }


}
```

Spring TextContext Framework

The Spring TestContext Framework is designed to support integration testing with the Spring Framework.

- It builds upon the core features of the JUnit and TestNG frameworks and adds functionality specific to Spring, such as support for loading Spring ApplicationContexts, transaction management, and dependency injection during testing.
- It allows for easy setup and teardown of application contexts, transaction management, and other Spring-specific testing requirements.
- Purpose(why use it):
    - Integration Testing: It facilitates testing components in a Spring context, allowing integration tests to run with dependency injection and other Spring features.
    - Transaction Management: It supports transactional testing, ensuring proper rollback of transactions after tests.
    - Test Execution: It provides lifecycle callbacks (@Before, @After) and setup (@Autowired, @MockBean) for testing components in a Spring environment efficiently.
- @MockBean is used in Spring testing to mock beans when running integration tests. It replaces a bean of the same type in the application context with a mock version.

```
@RunWith(SpringRunner.class)
@ContextConfiguration(classes = { AppConfig.class })
@Transactional
public class MyServiceIntegrationTest {

    @Autowired
    private MyService myService;

    @Test
    public void testSomeMethod() {
        // Test logic using myService
    }
}
```

Example of Transaction Management in Spring

- Transactions manage resource locks and ensure data consistency, rolling back changes on failure.

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    // Marks methods that should be executed within a transaction context.
    @Transactional
    public void updateUser(User user) {
        // Perform business logic
        userRepository.save(user);
        // Additional business logic
    }

    // readOnly = true: Indicates that the method only reads data and does
```

```
not modify the database. This can provide performance benefits.
    @Transactional(readOnly = true)
    public User getUserById(Long id) {
        // Retrieve user by ID
        return userRepository.findById(id).orElse(null);
    }

    // Other service methods can be added here
}
```

Hamcrest vs Mockito

- Hamcrest and Mockito are both used in Java unit testing, but they serve different purposes:
- Hamcrest is a framework for **writing matchers**, which are conditions that an object must satisfy. It is often used with JUnit to assert that a particular object meets specific criteria. For example, you can use Hamcrest to assert that a list contains certain elements or that a string starts with a particular prefix.

Example:

```
@Test
public void testHamcrest() {
    List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
    assertThat(names, hasItem("Alice"));
    assertThat("Bob", startsWith("B"));
}
```

- Mockito is a mocking framework that allows you to create mock objects for testing. It is used to simulate the behavior of real objects in a controlled way. This is particularly useful when you want to **isolate** the unit under test from its dependencies.

```
import static org.mockito.Mockito.*;

@Test
public void testMockito() {
    List<String> mockedList = mock(List.class);
    when(mockedList.get(0)).thenReturn("Alice");

    assertEquals("Alice", mockedList.get(0));
}
```

@Rule vs @ExtendWith and @RunWith

@Rule

@Rule is an annotation used in JUnit 4 to define a method or field that will be executed before or after a test method. It allows you to **apply specific behaviors** (like timeouts or exception handling) across multiple test

methods **without duplicating code**.

```
public class MyTest {
    @Rule
    public Timeout globalTimeout = Timeout.seconds(10);

    @Test
    public void testSomething() {
        // This test will fail if it runs longer than 10 seconds
    }
}
```

@RunWith

In JUnit 4, the @RunWith annotation is used to specify a custom test runner. This is required when you want to use a different test runner than the default one provided by JUnit 4. When using Mockito, the `MockitoJUnitRunner` class is often specified with @RunWith to enable the Mockito framework.

```
@RunWith(MockitoJUnitRunner.class)
public class MyTest {

    @Mock
    private MyService myService;

    @Test
    public void testSomething() {
        // Test code here using the mock
    }
}
```

@ExtendWith

@ExtendWith is an annotation in JUnit 5 that replaces @RunWith and allows you to register extensions, which are more powerful and flexible than rules in JUnit 4.

Extensions can intercept lifecycle events like test initialization, method execution, and result processing. The MockitoExtension is used with @ExtendWith to integrate Mockito with JUnit 5 tests.

```
@ExtendWith(MockitoExtension.class)
public class MyTest {

    @Mock
    private MyService myService;

    @InjectMocks
    private MyController myController;
```

```
    @Test
    public void testSomething() {
        // Test code here using the mock
    }
}
```

Differences Summary

- @RunWith specifies a test runner that replaces the default JUnit runner.
- @ExtendWith registers an extension that adds additional behavior to the test class.
- @ExtendWith in JUnit 5 allows combining **multiple** extensions, offering more flexibility.
- @RunWith in JUnit 4 is limited to **one** test runner, meaning you can't combine multiple runners.

Reflection

Reflection is a powerful feature in Java that allows you to **inspect and manipulate classes, fields, methods, and constructors at runtime**. It's often used in frameworks and libraries that need to dynamically access or modify class behavior without knowing their details at compile time.

```
import java.lang.reflect.Method;

public class ReflectionExample {
    public static void main(String[] args) throws Exception {
        Class<?> clazz = Class.forName("java.util.ArrayList");
        Method method = clazz.getMethod("size");
        Object instance = clazz.newInstance();
        int size = (int) method.invoke(instance);
        System.out.println("Size: " + size);
    }
}
```

Mockito Doesn't Allow Stubbing of Final, Private, or Static Methods

- Final Methods: Since final methods cannot be overridden, Mockito cannot intercept calls to them for stubbing.
- Private Methods: Private methods are not accessible outside their defining class, so Mockito cannot stub or mock them directly.
- Static Methods: Static methods belong to the class, not to instances, making them difficult to mock in a traditional object-oriented way.

ArgumentCaptor

- ArgumentCaptor is a feature in Mockito that allows you to capture arguments passed to mock methods for further assertions or checks.

```
@Test
public void testArgumentCaptor() {
    List<String> mockedList = mock(List.class);

    ArgumentCaptor<String> argumentCaptor =
ArgumentCaptor.forClass(String.class);

    mockedList.add("Hello");
    verify(mockedList).add(argumentCaptor.capture());

    assertEquals("Hello", argumentCaptor.getValue());
}
```

- In this example, ArgumentCaptor captures the argument passed to the add method and allows you to assert that it is as expected.

Parallel Programming

In Java, parallel programming involves executing tasks concurrently to improve performance by utilizing multiple threads. Concurrent collections support these operations by providing thread-safe data structures that can handle concurrent reads and writes efficiently.

- Thread-safe: This means multiple threads can read and modify the map concurrently without causing data corruption or inconsistency.
- Normal data structures like HashMap, ArrayList, and Queue are not thread-safe by default. If accessed concurrently, they can lead to data corruption or inconsistent state.
- Concurrent collections ensure thread safety internally, typically by using locking mechanisms or copy-on-write strategies to prevent concurrent modification issues.

Concurrent Collections

ConcurrentHashMap

A thread-safe implementation of the standard HashMap. It allows concurrent access to different parts of the map, ensuring that operations like put and get are safe without requiring external synchronization.

```
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
map.put("key1", 1);
map.put("key2", 2);
int value = map.get("key1");
```

ConcurrentLinkedQueue

A thread-safe implementation of a FIFO (First-In-First-Out) queue. It allows multiple threads to insert, remove, and peek elements concurrently without blocking.

- ConcurrentLinkedQueue are unbounded, meaning they can grow dynamically without a fixed size limit.
-

```
ConcurrentLinkedQueue<String> queue = new ConcurrentLinkedQueue<>();
queue.add("element1");
queue.add("element2");
String element = queue.poll(); // Retrieves and removes the head
```

CopyOnWriteArrayList

A thread-safe variant of ArrayList where all mutative operations (e.g., add, set, remove) are implemented by making a fresh copy of the underlying array.

```
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
list.add("element1");
list.add("element2");
String element = list.get(0);
```

Difference between HashMap and ConcurrentHashMap

- HashMap is non-Synchronized in nature i.e. HashMap is not Thread-safe whereas ConcurrentHashMap is Thread-safe in nature.
- HashMap performance is relatively high because it is non-synchronized in nature and any number of threads can perform simultaneously. But ConcurrentHashMap performance is low sometimes because sometimes Threads are required to wait on ConcurrentHashMap.
- While one thread is Iterating the HashMap object, if other thread try to add/modify the contents of Object then we will get Run-time exception saying ConcurrentModificationException.Whereas In ConcurrentHashMap we wont get any exception while performing any modification at the time of Iteration.

Example: Issues with Normal HashMap in Multi-threaded Environment

Consider a scenario where multiple threads are putting data into a normal HashMap concurrently:

```
// HashMap drawbacks
import java.util.HashMap;

class HashMapDemo extends Thread
{
    static HashMap<Integer,String> l=new HashMap<Integer,String>();

    public void run()
```

```
    {


        try
        {
            Thread.sleep(1000);
            // Child thread trying to add
            // new element in the object
            l.put(103,"D");
        }
        catch(InterruptedException e)
        {
            System.out.println("Child Thread going to add element");
        }
    }

    public static void main(String[] args) throws InterruptedException
    {
        l.put(100,"A");
        l.put(101,"B");
        l.put(102,"C");
        HashMapDemo t=new HashMapDemo();
        t.start();

        for (Object o : l.entrySet())
        {
            Object s=o;
            System.out.println(s);
            Thread.sleep(1000);
        }
        System.out.println(l);
    }
}
// Output:

// 100=A
// Exception in thread "main" java.util.ConcurrentModificationException
```

if we changed the hashmap into concurrentHashMap

```
static ConcurrentHashMap<Integer,String> l =
                        new ConcurrentHashMap<Integer,String>();
// Output:

// 100=A
// 101=B
// 102=C
// 103=D
// {100=A, 101=B, 102=C, 103=D}
```

CompletableFutures

CompletableFuture is a class in Java (introduced in java 8) that represents a future result of an asynchronous computation. It provides a more powerful and flexible way to handle asynchronous tasks compared to the traditional Future interface.

- Limitations of Future Class: Future class lacks features such as manual completion(Requires explicit handling to cancel a running task), callbacks(after completion), cchaining or combining multiple asynchronous computations, and exception handling, making it less efficient for complex asynchronous workflows.

Key features of include:

- Non-blocking: CompletableFutures allow non-blocking operations, meaning you can set up callbacks to run upon task completion without blocking the main thread.
- Chaining and Composition: Methods like thenApply, thenAccept, thenCombine, and allOf allow for chaining and combining multiple asynchronous operations.
- Exception Handling: CompletableFuture provides methods like exceptionally and handle to handle exceptions that may occur during the asynchronous computation.
- Manual Completion: You can manually complete a CompletableFuture using methods like complete or completeExceptionally.
- Support for Both Synchronous and Asynchronous Tasks: You can execute tasks either synchronously or asynchronously using methods like runAsync and supplyAsync.

CompletableFuture Methods:

1. `SupplyAsync`: Executes a task asynchronously that supplies a result.

```
CompletableFuture.supplyAsync(() -> "Result");
```

2. `get()`: Blocks until the future is complete and returns the result.

```
CompletableFuture<String> future = CompletableFuture.supplyAsync(() ->
"Hello, World!");
try {
    // This will block until the future is complete
    String result = future.get();
    System.out.println(result); // Output: Hello, World!
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
```

3. `complete(T value)`: Manually completes the future with the specified value.

```
CompletableFuture<String> future = new CompletableFuture<>();
future.complete("Manually Completed");
```

```
try {
    System.out.println(future.get()); // Output: Manually Completed
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
```

4. `RunAsync`: Executes a task asynchronously that doesn't return a result.

```
CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
    System.out.println("Running asynchronously");
});
```

5. `supplyAsync(Supplier<T> supplier)`: Creates a CompletableFuture instance out of a Supplier. Executes a task asynchronously and return a result.

```
CompletableFuture<String> future = CompletableFuture.supplyAsync(() ->
"Supplied Value");
try {
    System.out.println(future.get()); // Output: Supplied Value
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
```

Methods in CompletableFuture for Chaining and Combining

6. `thenApply`: Transforms the result of a CompletableFuture.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() ->
10)
    .thenApply(result -> result * 2);
try {
    System.out.println(future.get()); // Output: 20
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
```

7. `thenAccept`: Consumes the result of a CompletableFuture. Accepts the result of the CompletableFuture and returns void.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() ->
10)
    .thenAccept(result -> System.out.println("Result: " + result)); //
Output: Result: 10
```

8. `thenRun`: Executes a Runnable after the CompletableFuture is complete.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() ->
10)
    .thenRun(() -> System.out.println("Task completed")); // Output: Task
completed
```

9. `thenApplyAsync(Function<? super T, ? extends U> fn)`: Asynchronously transforms the result of the CompletableFuture.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() ->
10)
    .thenApplyAsync(result -> result * 2);
try {
    System.out.println(future.get()); // Output: 20
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
```

10. `thenAcceptAsync(Consumer<? super T> action)` Asynchronously accepts the result of the CompletableFuture and returns void.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() ->
10)
    .thenAcceptAsync(result -> System.out.println("Result: " + result));
// Output: Result: 10
```

11. `thenRunAsync(Runnable action)`: Asynchronously executes a Runnable when the CompletableFuture is complete.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() ->
10)
    .thenRunAsync(() -> System.out.println("Task completed")); // Output:
Task completed
```

12. `thenCompose(Function<? super T, CompletionStage<U>> fn)`: Combines two futures where one future is dependent on the other.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() ->
10)
    .thenCompose(result -> CompletableFuture.supplyAsync(() -> result *
2));
try {
```

```
    System.out.println(future.get()); // Output: 20
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}```

13. `thenCombine(CompletionStage<? extends U> other, BiFunction<? super T, ? super U, ? extends V> fn)`
Combines two futures that run independently and executes a function after both are complete.
```java
CompletableFuture<Integer> future1 = CompletableFuture.supplyAsync(() -> 10);
CompletableFuture<Integer> future2 = CompletableFuture.supplyAsync(() -> 20);

CompletableFuture<Integer> combinedFuture = future1.thenCombine(future2, (result1, result2) -> result1 + result2);
try {
    System.out.println(combinedFuture.get()); // Output: 30
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
```

14. `thenAcceptBoth(CompletionStage<? extends U> other, BiConsumer<? super T, ? super U> action)` Executes a BiConsumer when both futures are complete.

```
CompletableFuture<Integer> future1 = CompletableFuture.supplyAsync(() -> 10);
CompletableFuture<Integer> future2 = CompletableFuture.supplyAsync(() -> 20);

CompletableFuture<Void> combinedFuture = future1.thenAcceptBoth(future2, (result1, result2) ->
    System.out.println("Results: " + result1 + " and " + result2)); // Output: Results: 10 and 20
```

15. `allOf(CompletionStage<?>... stages)` Returns a new CompletableFuture that completes when all of the given futures complete.

```
CompletableFuture<String> future1 = CompletableFuture.supplyAsync(() -> "A");
CompletableFuture<String> future2 = CompletableFuture.supplyAsync(() -> "B");

CompletableFuture<Void> allOfFuture = CompletableFuture.allOf(future1, future2);
allOfFuture.thenRun(() -> {
    try {
        System.out.println(future1.get() + future2.get()); // Output: AB
```
```

```
    } catch (Exception e) {
        e.printStackTrace();
    }
});
```

16. `anyOf(CompletionStage<?>... stages)`: Returns a new CompletableFuture that completes when any of the given futures completes.

```
CompletableFuture<String> future1 = CompletableFuture.supplyAsync(() -> {
    try {
        Thread.sleep(100);
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
    return "Future 1";
});
CompletableFuture<String> future2 = CompletableFuture.supplyAsync(() ->
"Future 2");

CompletableFuture<Object> anyOfFuture = CompletableFuture.anyOf(future1,
future2);
try {
    System.out.println(anyOfFuture.get()); // Output: Future 2 (because
future2 completes first)
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
```

17. `join()`

- Retrieves the result from a completed CompletableFuture.
- It is similar to the get method, but it throws an unchecked exception in case the Future does not complete normally.

```
CompletableFuture<String> future = CompletableFuture.supplyAsync(() ->
"Hello, World!");
String result = future.join();
System.out.println(result); // Output: Hello, World!
```

18. `exceptionally(Function<Throwable, ? extends T> fn)` Recovers from errors generated from the original future.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> {
    throw new RuntimeException("Exception occurred");
}).exceptionally(ex -> {
    System.out.println("Exception: " + ex.getMessage());
    return 0;
```

```
});
try {
    System.out.println(future.get()); // Output: Exception: Exception
occurred, 0
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
```

19. `handle(BiFunction<? super T, Throwable, ? extends T> fn)` Recovers from exceptions, called whether or not an exception occurs.

```
CompletableFuture<Integer> future = CompletableFuture.supplyAsync(() -> {
    throw new RuntimeException("Exception occurred");
}).handle((result, ex) -> {
    if (ex != null) {
        System.out.println("Exception: " + ex.getMessage());
        return 0;
    }
    return result;
});
try {
    System.out.println(future.get()); // Output: Exception: Exception
occurred, 0
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}
```

Parallel Streams

- Java Parallel Streams is a feature of Java 8 and higher, meant for utilizing multiple cores of the processor
- Java uses it to perform parallel operations on collections.
- The order of execution, however, is not under our control.
- Therefore, it is advisable to use parallel streams in cases where no matter what is the order of execution, the result is unaffected and the state of one element does not affect the other as well as the source of the data also remains unaffected.
- There are two ways we can create which are listed below and described later as follows:
    - Using parallel() method on a stream
    - Using parallelStream() on a Collection

```
    // Using parallel() method on a Stream
    public static void main(String[] args) throws IOException {

        // Creating a file object which points to a pre-existent ‘txt’
file in the system
        File fileName = new File("M:\\Documents\\Textfile.txt");

        // Creating a Stream that reads from the text file one line at a
```

```
time.
        Stream<String> text = Files.lines(fileName.toPath());

        // Creating parallel streams using parallel() method to print the
read file on the console.
        text.parallel().forEach(System.out::println);
        // The order of execution is different for each run
        text.close();
    }
```

reduce function

- The reduce function in Java's Stream API performs a reduction on the elements of the stream using an associative accumulation function and returns an Optional describing the reduced value.
- It allows you to aggregate elements of a stream into a single result.
- The function takes two forms:
    - `reduce(BinaryOperator<T> accumulator)`
    - `reduce(T identity, BinaryOperator<T> accumulator)`
- With identity and accumulator: This version takes an identity value and an accumulator. The identity value is the initial value for the reduction, and the accumulator function specifies how the elements are combined. The identity value is returned if the stream is empty.
- With accumulator only: This version doesn't use an identity value and returns an Optional. It reduces the stream using the provided accumulator function.

```
public class ReduceExample {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

        // Reduce with identity and accumulator
        int sum = numbers.stream()
                         .reduce(0, (a, b) -> a + b);

        System.out.println("Sum: " + sum); // Output: Sum: 15

        // Reduce with accumulator only
        Optional<Integer> sumA = numbers.stream()
                                        .reduce((a, b) -> a + b);

        sumA.ifPresent(s -> System.out.println("Sum: " + s)); // Output:
Sum: 15
    }
}
```

Splitting Source

- Parallel streams make use of the fork-join framework and its common pool of worker threads.
- The fork-join framework was added to java.util.concurrent in Java 7 to handle task management between multiple threads.
- The fork-join framework is in charge of splitting the source data between worker threads and handling callback on task completion.
- Key Features of `ForkJoinPool`:
    - Work-Stealing Algorithm: ForkJoinPool uses a work-stealing algorithm to balance the load among worker threads. Each worker thread has its own deque (double-ended queue) of tasks. If a worker thread finishes its tasks, it can "steal" tasks from other threads to stay busy.
    - Task Splitting: The pool is designed to split tasks into smaller sub-tasks, which can then be processed independently and in parallel.

```
List<Integer> listOfNumbers = Arrays.asList(1, 2, 3, 4);
int sum = listOfNumbers.parallelStream().reduce(5, Integer::sum);
assertThat(sum).isNotEqualTo(15);
```

- In a sequential stream, the result of this operation would be 15.
- But since the reduce operation is handled in parallel, the number five actually gets added up in every worker thread
- The actual result might differ depending on the number of threads used in the common fork-join pool.
- In order to fix this issue, the number five should be added outside of the parallel stream:

```
List<Integer> listOfNumbers = Arrays.asList(1, 2, 3, 4);
int sum = listOfNumbers.parallelStream().reduce(0, Integer::sum) + 5;
assertThat(sum).isEqualTo(15);
```

- Therefore, we need to be careful about which operations can be run in parallel.
-

Common Thread Pool

- The number of threads in the common pool is equal to (the number of processor cores -1).
- However, the API allows us to specify the number of threads it will use by passing a JVM parameter:

```
-D java.util.concurrent.ForkJoinPool.common.parallelism=4
```

- It’s important to remember that this is a global setting and that it will affect all parallel streams and any other fork-join tasks that use the common pool. We strongly suggest that this parameter is not modified unless we have a very good reason for doing so.

Performance Implications

Parallel processing may be beneficial to fully utilize multiple cores. But we also need to consider the overhead of managing multiple threads, memory locality, splitting the source and merging the results.

The Overhead

We'll run a benchmark on a sequential and parallel reduction operation:

```
IntStream.rangeClosed(1, 100).reduce(0, Integer::sum);
IntStream.rangeClosed(1, 100).parallel().reduce(0, Integer::sum);
```

On this simple sum reduction, converting a sequential stream into a parallel one resulted in worse performance:

```
Benchmark                                                    Mode  Cnt
Score        Error  Units
SplittingCosts.sourceSplittingIntStreamParallel              avgt   25
35476,283 ±     204,446  ns/op
SplittingCosts.sourceSplittingIntStreamSequential            avgt   25
68,274 ±       0,963  ns/op
```

The reason behind this is that sometimes the overhead of managing threads, sources and results is a more expensive operation than doing the actual work.

Splitting Costs

Splitting the data source evenly is a necessary cost to enable parallel execution, but some data sources split better than others.

- Using an ArrayList and a LinkedList:

```
private static final List<Integer> arrayListOfNumbers = new ArrayList<>();
private static final List<Integer> linkedListOfNumbers = new LinkedList<>
();

static {
    IntStream.rangeClosed(1, 1_000_000).forEach(i -> {
        arrayListOfNumbers.add(i);
        linkedListOfNumbers.add(i);
    });
}

arrayListOfNumbers.stream().reduce(0, Integer::sum)
arrayListOfNumbers.parallelStream().reduce(0, Integer::sum);
linkedListOfNumbers.stream().reduce(0, Integer::sum);
linkedListOfNumbers.parallelStream().reduce(0, Integer::sum);
```

- Our results show that converting a sequential stream into a parallel one brings performance benefits only for an ArrayList:

```
Benchmark                                                        Mode  Cnt
Score        Error  Units
DifferentSourceSplitting.differentSourceArrayListParallel        avgt   25
2004849,711 ±    5289,437  ns/op
DifferentSourceSplitting.differentSourceArrayListSequential      avgt   25
5437923,224 ±   37398,940  ns/op
DifferentSourceSplitting.differentSourceLinkedListParallel       avgt   25
13561609,611 ±  275658,633  ns/op
DifferentSourceSplitting.differentSourceLinkedListSequential     avgt   25
10664918,132 ±  254251,184  ns/op
```

- The reason behind this is that arrays can split cheaply and evenly, while LinkedList has none of these properties.
- TreeMap and HashSet split better than LinkedList but not as well as arrays.

Merging Costs

Every time we split the source for parallel computation, we also need to make sure to combine the results in the end.

```
arrayListOfNumbers.stream().reduce(0, Integer::sum);
arrayListOfNumbers.stream().parallel().reduce(0, Integer::sum);
arrayListOfNumbers.stream().collect(Collectors.toSet());
arrayListOfNumbers.stream().parallel().collect(Collectors.toSet())
```

- Our results show that converting a sequential stream into a parallel one brings performance benefits only for the sum operation:

```
Benchmark                                                        Mode  Cnt
Score        Error  Units
MergingCosts.mergingCostsGroupingParallel                        avgt   25
135093312,675 ± 4195024,803  ns/op
MergingCosts.mergingCostsGroupingSequential                      avgt   25
70631711,489 ± 1517217,320  ns/op
MergingCosts.mergingCostsSumParallel                             avgt   25
2074483,821 ±    7520,402  ns/op
MergingCosts.mergingCostsSumSequential                           avgt   25
5509573,621 ±   60249,942  ns/op
```

- The merge operation is really cheap for some operations, such as reduction and addition, but merge operations like grouping to sets or maps can be quite expensive.

Memory Locality

- Modern computers use a sophisticated multilevel cache to keep frequently used data close to the processor. When a linear memory access pattern is detected, the hardware prefetches the next line

of data under the assumption that it will probably be needed soon.

```
private static final int[] intArray = new int[1_000_000];
private static final Integer[] integerArray = new Integer[1_000_000];

static {
    IntStream.rangeClosed(1, 1_000_000).forEach(i -> {
        intArray[i-1] = i;
        integerArray[i-1] = i;
    });
}

Arrays.stream(intArray).reduce(0, Integer::sum);
Arrays.stream(intArray).parallel().reduce(0, Integer::sum);
Arrays.stream(integerArray).reduce(0, Integer::sum);
Arrays.stream(integerArray).parallel().reduce(0, Integer::sum);
```

- Our results show that converting a sequential stream into a parallel one brings slightly more performance benefits when using an array of primitives:

```
Benchmark                                                    Mode  Cnt
Score        Error  Units
MemoryLocalityCosts.localityIntArrayParallel                 avgt   25
116247,787 ±     283,150  ns/op
MemoryLocalityCosts.localityIntArraySequential               avgt   25
293142,385 ±    2526,892  ns/op
MemoryLocalityCosts.localityIntegerArrayParallel             avgt   25
2153732,607 ±   16956,463  ns/op
MemoryLocalityCosts.localityIntegerArraySequential           avgt   25
5134866,640 ±  148283,942  ns/op
```

- An array of primitives brings the best locality possible in Java. In general, the more pointers we have in our data structure, the more pressure we put on the memory to fetch the reference objects. This can have a negative effect on parallelization, as multiple cores simultaneously fetch the data from memory.
-

4.5. The NQ Model

- Oracle presented a simple model that can help us determine whether parallelism can offer us a performance boost. In the NQ model, N stands for the number of source data elements, while Q represents the amount of computation performed per data element.

- The larger the product of N*Q, the more likely we are to get a performance boost from parallelization. For problems with a trivially small Q, such as summing up numbers, the rule of thumb is that N should be greater than 10,000. As the number of computations increases, the data size required to get a performance boost from parallelism decreases.

Parallelism vs Sequential

- Parallelism can bring performance benefits in certain use cases. But parallel streams cannot be considered as a magical performance booster. So, sequential streams should still be used as default during development.
- A sequential stream can be converted to a parallel one when we have actual performance requirements. Given those requirements, we should first run a performance measurement and consider parallelism as a possible optimization strategy.
- A large amount of data and many computations done per element indicate that parallelism could be a good option.
- On the other hand, a small amount of data, unevenly splitting sources, expensive merge operations and poor memory locality indicate a potential problem for parallel execution.

Thread Safety and Synchronization

Synchronization in Java

Synchronization in Java is a mechanism that ensures that two or more concurrent threads do not simultaneously execute some particular program segment known as a critical section. It helps to prevent thread interference and memory consistency errors.

synchronized Keyword

Intrinsic Locks and Synchronization

Each object in Java has an intrinsic lock (or monitor lock). When a thread enters a synchronized method or block, it acquires the intrinsic lock. Other threads trying to enter a synchronized method or block on the same object are blocked until the lock is released.

Types of Synchronization

- Method-level Synchronization:

```
public class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}
```

- Block-level Synchronization:

```
public class Counter {
    private int count = 0;
    private final Object lock = new Object();

    public void increment() {
        synchronized (lock) {
            count++;
        }
    }

    public int getCount() {
        synchronized (lock) {
            return count;
        }
    }
}
```

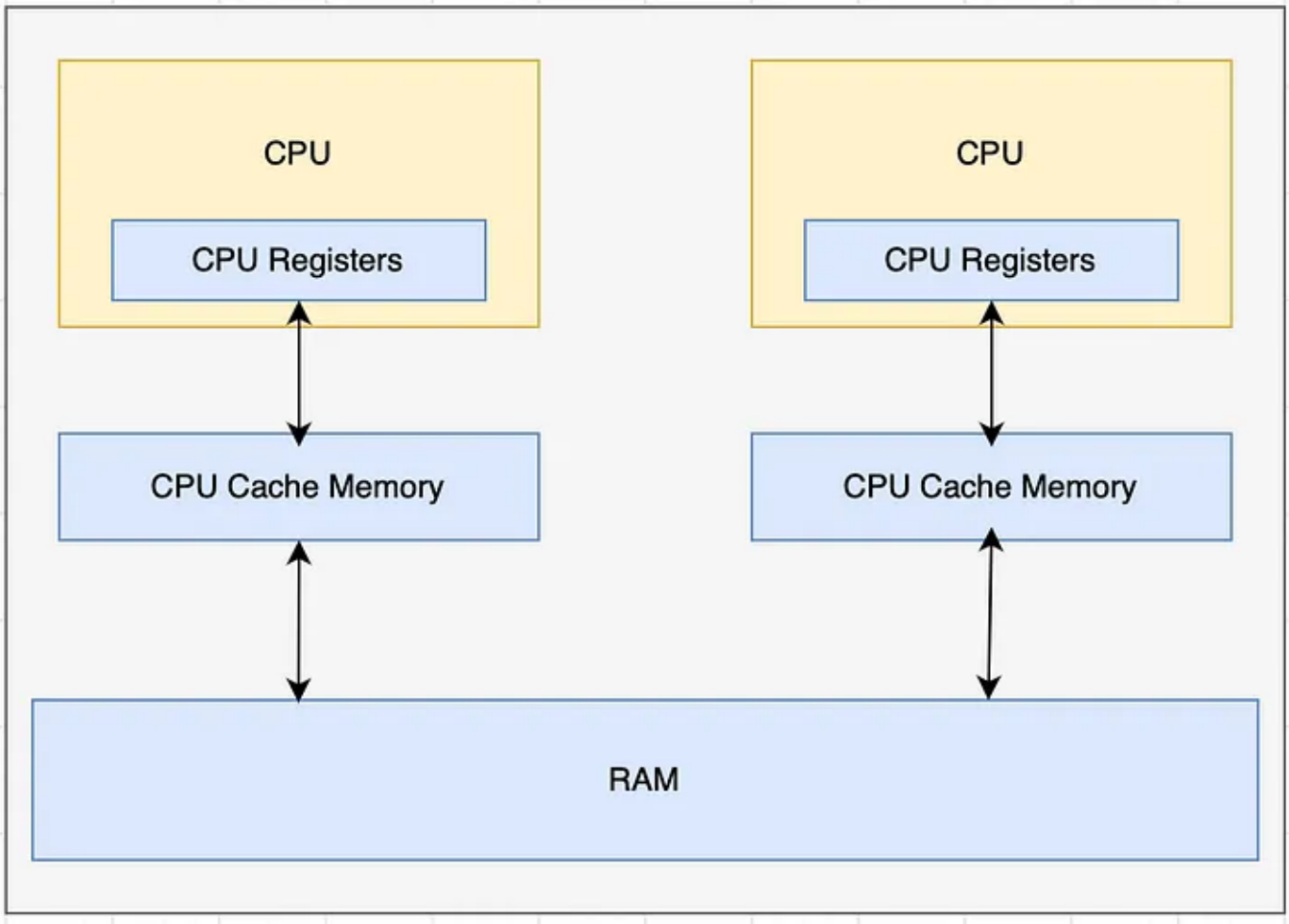
- The intrinsic lock (or monitor) associated with the Counter instance is acquired when either of these methods is called.
- If a thread is executing `increment`, no other thread can execute `increment` or `getCount` on the same `Counter` instance until the first thread releases the lock.
- This ensures that both incrementing and reading the count variable are thread-safe.
- Blocking Behavior: When a thread is accessing a synchronized method or block, it holds the intrinsic lock (or monitor) for the object. While a thread holds this lock, no other thread can access any other synchronized method or block that requires the same lock.

Threads and Memory

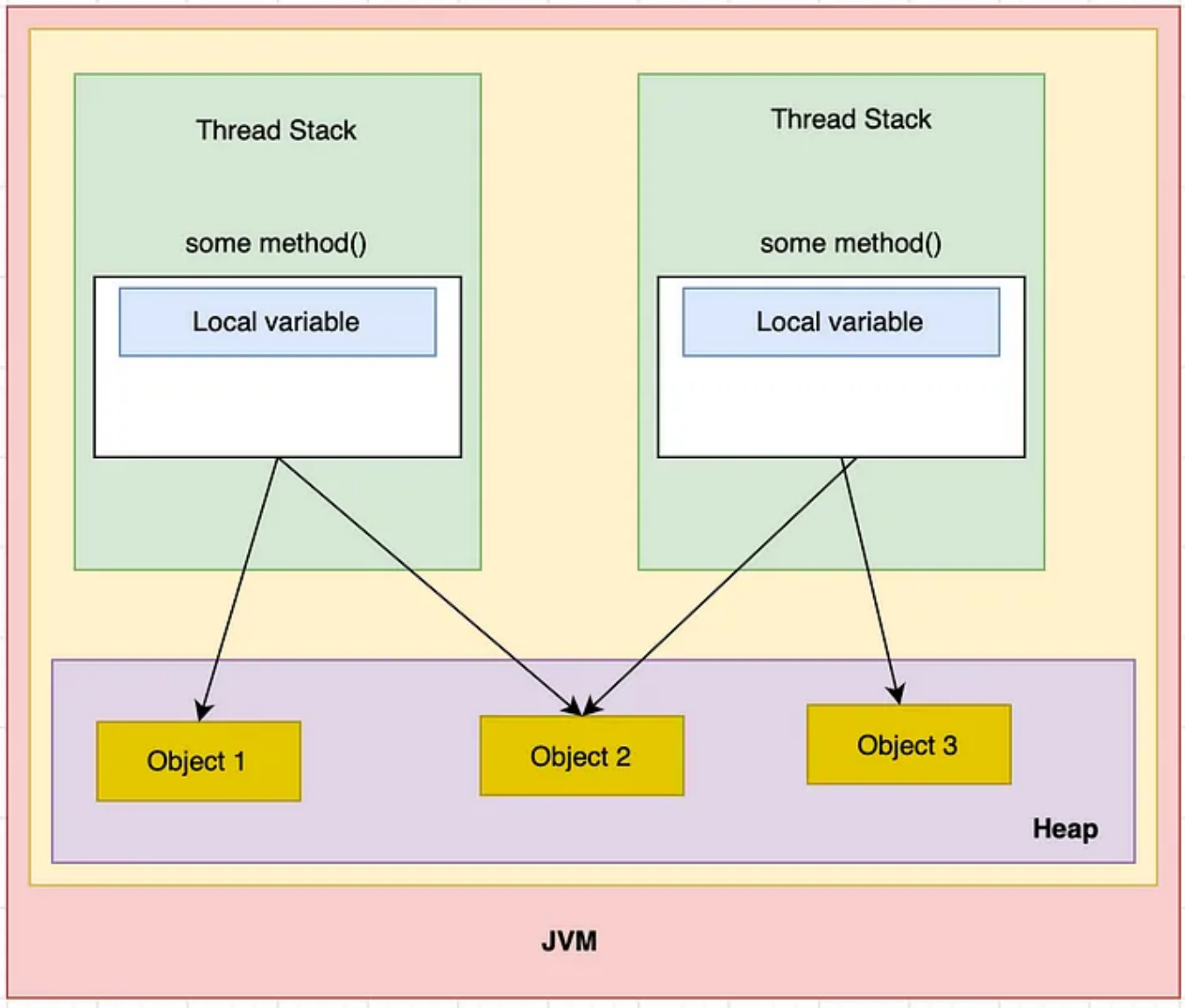
CPU Memory

- Modern computers have several powerful processors. Each of the processors interacts with computer memory. The processor loads data from memory, processes it, and then writes that data to memory.
- CPU (Central Processing Unit): Executes instructions from programs by performing basic arithmetic, logic, control, and input/output operations. Consists of multiple cores, each capable of executing instructions. While Processor: Generally synonymous with CPU in most contexts.But may sometimes refer to specific types of processors (e.g., graphics processor, network processor) depending on the context.
- RAM (Random Access Memory): Volatile memory used for temporary storage of data that the CPU needs to access quickly. Faster access speed compared to a hard disk. Data is lost when the computer is powered off.
- But RAM has a slower memory access speed than the processor can handle.
- To solve the problem of the fact that the processor is in standby mode while it is accessing data in memory, the processors have added their own cache.
- The cache consists of allocated memory for each CPU and register.
- A cache is faster than accessing the main memory but slower than accessing its internal registers.
- Some CPUs have multiple levels of cache (Level 1, Level 2, etc.).

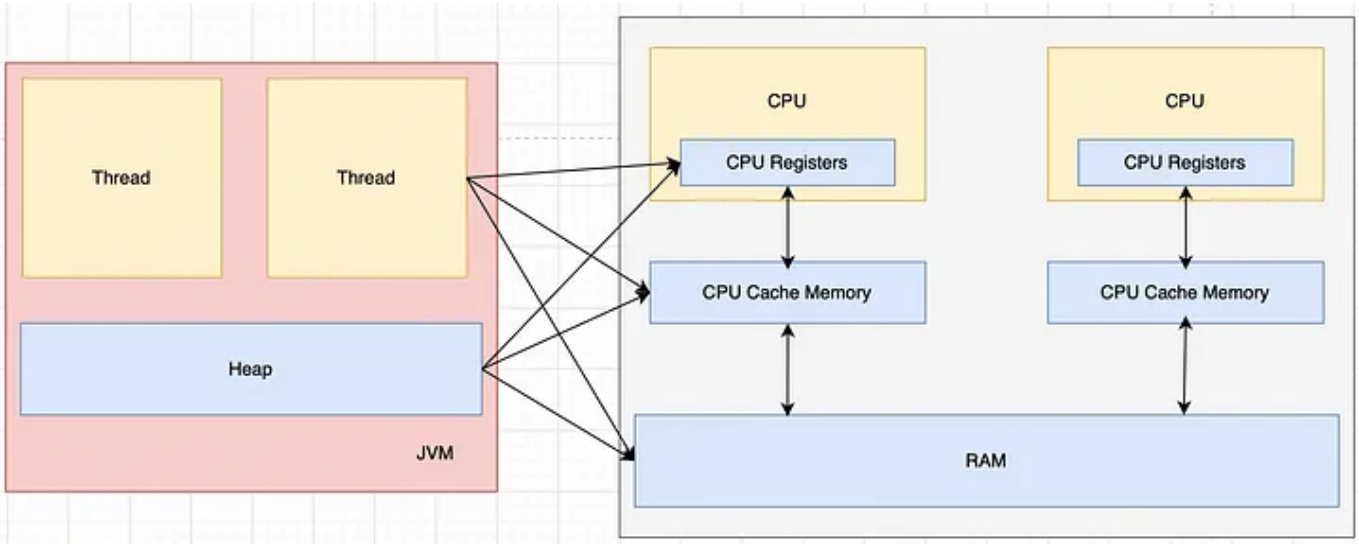
- How does the CPU read and write data to main memory?
    1. CPU reads data from main memory into it’s cache;
    2. CPU may read data into it’s internal register from it’s cache;
    3. CPU performs operations on the data;
    4. CPU writes data back to the cache if necessary;
    5. CPU writes data from the cache to main memory as needed.



- The JVM divides memory between thread stack and the heap.
- Each thread has its own stack, a memory are a that stores local variables, object references, and information about which methods the thread has called.
- Local reference variables, referencing objects on the heap, also only visible to the thread that owns it.
- JVM heap is an independent memory allocation.
- The heap contains all objects created by the application. Objects reside here regardless of which thread created them.
- Objects on the heap are visible to all threads.



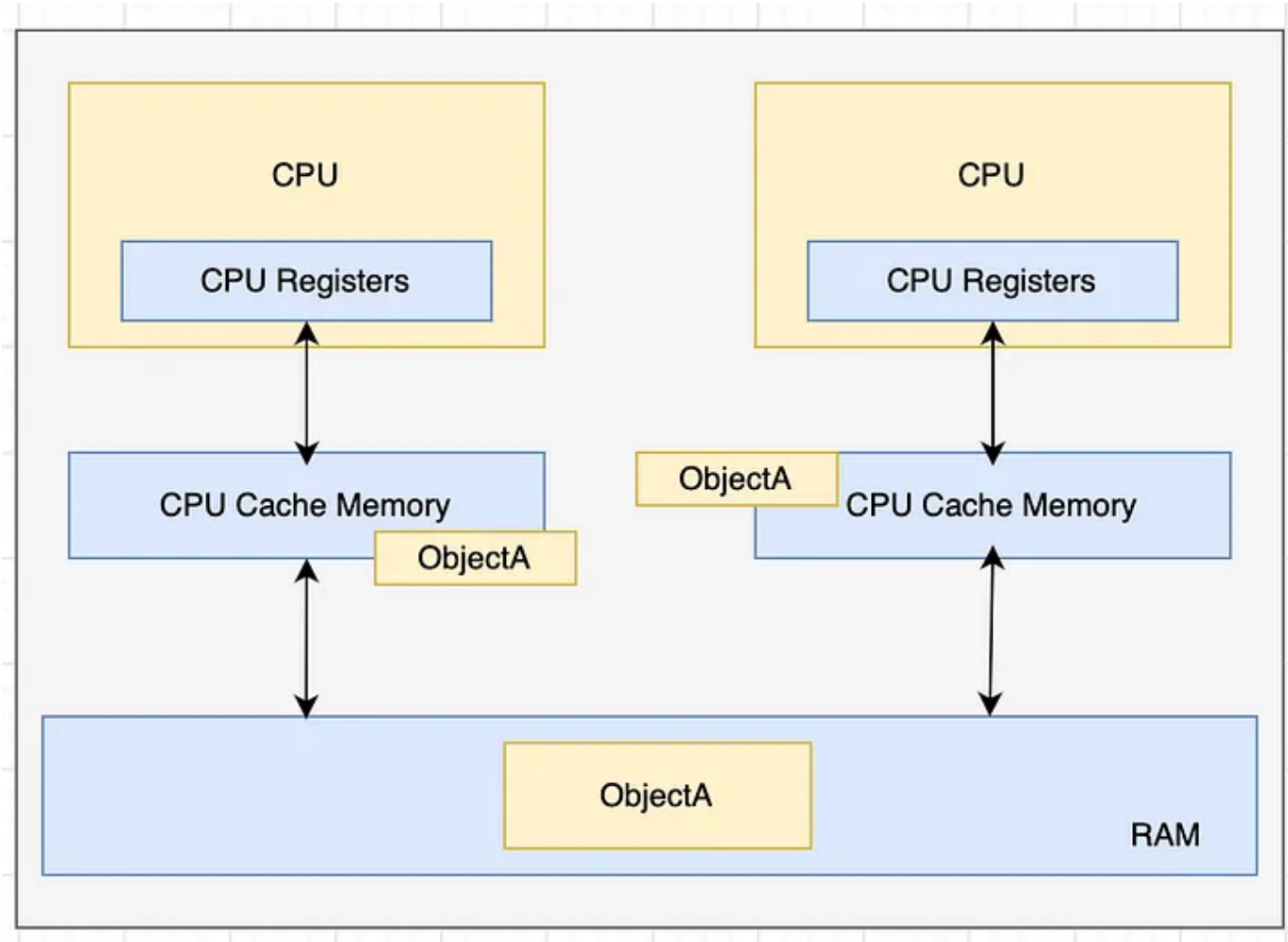
- Variables referenced by the stack of thread and heap can be cached in all levels of the computer's memory.



- Static class variables are also stored on the heap.

Synchronication problem

- If two threads share an object at the same time without synchronization or a proper object update message, then updates to the shared object made by one thread may not be visible to other threads.
- Let's imagine that ThreadA running on CPU1 reads data and caches it in its register while making some changes to this data.
- At the same time, ThreadB that is running on CPU2 is also reading this object's data.
- A situation arises in which each of the threads has its own copy of the shared object, each copy is in the CPU cache.
- This situation will continue until each of the CPUs has flushed its cache to the main memory (RAM).



- A program must be correctly synchronized to avoid the kinds of counterintuitive behaviors. Java includes certain language constructs that help solve concurrency problems.

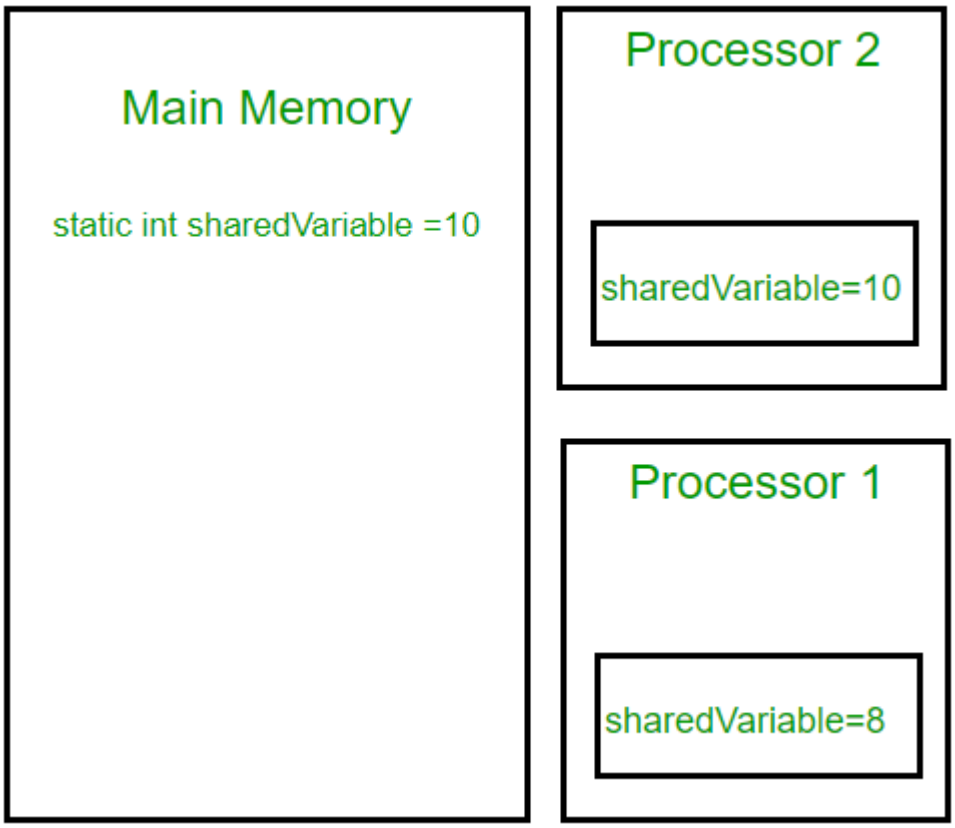
- Suppose that two threads are working on SharedObj.

```
class SharedObj
{
   // Changes made to sharedVar in one thread
   // may not immediately reflect in other thread
   static int sharedVar = 6;
}
```

- If two threads run on different processors each thread may have its own local copy of sharedVariable. If one thread modifies its value the change might not reflect in the original one in the main memory

instantly. This depends on the write policy of cache. Now the other thread is not aware of the modified value which leads to data inconsistency.

- The below diagram shows that if two threads are run on different processors, then the value of sharedVariable may be different in different threads.



volatile Keyword

The volatile keyword is used to mark a Java variable as "being stored in main memory". Every read of a volatile variable will be read from the computer's main memory, and not from the CPU cache, ensuring visibility of changes to variables across threads.

```
public class TaskRunner {

    private static int number;
    private static boolean ready;

    private static class Reader extends Thread {

        @Override
        public void run() {
            while (!ready) {
                Thread.yield();
            }

            System.out.println(number);
        }
    }
```

```
    public static void main(String[] args) {
        new Reader().start();
        number = 42;
        ready = true;
    }
}
```

- Once scenario:
    - The main thread has its copy of ready and number variables in its core cache.
    - The reader thread ends up with its copies, too.
    - The main thread updates the cached values.
- Most modern processors write requests that won't be applied right after they're issued. Processors tend to queue those writes in a special write buffer. After a while, they'll apply those writes to the main memory all at once.
- With all that being said, when the main thread updates the `number` and `ready` variables, there's no guarantee about what the reader thread may see. In other words, the reader thread may immediately see the updated value, with some delay, or never at all.
- We can use volatile to tackle the issues with Cache Coherence .
- To ensure that updates to variables propagate predictably to other threads, we should apply the volatile modifier to those variables.
- This way, we can communicate with runtime and processor to not reorder any instruction involving the volatile variable. Also, processors understand that they should immediately flush any updates to these variables.

```
public class TaskRunner {

    private volatile static int number;
    private volatile static boolean ready;

    // same as before
}
```

volatile and Thread Synchronization

- For multithreaded applications, we need to ensure a couple of rules for consistent behaviour:
    - Mutual Exclusion – only one thread executes a critical section at a time
    - Visibility – changes made by one thread to the shared data are visible to other threads to maintain data consistency
- `synchronized` methods and blocks provide both of the above properties at the cost of application performance.

final Keyword

The final keyword in the context of concurrency ensures that a variable' s state cannot be changed once it is initialized. It can provide thread-safety for immutable objects.

```
private final int value;
```

Atomic Variables

Java provides atomic classes (e.g., AtomicInteger, AtomicBoolean) in the java.util.concurrent.atomic package that support lock-free thread-safe operations on single variables.

```
AtomicInteger atomicInt = new AtomicInteger(0);
atomicInt.incrementAndGet();
```

- If two threads try to get and update the value at the same time, it may result in lost updates.
    - In multithreaded applications, each thread has its cached version of shared resources and updates the values in or from the main memory based on events or a schedule.
    - The thread cache and main memory values might differ. Therefore, even if one thread updates the values in the main memory, these changes are not instantly visible to other threads. This is called a **visibility** problem.
    - The volatile keyword helps us to resolve this issue by bypassing caching in a local thread.
    - In an increment example, we perform two operations with the variable count. First, we read this variable and then assign a new value to it. This means that the increment operation isn't atomic.
    - What we're facing here is a **race condition**. Each thread should read the value first, increment it, and then write it back. The problem will happen when several threads start working with the value, and read it before another one writes it.
    - This way, one thread may override the result written by another thread. The synchronized keyword can resolve this problem. However, this approach might create a bottleneck, and it's not the most elegant solution to this problem.
    - Using locks could solves the problem. However, the performance takes a hit.
    - When multiple threads attempt to acquire a lock, one of them wins, while the rest of the threads are either blocked or suspended.
    - The process of suspending and then resuming a thread is very expensive and affects the overall efficiency of the system.
- There is a branch of research focused on creating non-blocking algorithms for concurrent environments. These algorithms exploit low-level atomic machine instructions such as compare-and-swap (CAS), to ensure data integrity.

- A typical CAS operation works on three operands:
    - The memory location on which to operate (M)
    - The existing expected value (A) of the variable
    - The new value (B) which needs to be set
- The CAS operation updates **atomically** the value in M to B, but only if the existing value in M matches A, otherwise no action is taken.
- In both cases, the existing value in M is returned. This combines three steps – getting the value, comparing the value, and updating the value – into a single machine level operation.
- When multiple threads attempt to update the same value through CAS, one of them wins and updates the value. However, unlike in the case of locks, no other thread gets suspended; instead, they're simply informed that they did not manage to update the value. The threads can then proceed to do further work and context switches are completely avoided.
- One other consequence is that the core program logic becomes more complex. This is because we have to handle the scenario when the CAS operation didn't succeed. We can retry it again and again till it succeeds, or we can do nothing and move on depending on the use case.
- Atomic variables help with handling non-atomic operations like increment-decrement, or any operations, which need to read the value before assigning a new one. Atomic values are a simple and convenient way to resolve synchronization issues in our code.
- Java provides atomic classes such as AtomicInteger, AtomicLong, AtomicBoolean and AtomicReference. Objects of these classes represent the atomic variable of int, long, boolean, and object reference respectively. These classes contain the following methods.
    - set(int value): Sets to the given value
    - get(): Gets the current value
    - lazySet(int value): Eventually sets to the given value
    - compareAndSet(int expect, int update): Atomically sets the value to the given updated value if the current value == the expected value
    - addAndGet(int delta): Atomically adds the given value to the current value
    - decrementAndGet(): Atomically decrements by one the current value

Locks

ReentrantLock

The ReentrantLock class provides more extensive locking operations than synchronized methods and blocks.

- The traditional way to achieve thread synchronization in Java is through the synchronized keyword. While effective for basic synchronization, it is limited and rigid; for instance, a thread can only take a lock once, and there's no waiting queue mechanism, which can lead to resource starvation for some threads.
- Reentrant Locks are provided in Java to provide synchronization with greater flexibility.

- Reentrant Locks offer a more flexible approach. The ReentrantLock class implements the Lock interface, providing synchronization by surrounding shared resource access with lock and unlock calls. This ensures that only the current thread holds the lock, blocking others.
- As the name suggests, ReentrantLock allows a thread to acquire the same lock multiple times, with a hold count incrementing with each lock and decrementing with each unlock. When the hold count reaches zero, the resource is unlocked.
- ReentrantLock's ability to allow a thread to acquire the same lock multiple times is useful in scenarios where a thread needs to re-enter a locked section of code, particularly in complex, recursive, or nested method calls.
    - `Recursive Methods`: If a thread holding a lock calls a method that also tries to acquire the same lock, the thread would be blocked if the lock wasn't reentrant. With ReentrantLock, the thread can acquire the lock again without being blocked, allowing recursive method execution.
    - `Nested Method Calls`: In cases where methods that require the same lock are called within each other, ReentrantLock allows these nested calls without causing deadlock.
    - `Code Reuse and Modularity`: ReentrantLock allows methods to be designed independently, without worrying about whether they will be called within the context of another locked method. This promotes code reuse and modularity without the risk of introducing deadlocks.
- Reentrant Locks also support a fairness parameter, ensuring that the longest-waiting thread acquires the lock next. This fairness mode is enabled by passing true to the lock's constructor. These locks are used in the following way:

```
public void some_method()
{
        reentrantlock.lock();

          try{
            //Do some work
        }

        catch(Exception e){
            e.printStackTrace();
        }

        finally{
            reentrantlock.unlock();
        }
}
```

Unlock statement is always called in the finally block to ensure that the lock is released even if an exception is thrown in the method body(try block).

ReadWriteLock

The ReadWriteLock interface defines a pair of locks for read and write access. The ReentrantReadWriteLock is an implementation that allows multiple readers or one writer at a time.

- Usually, a lock grants exclusive access to a shared resource: just one thread at a flash can acquire the lock and everyone accesses to the shared resource requires that the lock be acquired first.
- ReadWriteLock allow side-by-side access to a shared resource
- The approach is, that multiple threads can read from a shared resource without causing concurrency errors. The concurrency errors first occur when writes and reads to a shared resource occur simultaneously, or if multiple writes take place simultaneously.
- Read lock and Write lock which allows a thread to lock the ReadWriteLock either for reading or writing.
    - Read lock: If there is no thread that has requested the write lock and the lock for writing, then multiple threads can lock the lock for reading. It means multiple threads can read the data at the very moment, as long as there's no thread to write the data or to update the data.
    - Write Lock: If no threads are writing or reading, only one thread at a moment can lock the lock for writing. Other threads have to wait until the lock gets released. It means, only one thread can write the data at the very moment, and other threads have to wait.

```
private final ReentrantReadWriteLock rwLock = new
ReentrantReadWriteLock();

// The readLock allows multiple threads to acquire the lock simultaneously
// as long as no thread holds the writeLock.
private final Lock readLock = rwLock.readLock();

// The writeLock is exclusive, meaning only one thread can hold it at a
time,
// and while it is held, no thread can acquire the readLock.
private final Lock writeLock = rwLock.writeLock();

/**
 * The readMethod acquires the readLock to ensure that the read-only
operations
 * are performed in a thread-safe manner. Multiple threads can execute
this method
 * concurrently as long as no thread holds the writeLock.
 */
public void readMethod() {
    readLock.lock();
    try {
        // Perform read-only operations here
    } finally {
        // Ensure the lock is always released to avoid deadlock
        readLock.unlock();
    }
}
```

```
/**
 * The writeMethod acquires the writeLock to ensure that the write
operations
 * are performed in a thread-safe manner. While a thread holds the
writeLock,
 * no other thread can acquire either the readLock or the writeLock,
ensuring
 * exclusive access for write operations.
 */
public void writeMethod() {
    writeLock.lock();
    try {
        // Perform write operations here
    } finally {
        // Ensure the lock is always released to avoid deadlock
        writeLock.unlock();
    }
}
```

Automic Variables Vs Locks

Feature	Atomic Variables	Locks (e.g., ReentrantLock)
Thread Safety	Provides thread safety for single variables	Provides thread safety for complex operations across multiple variables or blocks of code
Locking Mechanism	Lock-free (uses CAS operations)	Uses locks (threads may be blocked)
Performance	Generally faster under low contention	May incur overhead due to context switching or contention
Flexibility	Limited to simple atomic operations	Highly flexible, supports complex synchronization scenarios
Reentrancy	Not applicable	Reentrant locks allow the same thread to acquire the lock multiple times
Fairness	Not applicable	Fairness policies can be applied to ensure order of lock acquisition
Use Cases	Simple counters, flags, reference updates	Complex synchronization, fairness requirements, condition variables

Automic Variable Use Cases

- Simple Counters: When you need to increment or decrement a counter across multiple threads.
- Flags: When managing a simple boolean flag across threads.
- Reference Updates: When updating a shared reference or pointer in a thread-safe manner.

Locks Use Cases

- Complex Synchronization: When you need to coordinate access to multiple resources or manage complex conditions.
- Fairness: When you need to ensure that threads acquire locks in a fair order.
- Condition Variables: When threads need to wait for certain conditions before proceeding.

Hibernate

- Hibernate is a Java-based Object-Relational Mapping (ORM) framework that simplifies the process of mapping object-oriented domain models to relational databases.
- Well, it means that Hibernate allows developers to create Java classes that represent database tables, and it automatically maps those classes to their corresponding tables in the database.
- Hibernate features:
    - automatic schema generation
    - lazy loading of data
    - support for transactions
    - caching

Eager Loading vs Lazy Loading

- Eager Loading: Fetches related entities immediately along with the main entity in a single query. It is useful when you know you will need the related entities upfront.
- Lazy Loading: Delays the fetching of related entities until they are explicitly accessed. It can reduce the initial load time but may lead to multiple queries (N+1 problem).
- N+1 Problem: Occurs when lazy loading results in one query to fetch the main entity and N additional queries to fetch each related entity. If you have 'N' entities, this results in 'N+1' queries, hence the name.
- This can lead to performance issues. Not only does this strain the database, but it also slows down the user experience.

Preventing/Neutralizing the N+1 Problem with Eager and Lazy Loading Techniques

- Two primary strategies to address the N+1 problem are eager loading and lazy loading. Both techniques have their place depending on the context and requirements of your application.

- Eager loading is a technique where related data is loaded simultaneously with the main entity. This approach can be more efficient because it reduces the number of database queries.

- Conversely, lazy loading defers the loading of related data until it's actually needed. This prevents unnecessary data from being loaded, which can be beneficial for reducing memory usage and initial load times. However, if not managed carefully, lazy loading can inadvertently cause the N+1 problem as each access to related data may trigger a separate query.

Fetching strategies in Hibernate.

- We can enable Loading Strategy (eager - lazy) by using this annotation parameter:

```
fetch = FetchType.LAZY

fetch = FetchType.EAGER
```

- For lazy loading, we use a proxy object and fire a separate SQL query to load the lazyloaded data.
- The idea of disabling proxies or lazy loading is considered a bad practice in Hibernate. It can result in fetching and storing a lot of data, irrespective of the need for it.

Advantages vs Disadvantages

Lazy Loading Advantages:

- Much smaller initial load time than in the other approach
- Less memory consumption than in the other approach Disadvantages:
- Delayed initialization might impact performance during unwanted moments.
- In some cases we need to handle lazily initialized objects with special care, or we might end up with an exception.

Eager Loading Advantages:

- No delayed initialization-related performance impacts Disadvantages:
- Long initial loading time
- Loading too much unnecessary data might impact performance

JPA Default FetchType

By default, @OneToMany and @ManyToMany associations use the FetchType.LAZY strategy while the @OneToOne and @ManyToOne use the FetchType.EAGER strategy instead.

N+1 Query Problem

The N+1 problem occurs when your application needs to return a set of data that includes related data that exists in:

- Another table.
- Another database (in the case of microservices, for example)
- Or even another third-party service.
- Example : A relationship between Post and Comment is one-to-many. so when an application needs to return a list of posts and their related comments.
- So, N queries executed to retrieve every post's comments and 1 query to retrieve all posts (N comments queries + 1 posts query).

Why Is the N+1 Query Problem a Serious Issue?

- Your application makes a lot of database queries or external requests to retrieve a list of data like posts.
- The more data your application retrieves, the slower your request is going to be and the more resources your application is going to consume.
- A large data set might end up with notable network latency.

- It is going to be challenging to scale the application to handle larger data sets.

Strategies to Avoid the N+1 Query Problem

1. Eager Loading (Using SQL Joins, for example)

- In this strategy, instead of returning the post's comments separately for every post, you can use SQL Joins.
- It is only one database query to return all posts and their nested comments.

2. Batch Loading

- Execute one request to retrieve all posts.
- Execute another request to load a batch of posts' comments instead of loading every post's comments separately.
- Map every comment to its corresponding parent post.

3. Caching Caching the posts' comments or even the posts themselves which significantly minimizes the load on databases.

4. Lazy Loading

- In this strategy, Distributing the responsibility between the server side and the client side.
    - GET /api/posts: Retrieves all posts.
    - GT /api/comments/:postId: Retrieves a post's comments on demand. And n```ow, the data retrieval is up to the client side.
- The key point of this strategy is that it gets rid of nested data like comments and flattens all data sets in their own endpoint.

Caching

- Improves performance and less time-consuming by storing entities in memory.
- Caching is a technique that allows frequently accessed data to be stored in memory so that it can be quickly retrieved without having to go back to the database every time.
- By caching data, Hibernate can reduce the number of database queries it needs to execute, improving performance and reducing the load on the database server.

Levels of Caching in Hibernate

1. First-Level Cache (also known as Session Cache)

First-level caching is an in-built caching mechanism provided by Hibernate that stores the data in the memory of the Session object. Whenever an entity is fetched from the database, it is stored in the cache. If the same entity is requested again within the same session , Hibernate retrieves it from the cache instead of hitting the database. This can significantly improve the performance of the application by reducing the number of database queries.

The first-level cache has session-level scope, which means that the data stored in the cache is available only within the session. When the session is closed, the cache is cleared automatically. The cache is also not shared between different sessions, which means that each session has its own separate cache.

Hibernate manages the first-level cache automatically, and there is no need to configure it manually. The cache is enabled by default and **can't** be turned off. When an entity is fetched from the database, Hibernate checks if it is already present in the cache. If it is, the cached entity is returned, otherwise, Hibernate fetches the entity from the database and stores it in the cache.

- hibernate 1st cache can be accessed by entity manager

2. Second-Level Cache

Second-level caching is an optional caching mechanism provided by Hibernate that stores data across sessions.

To enable Second-level Caching in Hibernate, you need to configure a cache provider in your Hibernate configuration file. Hibernate supports multiple cache providers such as Ehcache, Infinispan, and Hazelcast. You also need to annotate the entities that you want to cache with the @Cacheable annotation. I'll show an example using Ehcache.

```
@Data
@Entity
@Table(name = "users")
@Cacheable
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String email;

}
```

- To enable caching for an entity, annotate it with the @Cacheable annotation. You can also specify the cache concurrency strategy using the @Cache annotation.
- The scope of Second-level Caching in Hibernate is is available across multiple sessions.
- Hibernate supports three cache modes for Second-level Caching: read-only, read-write, and transactional.
    - The read-only cache mode is used for data that is not expected to change frequently.
    - The read-write cache mode is used for data that is frequently updated.
    - The transactional cache mode is used for data that is updated within a transaction.
- The cache concurrency strategy determines how multiple threads access the cache.
- To ensure data consistency, Hibernate provides a cache synchronization mechanism that automatically updates the cache when the database is updated. This mechanism is called cache invalidation.
- When an entity is updated, Hibernate invalidates the corresponding cache entry, and the next time the entity is accessed, it is fetched from the database and stored in the cache.

3. Query Cache

Query caching is a caching mechanism provided by Hibernate to cache the results of queries. When a query is executed, Hibernate checks if the query results are already cached. If the results are found in the cache, they are returned without hitting the database, otherwise, the query is executed and the results are cached for future use.

```
spring.jpa.properties.hibernate.cache.use_query_cache=true
```

```
@Repository
public interface AuthorRepository extends CrudRepository<Author, Integer>
{
    @QueryHints({ @QueryHint(name = "org.hibernate.cacheable", value
="true") })
    Author findByName(String name);
}
```

2nd Level Cache Vs Query Cache

- Type of Data Cached:
  - Second-Level Cache: Caches entities and collections by their primary key.
  - Query Cache: Caches the result set of queries, including the parameters used in those queries.
- Dependencies:
- Second-Level Cache: Does not depend on the query cache.
- Query Cache: Depends on the second-level cache for storing entities.

Projection

- A way to query and retrieve specific columns or fields from the database rather than entire entities. This is useful for optimizing performance and reducing memory usage when only a subset of the data is required and reduce the amount of data loaded into memory.

1. Closed Projection

```
@Entity
public class Address {

    @Id
    private Long id;

    @OneToOne
    private Person person;

    private String state;

    private String city;

    private String street;
```

```
    private String zipCode;

    // getters and setters
}

@Entity
public class Person {

    @Id
    private Long id;

    private String firstName;

    private String lastName;

    @OneToOne(mappedBy = "person")
    private Address address;

    // getters and setters
}
```

```
// a projection interface for the Address class:
public interface AddressView {
    String getZipCode();
}
// us eprojection in repository interface
public interface AddressRepository extends Repository<Address, Long> {
    List<AddressView> getAddressByState(String state);
}
```

- It’s easy to see that defining a repository method with a projection interface is pretty much the same as with an entity class.
- The only difference is that the projection interface, rather than the entity class, is used as the element type in the returned collection.
- Behind the scenes, Spring creates a proxy instance of the projection interface for each entity object, and all calls to the proxy are forwarded to that object.
- We can use projections recursively.

```
public interface PersonView {
    String getFirstName();

    String getLastName();
}
```

```
public interface AddressView {
    // A method with the return type PersonView, a nested projection, in
the **Address** projection
    PersonView getPerson();
    // Notice the method that returns the nested projection must have the
same name as the method in the root class that returns the related entity.

}
```

- Note that recursive projections only work if we traverse from the owning side to the inverse side. If we do it the other way around, the nested projection would be set to null.

2. Open Projections

These projections enable us to define interface methods with unmatched names and with return values computed at runtime.

```
public interface PersonView {
    // ...

    @Value("#{target.firstName + ' ' + target.lastName}")
    String getFullName();
}


public interface PersonRepository extends Repository<Person, Long> {
    PersonView findByLastName(String lastName);
}
```

- The argument to the `@Value` annotation is a SpEL expression, in which the target designator indicates the backing entity object.
- Open projections do have a drawback though; Spring Data can't optimize query execution, as it doesn't know in advance which properties will be used.
- Thus, we should only use open projections when closed projections aren't capable of handling our requirements.

3. Class-Based Projections

- Instead of using proxies Spring Data creates from projection interfaces, we can define our own projection classes.

```
public class PersonDto {
    private String firstName;
    private String lastName;

    public PersonDto(String firstName, String lastName) {
        this.firstName = firstName;
```

```
        this.lastName = lastName;
    }

    // getters, equals and hashCode
}

public interface PersonRepository extends Repository<Person, Long> {
    // ...

    PersonDto findByFirstName(String firstName);
}
```

- For a projection class to work in tandem with a repository interface, the parameter names of its constructor must match the properties of the root entity class.
- We must also define equals and hashCode implementations; they allow Spring Data to process projection objects in a collection.

```
public record PersonDto(String firstName, String lastName) {

}
```

- Notice with the class-based approach, we can't use nested projections.

4. Dynamic Projections

- An entity class may have many projections. In some cases, we may use a certain type, but in other cases, we may need another type. Sometimes, we also need to use the entity class itself.
- Defining separate repository interfaces or methods just to support multiple return types is cumbersome. To deal with this problem, We can apply dynamic projections just by declaring a repository method with a Class parameter:

```
public interface PersonRepository extends Repository<Person, Long> {
    // By passing a projection type or the entity class to such a method,
we can retrieve an object of the desired type
    <T> T findByLastName(String lastName, Class<T> type);
}
```

-

Entity Inheritance

Inheritance is used in domain model. Unfortunately, the relational table model doesn't know this concept. So, Hibernate, or any other JPA implementation, has to apply a mapping strategy to map your inheritance hierarchy to one or more database tables.

You can choose between 4 strategies that map the inheritance structure of your domain model to different table structures. Each of these strategies has its advantages and disadvantages.

- Mapped Superclass
- Table per Class
- Single Table
- Joined

I will use the same simple domain model in all of the examples to show you the different inheritance strategies. It consists of an author who has written different kinds of publications. A publication can either be a book or a blog post. Both of them share most of their attributes, like the id, a title, and a publishing date. In addition to the shared attributes, the book also stores the number of pages, and the blog post persists its URL. alt text

1. Mapped Superclass

The mapped superclass strategy is the simplest approach to mapping an inheritance structure to database tables. It maps each concrete class to its own table.



- That allows you to share attributes and their mapping definitions between multiple entities. But it also has a huge drawback. A mapped superclass(parent class) is not an entity, and it's not mapped to a database table.
- That means that you can't use polymorphic queries that select all Publication entities or define an association between an Author entity and all Publications.
- Drawback: **Polymorphic** queries refer to the ability to write queries that can return instances of any class within an inheritance hierarchy. This means you can query a base class and retrieve objects of any subclass that extends from it.
- You either need to use a uni-directional association from the Publication to the Author entity or define associations between an Author and each kind of Publication.
- If you need such associations, you should use one of the other inheritance strategies. They are a much better fit for your use case.
- If you only want to share state and mapping information between your entities, the mapped superclass strategy is a good fit and easy to implement.
- You set up your inheritance structure, add the required mapping annotations to each attribute, and add the @MappedSuperclass annotation to your superclass.
- Without the @MappedSuperclass annotation, Hibernate will ignore the mapping information of your superclass.

```
// No Entity Annotation
@MappedSuperclass
public abstract class Publication {

    @Id
```

```
    @GeneratedValue(strategy = GenerationType.AUTO)
    protected Long id;

    protected String title;

    @Version
    private int version;

    private LocalDate publishingDate;

    …
}

// The subclasses Book and BlogPost extend the Publication class and add
their specific attributes with their mapping annotations. I annotated both
classes with @Entity, and the persistence provider will manage them.

@Entity
public class Book extends Publication {

    private int pages;

    …
}
@Entity
public class BlogPost extends Publication {

    private String url;

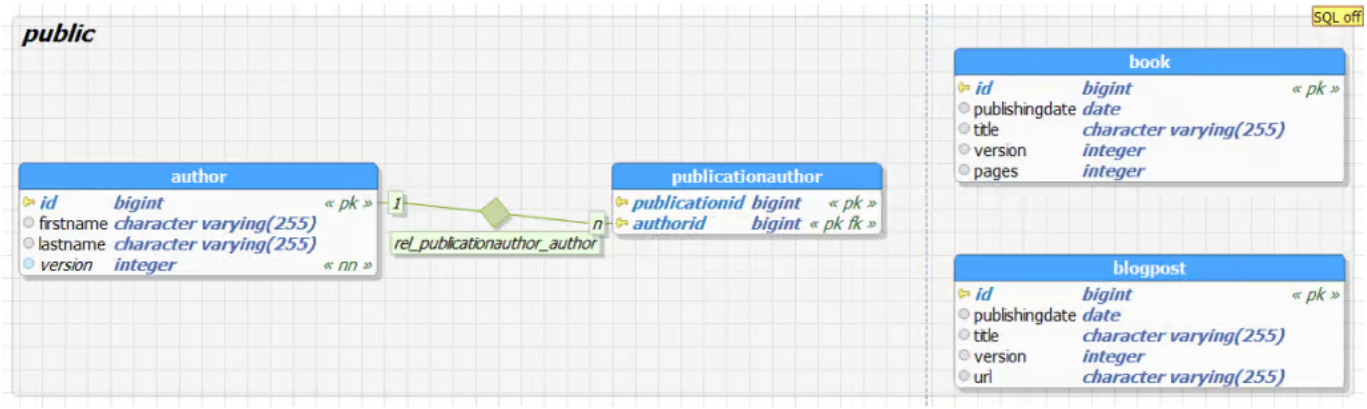
    …
}
```

```
List books = em.createQuery("SELECT b FROM Book b",
Book.class).getResultList();
```

```
15:38:36,020 DEBUG [org.hibernate.SQL] – select book0_.id as id1_2_,
book0_.publishingDate as publishi2_2_, book0_.title as title3_2_,
book0_.version as version4_2_, book0_.pages as pages5_2_ from Book book0_
```

2. Table per Class

The table-per-class strategy is similar to the mapped superclass strategy. The main difference is that the superclass is now also an entity. This mapping allows you to use polymorphic queries and to define associations to the superclass. However, the table structure adds a lot of complexity to polymorphic queries, and you should better avoid them.




```java
@Entity
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
public abstract class Publication {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    protected Long id;

    protected String title;

    @Version
    private int version;

        @ManyToMany
        @JoinTable(name = "PublicationAuthor",
                   joinColumns = { @JoinColumn(name = "publicationId",
referencedColumnName = "id") },
                   inverseJoinColumns = { @JoinColumn(name = "authorId",
referencedColumnName = "id") })
        private Set authors = new HashSet();

    private LocalDate publishingDate;

    …
}

@Entity
public class Book extends Publication {

    private int pages;

    …
}
@Entity
public class BlogPost extends Publication {

    private String url;

    …
}
```

The table-per-class strategy maps each entity to its own table, which contains a column for each entity attribute. That makes the query for a specific entity class easy and efficient.

```
List books = em.createQuery("SELECT b FROM Book b",
Book.class).getResultList();
```

- The superclass is now also an entity. You can use it to define an association between the Author and Publication entities. This allows you to call the getPublications() method to get all Publications written by that Author.
- When processing the result of a polymorphic query, Hibernate maps each record to the right class. So, in this example, Hibernate maps the query result to Book and BlogPost objects.

```
List authors = em.createQuery("SELECT a FROM Author a",
Author.class).getResultList();
for (Author a : authors) {
    for (Publication p : a.getPublications()) {
        if (p instanceof Book)
        log(p.getTitle(), "book");
        else
        log(p.getTitle(), "blog post");
    }
}
```

```
15:57:16,722 DEBUG [org.hibernate.SQL] - select author0_.id as id1_0_,
author0_.firstName as firstNam2_0_, author0_.lastName as lastName3_0_,
author0_.version as version4_0_ from Author author0_
15:57:16,765 DEBUG [org.hibernate.SQL] - select publicatio0_.authorId as
authorId2_4_0_, publicatio0_.publicationId as publicat1_4_0_,
publicatio1_.id as id1_3_1_, publicatio1_.publishingDate as
publishi2_3_1_, publicatio1_.title as title3_3_1_, publicatio1_.version as
version4_3_1_, publicatio1_.pages as pages1_2_1_, publicatio1_.url as
url1_1_1_, publicatio1_.clazz_ as clazz_1_ from PublicationAuthor
publicatio0_ inner join ( select id, publishingDate, title, version,
null::int4 as pages, null::varchar as url, 0 as clazz_ from Publication
union all select id, publishingDate, title, version, pages, null::varchar
as url, 1 as clazz_ from Book union all select id, publishingDate, title,
version, null::int4 as pages, url, 2 as clazz_ from BlogPost )
publicatio1_ on publicatio0_.publicationId=publicatio1_.id where
publicatio0_.authorId=?
Effective Java is a book.
```

- We store books and blog posts in separate tables. So, when generating the query, Hibernate has to join the author table with the result of a subselect, which uses a union to get all matching records from the book and blogpost tables. Depending on the amounts of records in both tables, this query might become a performance issue. And it gets even worse if you add more subclasses to the inheritance structure.

- Due to that, the table-per-class strategy shouldn't be your first choice if you expect many polymorphic associations or queries. The single table and the joined strategy are a much better fit for those applications.

3. Single Table

- The single table strategy maps all entities of the inheritance hierarchy to the same database table. This approach makes polymorphic queries very efficient and generally provides the best performance.

- But it also has some drawbacks. The attributes of all entities are mapped to the same database table. Each record uses only a subset of the available columns and sets the rest of them to null. You can, therefore, not use not null constraints on any column that isn't mapped to all entities. That can create data integrity issues, and your database administrator might not be too happy about it.



- When you persist all entities in the same table, Hibernate needs a way to determine the entity class each record represents. This information is stored in a discriminator column, which is not an entity attribute. You can either define the column name with a @DiscriminatorColumn annotation on the superclass, or Hibernate will use DTYPE(Discriminator value) as its default name(table name).

If you have to map a legacy database that doesn't provide a discriminator column, you can use a @DiscriminatorFormular instead. It expects an SQL snippet that returns the discriminator value for each class of your hierarchy. This approach is more complex and usually doesn't perform as well as a discriminator column.

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "Publication_Type")
public abstract class Publication {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    protected Long id;

    protected String title;

    @Version
    private int version;
```

```
        @ManyToMany
        @JoinTable(name = "PublicationAuthor",
                   joinColumns = { @JoinColumn(name = "publicationId",
referencedColumnName = "id") },
                   inverseJoinColumns = { @JoinColumn(name = "authorId",
referencedColumnName = "id") })
        private Set authors = new HashSet();

    private LocalDate publishingDate;

    …
}
```

- The @DiscriminatorValue annotation is optional if you use Hibernate.

```
@Entity
@DiscriminatorValue("Book")
public class Book extends Publication {

    private int pages;

    …
}
@Entity
@DiscriminatorValue("Blog")
public class BlogPost extends Publication {

    private String url;

    …
}
```

As I explained at the beginning of this section, the single table strategy allows easy and efficient data access. All attributes of each entity are stored in one table, and the query doesn't require any join statements. The only thing that Hibernate needs to add to the SQL query to fetch a particular entity class is a comparison of the discriminator value. In this example, it's a simple expression that checks that the column publication_type contains the value 'Book'.

```
List books = em.createQuery("SELECT b FROM Book b",
Book.class).getResultList();
```

```
16:02:47,411 DEBUG [org.hibernate.SQL] – select book0_.id as id2_1_,
book0_.publishingDate as publishi3_1_, book0_.title as title4_1_,
book0_.version as version5_1_, book0_.pages as pages6_1_ from Publication
book0_ where book0_.Publication_Type='Book'
```

- The previously discussed inheritance strategies had their issues with polymorphic queries. They were either not supported or required complex union and join operations.

```
List authors= em.createQuery("SELECT a FROM Author a",
Author.class).getResultList();
for (Author a : authors) {
    for (Publication p : a.getPublications()) {
        if (p instanceof Book)
        log(p.getTitle(), "book");
        else
        log(p.getTitle(), "blog post");
    }
}
```

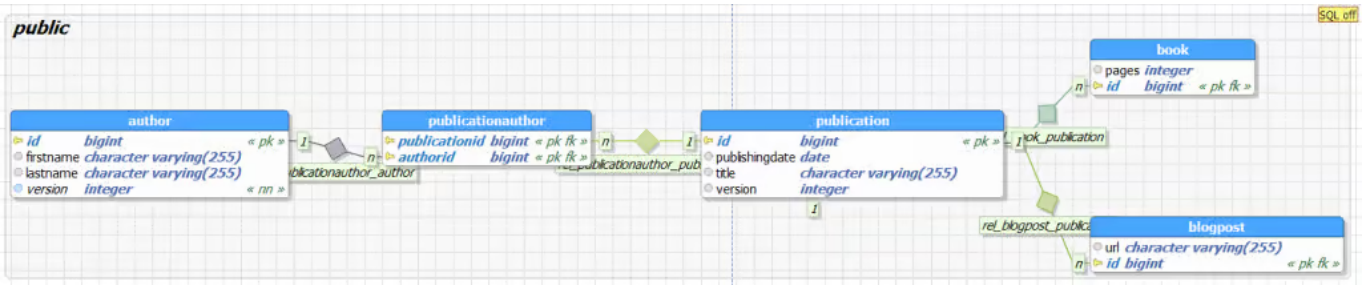
As you can see in the log messages, Hibernate selects all columns from the publication table, including the discriminator column publication_type. It then uses the discriminator value to choose the right entity class and map the database record. This query is much easier than the one created by the table-per-class strategy and generally causes fewer performance problems.

```
16:04:32,073 DEBUG [org.hibernate.SQL] – select author0_.id as id1_0_,
author0_.firstName as firstNam2_0_, author0_.lastName as lastName3_0_,
author0_.version as version4_0_ from Author author0_
16:04:32,118 DEBUG [org.hibernate.SQL] – select publicatio0_.authorId as
authorId2_2_0_, publicatio0_.publicationId as publicat1_2_0_,
publicatio1_.id as id2_1_1_, publicatio1_.publishingDate as
publishi3_1_1_, publicatio1_.title as title4_1_1_, publicatio1_.version as
version5_1_1_, publicatio1_.pages as pages6_1_1_, publicatio1_.url as
url7_1_1_, publicatio1_.Publication_Type as Publicat1_1_1_ from
PublicationAuthor publicatio0_ inner join Publication publicatio1_ on
publicatio0_.publicationId=publicatio1_.id where publicatio0_.authorId=?
Effective Java is a book.
```

4. Joined

The joined table approach maps each class of the inheritance hierarchy to its own database table. This sounds similar to the table-per-class strategy. But this approach maps each subclass to multiple tables. The abstract superclass Publication is mapped to its own database table. It contains the columns shared by all classes. The tables of the subclasses are much smaller than in the table-per-class strategy. They hold only the columns specific to the mapped entity class and share the primary key with the corresponding record in

the superclass table.



- Each query of a subclass requires a join to the table of the superclass. That increases the complexity of each query, but it also allows you to use subclass-specific constraints to ensure data integrity, e.g., a not null constraint on the url column of the blogpost table.

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Publication {

        @Id
        @GeneratedValue(strategy = GenerationType.AUTO)
        protected Long id;

        protected String title;

        @Version
        private int version;

        @ManyToMany
        @JoinTable(name = "PublicationAuthor",
                   joinColumns = { @JoinColumn(name = "publicationId",
referencedColumnName = "id") },
                   inverseJoinColumns = { @JoinColumn(name = "authorId",
referencedColumnName = "id") })
        private Set authors = new HashSet();

        private LocalDate publishingDate;

        …
}

@Entity
public class Book extends Publication {

    private int pages;

    …
}
@Entity
public class BlogPost extends Publication {

    private String url;
```

```
    …
}
```

- As I already explained, the columns mapped by each subclass are stored in separate database tables. The publication table contains all columns mapped by the superclass Publication, and the book table contains all columns mapped by the Book entity. Hibernate needs to join these 2 tables by their primary keys to select all attributes of the Book entity. This is an overhead that makes these queries slightly slower than the simpler queries generated for the single table strategy.

```
List books = em.createQuery("SELECT b FROM Book b",
Book.class).getResultList();
```

```
15:56:21,463 DEBUG [org.hibernate.SQL] – select book0_.id as id1_3_,
book0_.publishingDate as publishi2_3_, book0_.title as title3_3_,
book0_.version as version4_3_, book0_.pages as pages1_2_ from Book book0_
```

Hibernate has to use a similar approach for polymorphic queries. It has to left join the publication table with all tables of the subclasses to get all Pubications of an Author.

```
List authors= em.createQuery("SELECT a FROM Author a",
Author.class).getResultList();
for (Author a : authors) {
    for (Publication p : a.getPublications()) {
        if (p instanceof Book)
        log(p.getTitle(), "book");
        else
        log(p.getTitle(), "blog post");
    }
}
```

```
17:16:05,244 DEBUG [org.hibernate.SQL] – select author0_.id as id1_0_,
author0_.firstName as firstNam2_0_, author0_.lastName as lastName3_0_,
author0_.version as version4_0_ from Author author0_
17:16:05,280 DEBUG [org.hibernate.SQL] – select publicatio0_.authorId as
authorId2_4_0_, publicatio0_.publicationId as publicat1_4_0_,
publicatio1_.id as id1_3_1_, publicatio1_.publishingDate as
publishi2_3_1_, publicatio1_.title as title3_3_1_, publicatio1_.version as
version4_3_1_, publicatio1_1_.pages as pages1_2_1_, publicatio1_2_.url as
url1_1_1_, case when publicatio1_1_.id is not null then 1 when
publicatio1_2_.id is not null then 2 when publicatio1_.id is not null then
0 end as clazz_1_ from PublicationAuthor publicatio0_ inner join
Publication publicatio1_ on publicatio0_.publicationId=publicatio1_.id
left outer join Book publicatio1_1_ on publicatio1_.id=publicatio1_1_.id
left outer join BlogPost publicatio1_2_ on
```

```
publicatio1_.id=publicatio1_2_.id where publicatio0_.authorId=?
Effective Java is a book.
```

Choosing a Strategy

Choosing the right inheritance strategy is not an easy task. As so often, you have to decide which advantages you need and which drawbacks you can accept for your application.

- If data consistency is more important than performance and you need polymorphic queries and associations, the joined strategy is usually your best option.
- If you don’t need polymorphic queries or associations, my recommendation depends on the kind of inheritance hierarchy you’re modeling.
    - If all elements of the inheritance hierarchy exist in your application’s domain, your requirements might change, and you might need polymorphic queries in the future. In this case, the table-per-class strategy is most likely your best option. It allows you to use constraints to ensure data consistency and provides an option for polymorphic queries. But keep in mind that polymorphic queries are very complex for this table structure and that you should avoid them.
    - Sometimes, not all elements of the inheritance hierarchy exist in your application’s domain. This is often the case if you introduce a technical superclass to ensure all your entities have a standard set of attributes. In this case, the mapped superclass strategy is your best choice. It’s the simplest and most efficient way to share a set of attributes with their mapping definitions.

Hibernate Inheritance annotation

Formulas

- Allow you to define computed properties in entities using SQL expressions. They are read-only and calculated at the database level.
- The @Formula annotation is used to calculate the dynamic value of a property. @Formula takes an expression (this could be simple expression or a complex query) as a parameter. During fetch time, it evaluates the expression and assigns the evaluated value to the property.

```
// ex 1
@Formula("lower(datediff(curdate(), birth_date / 365)")
private int age;

// ex 2
@Formula("(select min(s.survey) from statistics s) ")
private float total;

// ex 3
private int num1;
private int num2;
private int num3;

@Formula(" num1 + num2 + num3")
private float totalSum;
```

- This is only inserted in a SELECT clause!
- Be careful when using this because the SQL command you may be using may be vendor specific.
    - When you use the @Formula annotation with a specific SQL expression, such as functions or queries, it ties your application more closely to the underlying SQL dialect of the database you're using (e.g., MySQL, PostgreSQL, Oracle).
    - Different SQL vendors may have variations in SQL syntax and functions. For example, functions like datediff, date handling, or aggregate functions (min, max, etc.) may differ slightly or have different names across databases.
- In other words, this will create coupling with a particular SQL vendor.
    - Using vendor-specific SQL in @Formula can create coupling or dependency on a particular database system.
    - If your application needs to switch databases in the future (e.g., from MySQL to PostgreSQL), you may need to modify these SQL expressions to be compatible with the new database system.
    - This can increase maintenance overhead and potentially introduce errors if not managed carefully during migration.

Maven

Dependency Management

Key Tags for Maven Dependencies

- `<dependency>`: The parent tag for defining a dependency.

1. `<groupId>`

- **Description**: Specifies the unique identifier for the group or organization that is responsible for the project.

2. `<artifactId>`

- **Description**: Defines the unique identifier for the specific project or library within the group. This is often the name of the library .

3. `<version>`

- **Description**: Specifies the version of the dependency to use. This is essential for ensuring compatibility.

4. `<scope>`

- **Description**: Indicates the visibility and lifecycle of the dependency in relation to the project. Common scopes include:
    - `compile`: Default scope; available in all classpaths (compile, test, runtime).
    - `provided`: Dependency is provided by the JDK or a container at runtime; not included in the packaged application.

- `runtime`: Dependency is not needed for compilation but is required at runtime.
- `test`: Dependency is only available for the test compilation and execution.
- `system`: Dependency is provided by the user and must be defined with an absolute path.

5. `<exclusions>`

- **Description**: Allows you to exclude specific transitive dependencies from being included in your project.
- **Example**:

```
<exclusions>
  <exclusion>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-log4j12</artifactId>
  </exclusion>
</exclusions>
```

6. `<optional>`

- **Description**: Indicates whether the dependency is optional. If set to `true`, the dependency is not required for the project and won't be included by default.

8. `<type>`

- **Description**: Specifies the type of the artifact (default is "jar"). Other types include "pom", "war", "ear", etc.

Example of a Maven Dependency

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <version>3.3.0</version>
    <scope>compile</scope>
    <exclusions>
        <exclusion>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-log4j12</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

- The pom.xml. does a million things—some of which include dependency management, build lifecycle customization and plug-in configuration. This is just the beginning of the list. Yes, this one file has everything!

- Let's begin with the easiest part — the `<dependency>` tag. Any dependency (library) that we wish to use in our project, we do so, by first including it under the `<dependencies>` tag of POM, like this:
- It is easy to understand what the `<dependencies>` tag does—Maven checks if the ***artifacts*** (jars mostly) of the libraries specified under this tag are available in our local repository; and if not, it download these artifacts from the remote repository into our local one.

Remote Repositories

- It is obvious that for a project to be compiled locally, all its dependencies, that is libraries it is dependent on, need to be present locally. Most of the dependencies we generally use are external dependencies, which means that these libraries have been developed by someone else. Remote repository is just a place where the artifacts or jars of these dependencies are hosted, in order to make it accessible for the public to download and use.
- Whenever anyone specifies in the POM, under the `<dependencies>` tag, that they need certain dependencies, Maven downloads the artifacts of those dependencies from the remote repository.
- The tag used in the pom.xml — `<repositories>` is the tg which the URLs of the remote repositories are configured.
- Maven, by default, uses the well-known remote repository — the Maven Central repository — (https://repo.maven.apache.org/maven2/) — to locate the artifacts of our dependencies. If all the dependencies we require are present in this Maven Central repository, we don't need to use the `<repositories>` tag. However, if the dependencies you wish to use aren't present in this central repository, say they are libraries internal to your organization and which are thus hosted only on your company's internal repository manager, you will need to add the address of that repository manager under the `<repositories>` tag.

```
<repositories>
  <repository>
    <id>company's-internal-repo-manager</id>
    <url>https://company-server/repo</url>
  </repository>
</repositories>
```

- The purpose of repository managers(compamy repo managers) is to cache artifacts from the Central Maven, reducing load on it and at the same time bringing down the time required to download these artifacts. It also serves the objective of hosting company-specific libraries that can't be hosted in public repositories.
- A few famous repository managers are JFrog Artifactory and Sonatype Nexus. These platforms host not only Maven JARs, but a plethora of totally unrelated artifacts and binaries, such as container (e.g., docker) images, configuration files and much more.
- Maven uses the Maven Central repository as the remote repository by default. But where is this default configured? How does our Maven project understand these default configurations when that's not been specified anywhere inside our POM?

The Super POM.

- All POMs inherit the Super POM implicitly. Find the Super POM here. Notice how the Central Maven repository URL has been specified under the `<repositories>` tag of the Super POM.
- The Super POM has not only the default repositories configured, but several other configurations too.

Local Repositories

- Local repository — a name for that central folder in your local system, where the downloaded artifacts of Maven libraries (both 'external' dependencies and those created by us) reside.
- This is the central place in your system where you'll find all the Maven libraries' artifacts that one of your projects used at a point. Thus, it acts like a cache (similar to a 'repository manager').
- When we add a dependency and build the project with Maven. Maven first checks if the specified version of the library is present in our 'local repository'. If yes, we just saved a lot of time and network in not having to download the same dependency again from the remote repository! And if not, it downloads that dependency into the local repository, thus removing the need to download it the next time.
- So, the idea is simple. 'Local repository' is just a repository of artifacts (you can say a 'local copy') downloaded from the 'remote repository'.
- And do note — this being the central folder, the library artifacts present here, can be used by not just the one project we built, but every project we built or will be building locally in the future and which uses the same dependency.
- The local repo exists in ~/.m2 (in Linux & Mac). Thus, the local repository is popularly known as the .m2 directory.And the directory structure of this repository is again interesting.

```
<groupId>io.github.resilience4j</groupId>
<artifactId>resilience4j-circuitbreaker</artifactId>
<version>1.7.1</versionId>
```

- The jar for this library would be present at the following path — ~/.m2/repository/io/github/resilience4j/resilience4j-circuitbreaker/1.7.1/
- Whenever you need to add a new dependency/library to your project, add it to `<dependencies>` in the POM -> run mvn compile in the directory containing the POM -> maven checks if the specific version of the artifact is already present in the .m2 directory a.k.a. the local repository -> If yes, great! Nothing more is to be done. If not, it checks the `<repositories>` tag in order to connect to and download the library artifact from the remote repository into the local one -> Done! Go ahead and import the library in your application and begin using it.

The `<dependencyManagement>` tag

- When you add a dependency to the `<dependencies>` tag without declaring the version, you will get the very version of the dependency which has been associated with that dependency under the

`<dependencyManagement>` tag. Thus through `<dependencyManagement>`, you are instructing Maven to use the stated version of a dependency, “in case” the user decides to use that dependency.

- It is similar to menus in a restaurant where the ingredients are listed below each dish (okay, not all menus mention the ingredients; most don’t, but we’re talking about the ones that do). In this tag, you define, that if you are using the dependencyX, use versionY of that dependencyX, as shown below:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>groupIdOfDependencyX</groupId>
      <artifactId>artifactIdOfDependencyX</artifactId>
      <version>versionYofDependencyX</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

- But remember that adding a dependency here, doesn’t include that dependency into your project. The listing of food items with ingredients in a menu doesn’t imply that a customer is served the entire menu. Its only purpose is to serve as a reference with regard to the ingredients that will be used in a dish, “if” the customer chooses to order it. When the customer does place an order for certain dishes, it’s then that those dishes will be cooked with the very ingredients that were stated under it in the menu.

- But if the version of a library has not been defined in the `<dependencyManagement>` tag, and you decide to use that library, you will need to mention the version of that library yourself directly under `<dependencies>`, without which the build will fail. You can also take advantage of this fact to override the version of a certain dependency configured under `<dependencyManagement>` by specifying your required version of that dependency directly under `<dependencies>`.

- On a side note, not just the versions of dependencies, but even other configurations that can be added under `<dependencies>`, such as the `<exclusions>` cascade from dependencyManagement to dependencies; which means that, if you have excluded a library from a dependency in the `<dependencyManagement>` section, you don’t have to exclude the same library from that dependency again when listing it under `<dependencies>`.

- So what’s the purpose of the `<dependencyManagement>` tag when you can those very things directly under the `<dependencies>` tag. Simple. To avoid repetitions of dependency configurations and to centralize them. You don’t have to update the version of a dependency in multiple places (modules) every time it needs to be changed. This will be clearer once you go through the next section — BOM.

BOM

- POM is now massive. It has the menu, the ingredients, the orders placed, the food — and it has become a big mess of a thing. That being the case, let’s move an entire section out of the POM, into the BOM.

- What is the BOM? The name is self-evident — it is just a 'Bill Of Materials'. Which section did we speak of moving out of the POM into the BOM? The `<dependencyManagement>`! That simple. It is just a POM, but with only that section. In simple words, BOM is a name given to a POM with just the `<dependencyManagement>` section. Thus, just like any other POM, it is and independent file of its own, and is identified via its own groupId, artifactId and version. This way, now our POM is much cleaner.

- We moved our entire `<dependencyManagement>` section into an entirely different file — the bom.xml a.k.a. the BOM. Now how do we tell our POM to use the `<dependencyManagement>` section of this BOM? Just like you would add any other dependency, add this to your POM:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>theGroupIdOfTheBOM</groupId>
      <artifactId>theArtifactIdOfTheBOM</artifactId>
      <version>theVersionOfTheBOMYouWantToUse</version>
      <scope>import</scope>
      <type>pom</type>
    </dependency>
  </dependencies>
</dependencyManagement>
```

- BOM can now be used by more than just one project.

- If you don't specify a library version in your POM or imported BOM, it is defined in a higher-level BOM that the initial BOM imports, requiring you to trace back through the chain to find the version.

Parent POM

Suppose you have a multi-module Maven project, and you have several things in common to the POMs of all the modules. This might include not just the dependencies, but several other sections of the POM, such as the properties, the plugin configuration and the repositories specification. Instead of having to repeat these configurations in the POMs of each of these modules, create a parent POM of all of them. And move the configurations common to all those modules into this one parent POM of theirs.

Another benefit of the parent POM is that, running a maven command on the parent POM, runs the command on all its child modules too.

- Create a pom.xml at the very root of your multi-module project. This will be the parent POM of all the modules of that project.

    1. Add the packaging type as 'pom'.

    2. Under the `<modules>` tag, list all the child modules of the multi-module project.

```
<packaging>pom</packaging>
<modules>
    <module>childModuleA</module>
```

```
    <module>childModuleB</module>
</modules>
```

- In the pom.xml of each of the child modules:

    1. Specify their `<parent>` as the parent POM we created above:

```
<parent>
<groupId>parentPOMsGroupId</groupId>
<artifactId>parentPOMsArtifactId</artifactId>
<version>version,OfCourse</version>
</parent>
```

BOM vs Parent POM

- In case of a BOM, we are mainly sharing the `<dependencyManagement>` section amongst the POMs that import it. While in the case of a parent POM, we are sharing not just the managed dependencies, but several other configurations of the pom.xml.
- a BOM contains something that can be used amongst several projects. While the parent POM's usefulness and role is limited to the project it is created in.
- And finally, you can have only one parent, but import multiple BOMs into your project.
- Managing dependency versions in your POM can lead to conflicts if a BOM already manages those versions, as indirect version management through transitive dependencies can cause issues if incompatible versions are specified, hence it's advisable to rely on the BOM's versions unless necessary.
- A transitive dependency in Java refers to a situation where
    - Project P depends on Library A.
    - Library A depends on Library B.
- Library B is a transitive dependency for Project P. This means that when Project P includes Library A, it also includes Library B automatically.

Dependency Tree

- If you have no idea where a certain dependency is coming from:
    - Run mvn dependency:tree, and you can see the entire tree of dependencies of your project — not just the direct dependencies, but even the transitive dependencies of those dependencies.
- This can be used in cases where a certain dependency has vulnerabilities and you are unable to track its origin.
- On a side note, if different versions of the same dependency is brought in through different dependencies, only that version of a dependency will be used, which is closest to the project, in the

dependency tree.

- For instance, in the example cited in the previous section, versionB1 of depB was brought in by depA, and versionB2 of the same depB was directly added as a dependency in our pom.xml. In that case, the version specified directly in our POM, i.e. versionB2 is used, since that version of depB is closest in the dependency tree.
- Thus if you want a specific version of a library, include it directly under `<dependencies>` in the POM, since it is the nearest definition.

Exclusion

To exclude a vulnerable library brought in as a transitive dependency, use the `<exclusions>` tag within the `<dependencies>` section. For example:

```
<dependencies>
  <dependency>
    <groupId>groupIdOfDepBringingInTheVulnerableDep</groupId>
    <artifactId>artifactIdOfThatDep</groupId>
    <exclusions>
      <exclusion>
        <groupId>groupIdOfTheVulnerableLibrary</groupId>
        <artifactId>artifactIdOfVulnerableLibrary</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
</dependencies>
```

This approach may cause functionality to break if the excluded dependency is needed by your project or its dependencies. If so, explicitly include the same library in project's dependencies with a version that is known to be free of vulnerabilities. This ensures that the project still has access to the necessary library, but with a secure version.

Build Lifecycle (validate, compile, test, package, install, deploy)

Maven is based around the central concept of a build lifecycle. What this means is that the process for building and distributing a particular artifact (project) is clearly defined.

- For the person building a project, this means that it is only necessary to learn a small set of commands to build any Maven project, and the POM will ensure they get the results they desired.
- There are three most common built-in build lifecycles: default(build), clean and site.
    - The default/build lifecycle handles your project deployment
    - The clean lifecycle handles project cleaning
    - The site lifecycle handles the creation of your project's web site.
- Each of these build lifecycles is defined by a different list of build phases, wherein a build phase represents a stage in the lifecycle.

- For example, the default lifecycle comprises of the following phases :
    - validate - validate the project is correct and all necessary information is available
    - verify - run any checks on results of integration tests to ensure quality criteria are met
    - install - install the package into the local repository, for use as a dependency in other projects locally
    - deploy - done in the build environment, copies the final package to the remote repository for sharing with other developers and projects.
- These lifecycle phases are executed sequentially to complete the default lifecycle. Given the lifecycle phases above, this means that when the default lifecycle is used, Maven will first validate the project, then will try to compile the sources, run those against the tests, package the binaries (e.g. jar), run integration tests against that package, verify the integration tests, install the verified package to the local repository, then deploy the installed package to a remote repository.

Predefined Goals

Goals are specific tasks that are bound to these phases. For example, the compiler:compile goal is bound to the compile phase, and the surefire:test goal is bound to the test phase.

- Each phase of the lifecycle is associated with one or more goals.
- While many goals are predefined and bound to specific phases by default, you can customize this in your pom.xml file by specifying plugins and binding their goals to specific phases.

Maven Lifecycle Phases

- Maven defines a series of lifecycle phases in a specific order. When you execute a phase, Maven automatically runs all preceding phases.
- When you run mvn clean install, you are specifying the install phase in the default lifecycle. Maven interprets this as an instruction to execute not just the install phase, but all the preceding phases as well.

Plugins

- Maven goals represent specific tasks within the build lifecycle, and plugins provide the implementations for those tasks. Together, they allow you to automate complex tasks, manage dependencies, generate documentation, and integrate with external tools.
- Maven is actually a plugin execution framework(execute plugins) where every task is actually done by plugins. Maven Plugins are generally used to:
    - Testing: Running unit tests using the Surefire Plugin.
    - Packaging: Creating JAR, WAR, or other packages using plugins like the JAR Plugin or WAR Plugin.
    - Deployment: Deploying artifacts to a remote repository using the Deploy Plugin. Configuring Plugins
    - Documentation: create project documentation
- A plugin generally provides a set of goals, which can be executed using the following syntax

```
mvn [plugin-name]:[goal-name]
```

For example, a Java project can be compiled with the maven-compiler-plugin's compile-goal by running the following command.

```
mvn compiler:compile
```

- Plugins are configured in the pom.xml file under the `<build>` section. Each plugin configuration can include specific goals, executions, and custom settings to control their behavior during the build process.

Compiler Plugin

```
<!--
 * It sets the source and target compatibility to Java 8 (1.8).
 -->
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
            <configuration>
                <source>1.8</source>
                <target>1.8</target>
            </configuration>
        </plugin>
    </plugins>
</build>
```

- target: Specifies the version of the bytecode (e.g., Java 1.8).
- Usage: This plugin is essential for compiling Java source files into .class files.

Surefire Plugin

The Maven Surefire Plugin is used for running unit tests. It is bound to the test phase by default. It executes the tests during the test phase of the build lifecycle and generates reports based on the test results.

```
<!-- This configuration is for the Maven Surefire Plugin. It specifies the
includes patterns for test classes to be executed during the build
process.  -->
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
```

```
            <artifactId>maven-surefire-plugin</artifactId>
            <version>2.22.2</version>
            <configuration>
                <includes>
                    <include>**/Test*.java</include>
                    <include>**/*Test.java</include>
                    <include>**/*Tests.java</include>
                    <include>**/*TestCase.java</include>
                </includes>
            </configuration>
        </plugin>
    </plugins>
</build>
```

- includes: Specifies which test classes to include. This example includes various naming conventions for test classes.

JAR Plugin

```
<!--
 * It includes the Maven Jar Plugin configuration, which is used to create
a JAR file for the project.
 * The plugin is configured to add default implementation entries to the
JAR manifest.
 -->
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-jar-plugin</artifactId>
            <version>3.1.0</version>
            <configuration>
                <archive>
                    <manifest>

<addDefaultImplementationEntries>true</addDefaultImplementationEntries>
                    </manifest>
                </archive>
            </configuration>
        </plugin>
    </plugins>
</build>
```

- archive: Allows for customization of the JAR file's contents.
- manifest: Contains entries for the JAR manifest file.
- A JAR (Java ARchive) manifest file is a special metadata file included in JAR files, which are used to bundle Java applications and libraries. The manifest file provides information about the contents of the JAR file and configuration data needed by the application or library.

- addDefaultImplementationEntries: Adds default entries to the manifest file, such as version and implementation details.
- By using and configuring Maven plugins, you can tailor the build process to meet the specific needs of your project, ensuring efficient and consistent build outcomes.
- Maven has a vast ecosystem of plugins that cover a wide range of tasks and integrations.
- Maven resolves and downloads plugins from remote repositories when needed. Maven will then download the plugin if not available in local repository and start its processing.

Check out this link

POM Packaging vs JAR

- POM Packaging refers to a Maven project that is solely used for **managing dependencies**, **plugins**, and other modules. This type of project doesn't produce a JAR or WAR file; instead, it defines the structure and dependencies for a multi-module Maven project.
- JAR Packaging is the default packaging for Java projects. When you build a Maven project with JAR packaging, it produces a .jar file containing the compiled classes, resources, and metadata. This JAR can be executed or included as a dependency in other projects.

Module Export Packages

In Java 9 and later, the module system was introduced to provide **strong encapsulation and dependency management**. A **module can export packages**, making them available to other modules.

```
module my.module {
    exports com.example.myapp;
}
```

- This exports statement makes the com.example.myapp package available to other modules that require my.module.
- When you export a package from a module, you're making its public types (classes, interfaces, etc.) accessible to other modules. This is important when you want to define a clear API for your module that other modules can use while keeping the internal implementation details hidden.
- One of the key goals of the Java module system is to provide strong encapsulation. By default, the types in a module are not accessible outside the module unless they are in a package that is explicitly exported
- When you define a module, you might depend on other modules, and those modules might need to depend on yours. By exporting certain packages, you allow other modules that require your module to access specific parts of your module's functionality.
- Exporting packages allows you to explicitly define what is part of your module's public API and what is not. This leads to a cleaner, more maintainable codebase, where only the necessary parts of a module are exposed.

Security

At its core, Spring Security is really just a bunch of servlet filters that help you add authentication and authorization to your web application.

It also integrates well with frameworks like Spring Web MVC (or Spring Boot), as well as with standards like OAuth2 or SAML. And it auto-generates login/logout pages and protects against common exploits like CSRF.

Securing REST APIs with Spring Security

Without proper security measures, Project APIs can become targets for attacks, leading to unauthorized access and data breaches. Spring Security is a powerful and customizable authentication and access control framework for Java applications that can be used to secure REST APIs.

Steps and Components OF securing an applction:

- Authentication: It is the process of verifying the identity of the user or system. Common methods include the username/password, OAuth2, and JWT.
- Authorization: It is the process of granting or denying access to resources based on the authenticated identity of the permissions or roles of the application.
- Token-Based Authentication: Using the tokens like JWT to handle the authentication. This is especially useful for the stateless APIs.
- Role-Based Access Control (RBAC): It defines the access control based on the user roles and permissions of the application.
- Security Filters: Using the filters to the intercept requests and apply the security checks of the application.

Authentication

First off, if you are running a typical (web) application, you need your users to authenticate. That means your application needs to verify if the user is who he claims to be, typically done with a username and password check.

- Definition: The process of verifying the identity of a user or system.
- Purpose: Ensures that the entity is who they claim to be.

Authorization

In simpler applications, authentication might be enough: As soon as a user authenticates, she can access every part of an application. Customer and Administrators need to login, but the mere fact of authentication doesn’t say anything about what each are allowed to do in your system. Hence, you also need to check the permissions of an authenticated user, i.e. you need to authorize the user.

- Definition: The process of determining whether an authenticated entity has permission to perform a specific action or access certain resources.
- Purpose: Ensures that the entity has the right to access the requested resource or perform the action.

Token (JWT - JSON Web Token)

Tokens, specifically JSON Web Tokens (JWT), are a compact and self-contained way to securely transmit information between parties as a JSON object. JWTs are commonly used for authentication and information exchange.

A JWT is divided into three parts:

1. Header: The header typically consists of two parts: the type of token, which is JWT, and the signing algorithm being used, such as HMAC SHA256 or RSA.

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload: The payload contains the claims, which are statements about an entity (typically, the user) and additional data. There are three types of claims: registered, public, and private claims.

- Registered claims include standardized fields like iss (issuer), exp (expiration time), sub (subject), and aud (audience).

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true
}
```

3. Signature: To create the signature part, you have to take the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that.

The final JWT looks like this:

Refresh Token

Refresh Tokens are used to obtain a new access token once the original access token expires. They are issued alongside the access token and are typically long-lived, whereas access tokens are short-lived.

- When a user logs in, the server issues both an access token and a refresh token.
- The access token is used for authenticating API requests and has a short expiration time (e.g., 15 minutes).
- When the access token expires, the client can use the refresh token to request a new access token without requiring the user to re-authenticate.
- Refresh tokens should be stored securely, typically in an HttpOnly cookie, to prevent XSS attacks.

- Rotating Refresh Tokens: Each time a refresh token is used, a new one is issued along with the new access token. The old refresh token is invalidated, reducing the risk of token theft.
- Revoke on Logout: When a user logs out, both the access token and refresh token should be revoked or invalidated.

Outh2

Key Roles and Components

- **Resource Owner (User)**: The individual who owns the data and grants permission.
- **Client (App)**: The application requesting access to the user's data (e.g., a third-party fitness app).
- **Authorization Server (Google)**: The server that handles the authorization process and issues tokens (e.g., Google's OAuth 2.0 server).
- **Resource Server (Google APIs)**: The server hosting the protected resources (e.g., Google Calendar API, Google Drive API).

Detailed Flow

Authorization Request

- **Description**: The client (app) requests authorization from the resource owner (user) to access their Google account data.
- **Flow**:
  - User clicks a "Login with Google" button on the app.
  - The app redirects the user to Google's authorization endpoint with a request URL that includes client ID, redirect URI, scope, response type, and state.
  - Example URL:

```
https://accounts.google.com/o/oauth2/v2/auth?
client_id=YOUR_CLIENT_ID&
redirect_uri=YOUR_REDIRECT_URI&
response_type=code&
scope=profile email&
state=xyz
```

Authorization Grant

- **Description**: The resource owner (user) logs into their Google account and grants permission to the app to access the requested scopes.
- **Flow**:
  - User logs into Google and sees a permission screen.
  - User clicks "Allow" to grant permission.
  - Google redirects the user back to the app's specified redirect URI with an authorization code.
  - Example of redirect URI with authorization code:

```
https://yourapp.com/callback?code=AUTHORIZATION_CODE&state=xyz
```

Access Token Request

- **Description**: The client (app) exchanges the authorization code for an access token by making a request to the authorization server (Google).
- **Flow**:
  - App sends a POST request to Google's token endpoint with the authorization code, client ID, client secret, redirect URI, and grant type.
  - Example POST request:

```
POST /oauth2/v4/token HTTP/1.1
Host: www.googleapis.com
Content-Type: application/x-www-form-urlencoded

code=AUTHORIZATION_CODE&
client_id=YOUR_CLIENT_ID&
client_secret=YOUR_CLIENT_SECRET&
redirect_uri=YOUR_REDIRECT_URI&
grant_type=authorization_code
```

Access Token Response

- **Description**: The authorization server (Google) validates the authorization code and issues an access token (and optionally a refresh token).
- **Flow**:
  - Google responds with an access token in JSON format.
  - Example response:

```
{
  "access_token": "ACCESS_TOKEN",
  "expires_in": 3600,
  "token_type": "Bearer",
  "refresh_token": "REFRESH_TOKEN"
}
```

Access Protected Resource

- **Description**: The client (app) uses the access token to access the resource server (Google APIs) on behalf of the user.
- **Flow**:
  - App makes an authenticated request to Google APIs using the access token.
  - Example GET request to Google Calendar API:

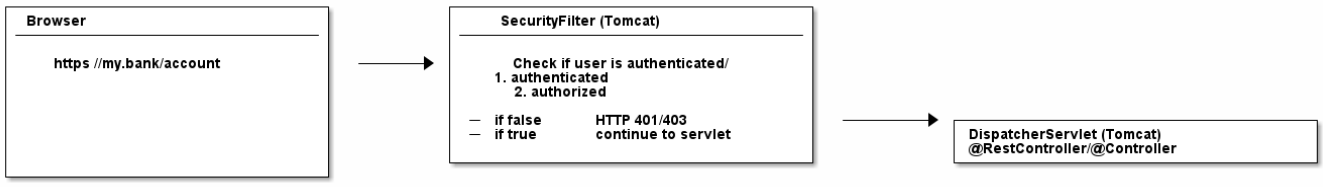
```
GET /calendar/v3/calendars/primary/events HTTP/1.1
Host: www.googleapis.com
```

```
Authorization: Bearer ACCESS_TOKEN
```

Servlet Filters

- Basically any Spring web application is just one servlet: Spring's good old `DispatcherServlet`, that redirects incoming HTTP requests (e.g. from a browser) to your @Controllers or @RestControllers.
- The thing is: There is no security hardcoded into that DispatcherServlet and you also very likely don't want to fumble around with a raw HTTP Basic Auth header in your @Controllers. Optimally, the authentication and authorization should be done before a request hits your @Controllers.

Luckily, there's a way to do exactly this in the Java web world: you can put filters in front of servlets, which means you could think about writing a SecurityFilter and configure it in your Tomcat (servlet container/application server) to filter every incoming HTTP request before it hits your servlet.



A naive SecurityFilter

A SecurityFilter has roughly 4 tasks and a naive and overly-simplified implementation could look like this:

```
public class SecurityServletFilter extends HttpFilter {

    @Override
    protected void doFilter(HttpServletRequest request,
HttpServletResponse response, FilterChain chain) throws IOException,
ServletException {

        UsernamePasswordToken token =
extractUsernameAndPasswordFrom(request);  // (1)

        if (notAuthenticated(token)) {  // (2)
            // either no or wrong username/password
            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED); //
HTTP 401.
            return;
        }

        if (notAuthorized(token, request)) { // (3)
            // you are logged in, but don't have the proper rights
            response.setStatus(HttpServletResponse.SC_FORBIDDEN); // HTTP
403
            return;
        }
```

```
        // allow the HttpRequest to go to Spring's DispatcherServlet
        // and @RestControllers/@Controllers.
        chain.doFilter(request, response); // (4)
    }
}
```

FilterChains

check link FilterChains

Spring Security uses a default set of 15 filters, which handle various aspects of authentication and authorization. To configure these filters, use WebSecurityConfigurerAdapter @Configuration class, where you specify which URLs to protect, which to ignore, and how to handle authentication. It also configure Spring Security.

1. annotated with @EnableWebSecurity.
2. Extends WebSecurityConfigurer, which basically offers you a configuration DSL/methods. With those methods, you can specify what URIs in your application to protect or what exploit protections to enable/disable.

```
@Configuration
@EnableWebSecurity // (
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {

  // By overriding the adapter's configure(HttpSecurity) method, you get a
nice   little DSL (Domain-Specific Language) with which you can configure
your FilterChain.
  @Override
  protected void configure(HttpSecurity http) throws Exception {
      http
      // All requests going to / and /home are allowed (permitted) - the
user does not have to authenticate. You are using an antMatcher, which
means you could have also used wildcards (*, \*\*, ?) in the string.
        .authorizeRequests()
          .antMatchers("/", "/home").permitAll()
          //    Any other request needs the user to be authenticated
first, i.e. the user needs to login.
          .anyRequest().authenticated()
          .and()
       .formLogin()
         .loginPage("/login")
         .permitAll()
         .and()
      .logout()
        .permitAll()
        .and()
      .httpBasic();
  }
}
```

- You are allowing form login (username/password in a form), with a custom loginPage (/login, i.e. not Spring Security's auto-generated one). Anyone should be able to access the login page, without having to log in first (permitAll; otherwise we would have a Catch-22!). The same goes for the logout page
- On top of that, you are also allowing Basic Auth, i.e. sending in an HTTP Basic Auth Header to authenticate.
- Note: You wouldn't have needed to immediately override the adapter's configure method, because it comes with a pretty reasonable implementation - by default. This is what it looks like:

```
public abstract class WebSecurityConfigurerAdapter implements
        WebSecurityConfigurer<WebSecurity> {

    protected void configure(HttpSecurity http) throws Exception {
            http
            // To access any URI (anyRequest()) on your application, you
need to authenticate (authenticated()).
                .authorizeRequests()
                    .anyRequest().authenticated()
                    .and()
                //  Form Login (formLogin()) with default settings is
enabled.
                .formLogin().and()
                // As is HTTP Basic authentication (httpBasic()).
                .httpBasic();
        }
}
```

Authentication with Spring Security

- When it comes to authentication and Spring Security you have roughly three scenarios:
    - The default: Spring Security can access the (hashed) password of the user, because user's details (username, password) saved in e.g. a database table.
    - Less common: You cannot access the (hashed) password of the user. This is the case if your users and passwords are stored somewhere else, like in a 3rd party identity management product offering REST services for authentication. Think: Atlassian Crowd.
    - Also popular: You want to use OAuth2 or "Login with Google/Twitter/etc." (OpenID), likely in combination with JWT.

Note: Depending on your scenario, you need to specify different @Beans to get Spring Security working, otherwise you'll end up getting pretty confusing exceptions (like a NullPointerException if you forgot to specify the PasswordEncoder). Keep that in mind.

Having access to the user's password

Using Spring Security and have access to the user's password, then:

- Specify a UserDetailsService. Either a custom implementation or use and configure one that Spring Security offers.

```
@Bean
public UserDetailsService userDetailsService() {
    // MyDatabaseUserDetailsService implements UserDetailsService, a very
simple interface, which consists of one method returning a UserDetails
object
    return new MyDatabaseUserDetailsService();
}

public class MyDatabaseUserDetailsService implements UserDetailsService {

    UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException { // (1)
         // 1. Load the user from the users table by username. If not
found, throw UsernameNotFoundException.
         // 2. Convert/wrap the user to a UserDetails object and return
it.
        return someUserDetails;
    }
}
```

- Specify a PasswordEncoder => If you want to, say, use the BCrypt password hashing function (Spring Security's default) for all your passwords, you would specify this @Bean in your SecurityConfig.

```
@Bean
public BCryptPasswordEncoder bCryptPasswordEncoder() {
    return new BCryptPasswordEncoder();
}
```

AuthenticationProvider: Not having access to the user's password

Now, imagine that you are using Atlassian Crowd for centralized identity management. That means all your users and passwords for all your applications are stored in Atlassian Crowd and not in your database table anymore.

- You have a REST API that you can login against, with your username and password. (A POST request to the /rest/usermanagement/1/authentication REST endpoint).
- You cannot use a UserDetailsService anymore, instead you need to implement and provide an AuthenticationProvider @Bean.

```
@Bean
public AuthenticationProvider authenticationProvider() {
        return new AtlassianCrowdAuthenticationProvider();
}
```

```
// An AuthenticationProvider consists primarily of one method and a naive
implementation could look like this:

public class AtlassianCrowdAuthenticationProvider implements
AuthenticationProvider {

        Authentication authenticate(Authentication authentication)  // (1)
                throws AuthenticationException {
            // Extract the username/password combination from the HTTP
Basic Auth header in a filter.

            // Call your AuthenticationProvider (e.g.
AtlassianCrowdAuthenticationProvider) with that username and password for
you to do the authentication (e.g. REST call)
            User user = callAtlassianCrowdRestService(username, password);
            if (user == null) {
                throw new Exception("could not login");
            }
            // If authentication succeeded, you need to return a fully
initialized UsernamePasswordAuthenticationToken. It is an implementation
of the Authentication interface and needs to have the field authenticated
be set to true (which the constructor used above will automatically set).
            return new
UserNamePasswordAuthenticationToken(user.getUsername(),
user.getPassword(), user.getAuthorities()); // (4)

        }
}
```

- The takeaway for this is: if you are using Spring Security and do not have access to the user's password, then implement and provide an AuthenticationProvider @Bean.

Authorization with Spring Security

- An authority (in its simplest form) is just a string, it can be anything like: user, ADMIN, ROLE_ADMIN or 53cr37_r0l3.
- A role is an authority with a ROLE_ prefix. So a role called ADMIN is the same as an authority called ROLE_ADMIN.
- The distinction between roles and authorities is purely conceptual and something that often bewilders people new to Spring Security.

GrantedAuthorities nd SimpleGrantedAuthorities?

Of course, Spring Security doesn't let you get away with just using Strings. There's a Java class representing your authority String, a popular one being SimpleGrantedAuthority.

```
public final class SimpleGrantedAuthority implements GrantedAuthority {
```

```
    private final String role;

    @Override
    public String getAuthority() {
        return role;
    }
}
```

- Note: There's other authority classes as well, that let you store additional objects (e.g. the principal) alongside your string.
- Using Users table, Add a column called "authorities" to it. It could be a completely separate table AUTHORITIES, but for the scope of this article this will do.

username	password	authorities
john@doe.com	{bcrypt}	ROLE_ADMIN
my@callcenter.com	{sha256}	ROLE_CALLCENTER

- The only thing that's left to do is to adjust your UserDetailsService to read in that authorities column.

```
public class MyDatabaseUserDetailsService implements UserDetailsService {

  UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
     User user = userDao.findByUsername(username);
     // simply map whatever is inside your database column to a list of
SimpleGrantedAuthorities.
     List<SimpleGrantedAuthority> grantedAuthorities =
user.getAuthorities().map(authority -> new
SimpleGrantedAuthority(authority)).collect(Collectors.toList());
     return new
org.springframework.security.core.userdetails.User(user.getUsername(),
user.getPassword(), grantedAuthorities);
  }

}
```

AuthenticationManager: Where to store and get authorities?

- When the users comes from a third-party application, like Atlassian Cloud, you'll need to find out what concept they are using to support authorities. Atlassian Crowd had the concepts of "roles", but deprecated it in favour of "groups".
- So, depending on the actual product you are using, you need to map this to a Spring Security authority, in your AuthenticationProvider.

```
public class AtlassianCrowdAuthenticationProvider implements
AuthenticationProvider {
```

```
    Authentication authenticate(Authentication authentication)
            throws AuthenticationException {
        String username = authentication.getPrincipal().toString();
        String password = authentication.getCredentials().toString();

        atlassian.crowd.User user =
callAtlassianCrowdRestService(username, password);
        if (user == null) {
            throw new AuthenticationException("could not login");
        }
        return new UserNamePasswordAuthenticationToken(user.getUsername(),
user.getPassword(), mapToAuthorities(user.getGroups()));
    }
}
```

WebSecurityConfigurerAdapter for Authorities

- To protect URLs with different authorities with Spring Security's DSL :

```
@Configuration
@EnableWebSecurity
public class WebSecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
          .authorizeRequests()
            // To access the /admin area you (i.e. the user) need to be
authenticated AND have the authority (a simple string) ROLE_ADMIN.
            .antMatchers("/admin").hasAuthority("ROLE_ADMIN")
            // To access the /callcenter area you need to be authenticated
AND have either the authority ROLE_ADMIN OR ROLE_CALLCENTER.
            .antMatchers("/callcenter").hasAnyAuthority("ROLE_ADMIN",
"ROLE_CALLCENTER")
            // For any other request, you do not need a specific role but
still need to be authenticated.
            .anyRequest().authenticated()
            .and()
         .formLogin()
           .and()
         .httpBasic();
    }
}
```

```
  http
    .authorizeRequests()
    // Instead of calling "hasAuthority", you now call "hasRole".
```

```
        .antMatchers("/admin").hasRole("ADMIN")
        .antMatchers("/callcenter").hasAnyRole("ADMIN", "CALLCENTER")
```

- Last, but not least, the most powerful way to configure authorizations, is with the access method. It lets you specify pretty much any valid SpEL expressions.

```
  http
    .authorizeRequests()
      .antMatchers("/admin").access("hasRole('admin') and
hasIpAddress('192.168.1.0/24') and
@myCustomBean.checkAccess(authentication,request)")
```

Common Exploit Protections

There is a variety of common attacks that Spring Security helps you to protect against. It starts with timing attacks (i.e. Spring Security will always hash the supplied password on login, even if the user does not exist) and ends up with protections against cache control attacks, content sniffing, click jacking, cross-site scripting and more.

Cache Control Attacks

- Sensitive data can be stored in the browser cache and accessed later by unauthorized users.
- Spring Security Protection:
  - Cache-Control Headers: Spring Security can set HTTP headers to prevent caching of sensitive information.

```
http.headers().cacheControl();
```

Content Sniffing

- Browsers may incorrectly interpret the MIME type of responses, potentially leading to the execution of malicious scripts.
- MIME Type: A label used to identify the type of data a file contains. It consists of a type and a subtype, separated by a slash (/), such as text/html or image/png.
- Spring Security Protection:
  - X-Content-Type-Options Header: Spring Security can set this header to nosniff, preventing browsers from MIME-sniffing.

```
http.headers().contentTypeOptions();
```

Clickjacking

- A malicious site can embed a legitimate site in an iframe, tricking users into clicking on elements within the iframe.
- An iframe (short for Inline Frame) is an HTML element that allows you to embed another HTML document within the current document. Essentially, it creates a nested browsing context, enabling one webpage to be displayed within another. This can be useful for embedding videos, advertisements, external content, or other web pages within a page.
- Spring Security Protection:
    - X-Frame-Options Header: Spring Security can set this header to prevent the page from being displayed in an iframe.

```
http.headers().frameOptions().deny();
```

Cross-Site Scripting (XSS):

- XSS involves injecting malicious scripts into web pages viewed by other users. These scripts can steal data, manipulate the DOM, or perform actions on behalf of the user.
- Spring Security Protection:
    - X-XSS-Protection Header: Spring Security can set this header to enable the browser's XSS filter.

```
http.headers().xssProtection();
```

Cross-Site-Request-Forgery: CSRF

A Cross-Site Request Forgery (CSRF) attack occurs when a malicious web site, email, blog, instant message, or program tricks an authenticated user's web browser into performing an unwanted action on a trusted site. If a target user is authenticated to the site, unprotected target sites cannot distinguish between legitimate authorized requests and forged authenticated requests.

Since browser requests automatically include all cookies including session cookies, this attack works unless proper authorization is used, which means that the target site's challenge-response mechanism does not verify the identity and authority of the requester. In effect, CSRF attacks make a target system perform attacker-specified functions via the victim's browser without the victim's knowledge (normally until after the unauthorized actions have been committed).

However, the quick takeaway is, that by default Spring Security protects any incoming POST (or PUT/DELETE/PATCH) request with a valid CSRF token.

Typical CSRF Attack Flow

- User Authentication:
    - The user logs into a legitimate website (Site A) and receives a session cookie.
- Malicious Request:

    - While logged in, the user visits a malicious website (Site B).
    - Site B contains a hidden form or a script that makes a request to Site A, exploiting the user's session.

- Request Execution:

    - The browser sends the request to Site A with the user's credentials.
    - If Site A does not validate the request properly (e.g., by checking for a CSRF token), the action is performed without the user's consent.

- CSRF attacks manipulate existing sessions and do not involve accessing the memory of the original page directly.

How CSRF Protection Works in Restless/stateless application

1. CSRF Token Generation and Storage
    - Token Generation: When a user first accesses the application, the server generates a CSRF token.
    - Token Storage: This token is stored in a cookie named XSRF-TOKEN and sent to the client.

- Server-side Configuration:

```
@Configuration
public class SpringSecurityConfiguration {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {
        // This configuration ensures the CSRF token is sent as a cookie
and can be accessed by JavaScript.

http.csrf().csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFals
e());
        return http.build();
    }
}
```

2. Client-side Token Retrieval

    - The client-side application retrieves the CSRF token from the cookie.
    - This token is then included in the headers of any state-modifying requests (POST, PUT, DELETE, PATCH).

3. Token Inclusion in Requests

    - The retrieved token is sent with each request in a header named X-XSRF-TOKEN.

4. Server-side Token Validation

    - When the server receives a request, it compares the token in the X-XSRF-TOKEN header with the one stored in the cookie.

    - If the tokens match, the request is considered valid. If they do not match, the server rejects the request.

Why It's Secure

- Token Storage in Cookie: The CSRF token is stored in a cookie that is only sent with requests to your server. This prevents external websites from accessing it.
- Different Headers: By using a custom header (X-XSRF-TOKEN), the server can differentiate between legitimate requests from your application and potentially malicious requests from other sites.
- The Same-Origin Policy is a fundamental security mechanism that restricts how documents or scripts loaded from one origin can interact with resources from another origin.
- It prevents a malicious website from reading or manipulating the cookies set by your domain.
- An origin is defined by the scheme (http/https), hostname, and port number.
- Example:
    - User Logs into Your Website: The user logs into https://yourwebsite.com, and the server sets a XSRF-TOKEN cookie.
    - User Visits Malicious Site: The user is tricked into visiting https://attacker.com.
    - Malicious Request: The malicious site attempts to make a request to https://yourwebsite.com on behalf of the user.
    - Token Validation: - The request from https://attacker.com does not have access to the XSRF-TOKEN cookie because of the Same-Origin Policy. - The server at https://yourwebsite.com checks for the presence of the valid CSRF token in the request header. -Since the malicious request lacks the correct CSRF token, the server rejects the request.
- The Same-Origin Policy isolates resources between different origins, preventing malicious websites from accessing cookies and other resources from your site.
- The CSRF token mechanism ensures that only requests from your domain, with the correct token, are processed by your server.
- Cookies are restricted to and only accessible by the domain that originally set them.

In a stateless REST API, CSRF protection is typically not necessary

- In a stateless REST API, CSRF protection is typically not necessary because the API does not rely on cookies or sessions to maintain user state. Instead, REST APIs often use tokens (such as JWTs) sent in the request headers for authentication, which are not vulnerable to CSRF attacks. Therefore, you can disable CSRF protection in such cases. Here is how you would do it in Spring Security:

```
@Configuration
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .csrf().disable() // Disable CSRF protection
            .authorizeRequests()
            .anyRequest().authenticated() // Require authentication for
all requests
            .and()
            .httpBasic(); // Enable basic authentication
```

```
    }
}
```

Session-Centric Applications:

- In traditional web applications, HTTP sessions are often used to manage user state. When a user first accesses the application, a session is created, and a session ID is typically stored in a cookie.
- Spring Security generates a CSRF token and associates it with the session. This token must be included in any subsequent POST (or PUT/DELETE/PATCH) requests to the server.
- The CSRF token helps prevent CSRF attacks by ensuring that the request originated from a legitimate user and not from a malicious site.

CSRF Protection with Spring MVC and Thymeleaf

Imagine a bank transfer form or any form (like a login form) for that matter, that gets rendered by your @Controllers with the help of a templating technology like Thymeleaf or Freemarker.

```
<form action="/transfer" method="post">  <!-- 1 -->
  <input type="text" name="amount"/>
  <input type="text" name="routingNumber"/>
  <input type="text" name="account"/>
  <input type="submit" value="Transfer"/>
</form>
```

- With Spring Security enabled, you won't be able to submit that form anymore. Because Spring Security's CSRFFilter is by default looking for an additional hidden parameter on any POST (PUT/DELETE) request: a so-called CSRF token.
- It generates such a token, by default, per HTTP session and stores it there. And you need to make sure to inject it into any of your HTML forms.
- As Thymeleaf has good integration with Spring Security (when used together with Spring Boot), you can simply add the following snippet to any form and you'll get the token injected automatically, from the session, into your form. Even better, in cerntain case Thymeleaf can automatically inject that hidden field for you, without having to do it manually.

```
<form action="/transfer" method="post">
  <input type="text" name="amount"/>
  <input type="text" name="routingNumber"/>
  <input type="text" name="account"/>
  <input type="submit" value="Transfer"/>
  <input type="hidden" name="${_csrf.parameterName}"
value="${_csrf.token}"/>
</form>
```

Password encryption techniques

- Password encryption is essential to store user credentials stored in a database securely. Without password encryption, anyone accessing a user database on a company's servers (including hackers) could easily view any stored passwords.

- Encryption scrambles your password so it's unreadable and/or unusable by hackers before saving it on the server. That simple step protects your password while it's sitting in a server, and it offers more protection as your password zooms across the internet. So, if someone hacks the server, instead of finding password123, they find a random series of letters and numbers.

Imagine that you've created the strongest password possible. Now, imagine that all of your hard work is stored in plain text on your company's server. If a hacker gets inside, what happens next? All of your efforts go to waste, and your username and password are sold on the open market to the highest bidder.

- Password Encryption terminology.
    - Key: Used to lock and unlock passwords using a random string of bits. You get private and public keys that crypt and decrypt data differently.
    - Bits: A logical state with one of two possible values, including 1/0, true/false, yes/no, or on/off as typical examples.
    - Block (block cipher): A deterministic algorithm operating on fixed-length groups of bits, called blocks.
    - Hash: A random series of numbers and letters representing your password. The hash function uses your hash instead of the raw password for authentication.
    - Salt: Additional letters and numbers appended to the hash
    -

Four main types of Encryption

- Symmetric key: Your system has a key for encryption/decryption. Move your password through this key to scramble it, and push it back through the key to make it readable once more. A hacker must steal the key to take over your password.

- Public key: Two keys play a role in altering your password. One, a public key, is available for anyone to use. The other, a private key, is available only to a select few. Use one to encode a message, and the recipient needs the other to make sense of it.

- Hashed: A computer algorithm transforms your password into a random series of numbers and letters. A hacker must know what algorithm you used to change your password, and that isn't always easy to discern.

- Salted: A few random numbers or letters are appended to the beginning or end of your password before it moves through the hashing process. A hacker must know the hash (which isn't always easy) as well as your hashing algorithm to decode the message. Your administrator can use a fixed salt that's the same for every password, or they might use a variable salt that changes with each password. When a password is salted with a random value, the process typically involves storing the salt alongside the hashed password

No matter how your company handles encryption, the result is the same. Your password changes from something you know and can remember into something that looks completely random.

How Salt Works

- A salt appends a unique value of 8 bytes (16 characters) to the password before the hash function creates a hash. This way, even identical passwords are unique before the hash function.

```
Two identical passwords: Pa$$w0rd123
Salt value one: E1F53135E559C253
Salt value two: 84B03D034B409D4E
Password one before hash: Pa$$w0rd123E1F53135E559C253
Password two before hash: Pa$$w0rd12384B03D034B409D4E
Password one hashed value (SHA256): 72AE25495A7981C40622D49F9A52E4F15
65C90F048F59027BD9C8C8900D5C3D8
Password two hashed value (SHA256): B4B6603ABC670967E99C7E7F1389E40
CD16E78AD38EB1468EC2AA1E62B8BED3A
```

5 Common Password Encryption Methods

1. Data Encryption Standard (DES)
    - Applications no longer use Data Encryption Standard (DES).
2. Triple DES [68-bit encryption]
    - Although Triple-DES is slowly being phased out, many financial institutions still use it to encrypt ATM PINs.
3. Advanced Encryption Standard (AES)
    - AES is the new encryption standard—trusted by the United States Government and many other prominent organizations globally.
    - At 128 bits, AES is sufficiently secure, but most organizations prefer heavy-duty 256-bit encryption.
    - Hackers can only break an AES encrypted password through a brute-force attack—trying password combinations to find the right one.
    - To counter brute-force attacks, applications will lock an account after a certain number of attempts.
4. Blowfish [32 to 448 bits]
5. Rivest-Shamir-Adleman (RSA)
    - RSA is one of the oldest and widely used to transfer data securely.
    - Due to the complicated encrypting and decrypting process, there is no known method for breaking RSA encryption.
    - Although widely used for transferring data, RSA is slow and not suitable for password encryption.

Common Password Encryption Formats

- Imagine that our password is R@nT4g*Ne! (Rent Forgone, in common terminology.) Let's pass this through several different encryption tools:
    - SHA-1: Our password is made up of 40 characters, and there is no clear decryption method. Our secure password is 12bf203295c014c580302f4fae101817ec085949.
    - SHA-1 with Salt: Our password is still made up of 40 characters, but we appended the word "Free." Our secure password is bc6b79c7716722cb383321e40f31734bce0c3598.
    - MD5: Our password is encoded into a 128-bit string. Our secure password is 4e84f7e8ce5ba8cdfe99d4ff41dc2d41.
    - AES: Use this symmetric encryption algorithm, and you can choose bit length. It's nearly impossible to model what our completed password might look like, as we have far too many variables to choose one, and each will impact the outcome.
- A persistent hacker can jump through your defenses and access your files. But use systems like this, and you're likely to cause significant delays. Hackers grow frustrated when they can't discover the algorithms you use to protect passwords. They may choose a different target instead.

Method Security

- To secure a RESTful API, you can configure Spring Security to require authentication and authorization for different endpoints. Method-level security allows you to control access to specific methods in your controllers or services based on roles or permissions.
- That approach is called method security and works through annotations that you can basically put on any public method of your Spring beans.
- You also need to explicitly enable method security by putting the @EnableGlobalMethodSecurity annotation on your ApplicationContextConfiguration.

```
@Configuration
@EnableGlobalMethodSecurity(
  prePostEnabled = true, // The prePostEnabled property enables support
for Spring’s @PreAuthorize and @PostAuthorize annotations.
  securedEnabled = true, // The securedEnabled property enables support
for the @Secured annotation.
  jsr250Enabled = true) // The jsr250Enabled property enables support for
the @RolesAllowed annotation.
public class YourSecurityConfig extends WebSecurityConfigurerAdapter{
}


@RestController
public class MyController {

    @GetMapping("/public")
    public String publicEndpoint() {
        return "This is a public endpoint";
    }

    @PreAuthorize("hasRole('ROLE_USER')")
```

```
    @GetMapping("/user")
    public String userEndpoint() {
        return "This is a user-only endpoint";
    }

    @PreAuthorize("hasRole('ROLE_ADMIN')")
    @GetMapping("/admin")
    public String adminEndpoint() {
        return "This is an admin-only endpoint";
    }
}
```

- Support means, that Spring will ignore this annotation unless you set the flag to true.

What is the difference between @PreAuthorize, @Secured and @RolesAllowed?

- All these annotations will raise an AccessDeniedException if you try and access a protected method with an insufficient authority/role.
- @Secured and @RolesAllowed are basically the same, though @Secured is a Spring-specific annotation coming with the spring-security-core dependency and @RolesAllowed is a standardised annotation, living in the javax.annotation-api dependency. Both annotations are used to specify a list of roles on a method. So, a user only can access that method if she has at least one of the specified roles. take in an authority/role string as value.
- The @Secured and @RolesAllowed annotations don't support Spring Expression Language (SpEL).
- @PreAuthorize/@PostAuthorize are also (newer) Spring specific annotations and more powerful than the above annotations, as they can contain not only authorities/roles, but also any valid SpEL expression.
- The @PreAuthorize annotation checks the given expression before entering the method, whereas the @PostAuthorize annotation verifies it after the execution of the method and could alter the result.

```
@Service
public class SomeService {

    // As mentioned, @Secured takes an authority/role as parameter.
@RolesAllowed, likewise. Note: Remember that @RolesAllowed("ADMIN") will
check for a granted authority ROLE_ADMIN.
    @Secured("ROLE_CALLCENTER")
    // == @RolesAllowed("ADMIN")
    // Also can specify a list of roles => @Secured({ "ROLE_VIEWER",
"ROLE_EDITOR" })
    public BankAccountInfo get(...) {

    }

    //  As mentioned, @PreAuthorize takes in authorities, but also any
valid SpEL expression.
    // For a list of common built-in security expressions like
isAnonymous() above, as opposed to writing your own SpEL expressions,
check out the official documentation.
```

```
    // This means that the method trackVisit can only be invoked by
anonymous users. In other words, this method can only be accessed if the
user is not authenticated.
    @PreAuthorize("isAnonymous()")
    // @PreAuthorize("#contact.name == principal.name")
    // @PreAuthorize("ROLE_ADMIN")
    public void trackVisit(Long id);

}
```

- Which annotation should I use? This is mainly a matter of homogenity. To start off, you can always use @Secured and switch to @PreAuthorize as soon as the need arises.

PostAuthorize

```
@PostAuthorize
  ("returnObject.username == authentication.principal.nickName")
public CustomUser loadUserDetail(String username) {
    return userRoleRepository.loadUserByUserName(username);
}
```

- In the previous example, however, the authorization would get delayed after the execution of the target method.
- Additionally, the @PostAuthorize annotation provides the ability to access the method result.
- Here the loadUserDetail method would only execute successfully if the username of the returned CustomUser is equal to the current authentication principal's nickname.
- By default, Spring AOP proxying is used to apply method security. If a secured method A is called by another method within the same class, security in A is ignored altogether. This means method A will execute without any security checking. The same applies to private methods.
- Spring SecurityContext is **thread-bound**. By default, the security context isn't propagated to child threads. For more information, refer to our Spring Security Context Propagation article.

Using @PreFilter and @PostFilter Annotations

- Spring Security provides the @PreFilter annotation to filter a collection argument before executing the method:

```
// here  we're joining all usernames except for the one that is
authenticated.
@PreFilter("filterObject != authentication.principal.username")
public String joinUsernames(List<String> usernames) {
    return usernames.stream().collect(Collectors.joining(";"));
}
```

- Here, in our expression, we use the name filterObject to represent the current object in the collection.
- However, if the method has more than one argument that is a collection type, we need to use the filterTarget property to specify which argument we want to filter:

```
@PreFilter
  (value = "filterObject != authentication.principal.username",
  filterTarget = "usernames")
public String joinUsernamesAndRoles(
  List<String> usernames, List<String> roles) {

    return usernames.stream().collect(Collectors.joining(";"))
      + ":" + roles.stream().collect(Collectors.joining(";"));
}
```

- Additionally, we can also filter the returned collection of a method by using the @PostFilter annotation:

```
@PostFilter("filterObject != authentication.principal.username")
public List<String> getAllUsernamesExceptCurrent() {
    // Spring Security will iterate through the returned list and remove
any value matching the principal's username.
    return userRoleRepository.getAllUsernames();
}
```

- In this case, the name filterObject refers to the current object in the returned collection.

Method Security Meta-Annotation(Creating a global interafce)

- We typically find ourselves in a situation where we protect different methods using the same security configuration.

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@PreAuthorize("hasRole('VIEWER')")
public @interface IsViewer {
}

//  Next, we can directly use the @IsViewer annotation to secure our
method:
@IsViewer
public String getUsername4() {
    //...
}
```

- Security meta-annotations are a great idea because they add more semantics and decouple our business logic from the security framework.

Security Annotation at the Class Level

If we find ourselves using the same security annotation for every method within one class, we can consider putting that annotation at class level:

```
//  the security rule hasRole('ROLE_ADMIN') will be applied to both
getSystemYear and getSystemDate methods.
@Service
@PreAuthorize("hasRole('ROLE_ADMIN')")
public class SystemService {

    public String getSystemYear(){
        //...
    }

    public String getSystemDate(){
        //...
    }
}
```

Multiple Security Annotations on a Method

```
// Spring will verify authorization both before and after the execution of
the securedLoadUserDetail method.
@PreAuthorize("#username == authentication.principal.username")
@PostAuthorize("returnObject.username ==
authentication.principal.nickName")
public CustomUser securedLoadUserDetail(String username) {
    return userRoleRepository.loadUserByUserName(username);
}
```

Java Versions

Java 8 Features

1. `Streams` are used for processing sequences of elements. They support operations like map, filter, and reduce.
2. `Lambdas` provide a concise way to represent anonymous functions.
3. `Optional` is used to represent a value that may or may not be present.

```
Optional<String> name = names.stream().filter(name ->
name.startsWith("A")).findFirst(name -> name == "Hello);
```

1. `Functional Interfaces` are interfaces with a single abstract method, which makes them ideal for lambda expressions and method references.

- Predicate
    - Represents a boolean-valued function of one argument.
    - Method: `boolean test(T t)`
- Function<T, R>
    - Represents a function that accepts one argument and produces a result.
    - Method: `R apply(T t)`
- Supplier
    - Represents a supplier of results, providing a result without any input.
    - Method: `T get()`
- Consumer
    - Represents an operation that accepts a single input argument and returns no result.
    - Method: `void accept(T t)`

Spring 1 Vs 2 Vs 3

Spring Framework 1

- Inversion of Control (IoC) container
- Aspect-oriented programming (AOP).
- Configuration was primarily XML-based.
- Limited support for annotations.
- Basic support for integration with other frameworks and technologies.

Spring Framework 2

- Introduced support for annotation-based configuration (e.g., @Autowired, @Component, @Service, @Repository, etc.).
- Improved integration with other frameworks like Hibernate and JPA.
- Enhanced testing support with the introduction of Spring TestContext Framework.
- Enhanced AOP support with improved proxy-based and aspectJ-based implementations.
- Introduced support for AspectJ annotations for AOP.
- Introduced support for generic-based dependency injection (e.g., generics in autowiring).

Spring Framework 3 (Focus on simplification and ease of use)

- Enhanced support for RESTful web services with the introduction of @RequestBody and @ResponseBody annotations.
- Enhanced support for asynchronous processing.
- Introduced support for Java 5 features such as generics and varargs.
- Improved support for declarative transaction management.

- Introduced support for Spring Expression Language (SpEL) for more flexible bean wiring and manipulation.
- Introduced the @Configuration annotation for Java-based configuration.

Table

Feature	Spring Framework 1	Spring Framework 2	Spring Framework 3
Inversion of Control (IoC) container	✓	✓	✓
Aspect-oriented programming (AOP)	✓	✓	✓
Configuration	XML-based	XML-based	XML-based / Java-based
Annotation support	limited	✓	✓
Integration with other frameworks/technologies	Basic	Improved	Improved
Annotation-based configuration		✓	✓
Support for AspectJ annotations for AOP		✓	✓
Generic-based dependency injection		✓	✓
Enhanced testing support(Spring TestContext Framework)		✓	✓
Support for RESTful web services			✓ (@RequestBody and @ResponseBody annotations)
Support for Java 5 features			✓ (e.g., generics and varargs)
Improved support for declarative transaction management			✓
Support for Spring Expression Language (SpEL)			✓
Enhanced support for asynchronous processing			✓

- In the table:
  - ✓ represents the presence of the feature in the respective Spring Framework version.
  - Empty cells indicate that the feature was not specifically introduced or highlighted in that version.

AOP

Proxy-based and AspectJ-based implementations are two approaches to implementing Aspect-Oriented Programming (AOP) in the Spring Framework:

Proxy-based Implementation:

- In proxy-based AOP, Spring creates a proxy object that wraps around the target object.
- This proxy intercepts method invocations made on the target object and applies the aspects (cross-cutting concerns) before or after the method execution.

Usage: Proxy-based AOP is commonly used when you have Spring beans that implement interfaces. It's applicable when you want to apply aspects to method invocations on these interfaces.

How it works: Spring creates a proxy object that implements the same interface(s) as the target bean. When a method is invoked on the interface, the proxy intercepts the call, applies the aspect, and then delegates the call to the actual bean.

AspectJ-based Implementation:

- AspectJ is a powerful and mature AOP framework that provides a more comprehensive and flexible approach to AOP compared to proxy-based AOP.
- AspectJ offers a more extensive set of features for defining and applying aspects, including **pointcut expressions** for fine-grained control over join points.

Usage: AspectJ-based AOP is more flexible and powerful. It's suitable when you need fine-grained control over cross-cutting concerns or when you want to apply aspects to classes that don't necessarily implement interfaces.

How it works: AspectJ weaves aspects directly into your compiled classes at compile time, load time, or runtime. This means that aspects become part of the bytecode, and there's no need for proxy objects. You can target specific methods, constructors, or fields with AspectJ expressions.

- AspectJ Annotations for AOP:
  - `@Aspect`: This annotation is used to define an aspect in AspectJ. An aspect is a modular unit of cross-cutting concerns.
  - `@Pointcut`: This annotation is used to define a pointcut expression, which specifies a set of join points in the application where advice should be applied.
  - `@Before`: This annotation is used to define advice that should be executed before a method execution.
  - `@AfterReturning`: This annotation is used to define advice that should be executed after a method successfully returns a value.
  - `@AfterThrowing`: This annotation is used to define advice that should be executed after a method throws an exception.
  - `@After`: This annotation is used to define advice that should be executed after a method execution (regardless of its outcome).

```
@Aspect
public class LoggingAspect {

    @Before("execution(* com.example.service.*.*(..))")
```

```
    public void logBefore(JoinPoint joinPoint) {
        System.out.println("Before method: " +
joinPoint.getSignature().getName());
    }

    // Define a pointcut for all methods in the service package
    @Pointcut("execution(* com.example.service.*.*(..))")
    public void serviceMethods() {}

    // Define an advice that logs execution time
    @Around("serviceMethods()")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws
Throwable {
        long startTime = System.currentTimeMillis();

        Object proceed = joinPoint.proceed();  // Execute the method

        long executionTime = System.currentTimeMillis() - startTime;

        System.out.println(joinPoint.getSignature() + " executed in " +
executionTime + "ms");

        return proceed;
    }
}
```

- In this example, @Aspect annotation defines the LoggingAspect class as an aspect, and @Before annotation defines the logBefore advice, which is executed before the execution of methods in the com.example.service package.
- `@Pointcut`: The pointcut expression "execution(* com.example.service..(..))" matches all methods in classes under the com.example.service package. The serviceMethods() method is just a placeholder for this pointcut and is used in the @Around advice.
- `@Around`: The logExecutionTime() method is an advice that wraps the execution of the matched methods. It calculates the time taken to execute the method and logs it.

Varags

Varargs, short for "variable-length arguments," is a feature introduced in Java 5 that allows methods to accept a variable number of arguments. This is useful when the number of arguments passed to a method may vary, providing flexibility and convenience in method invocation.

```
public class VarargsExample {

    // Method with varargs parameter
    public static void printNumbers(String... numbers) {
        System.out.println("Printing numbers:");
        for (String number : numbers) {
            System.out.println(number);
        }
    }
```

```
    public static void main(String[] args) {
        // Invoking the method with different numbers of arguments
        printNumbers("1", "2", "3");
        printNumbers("4", "5");
        printNumbers("6");
        printNumbers(); // No arguments
    }
}
```

```
Printing numbers:
1
2
3
Printing numbers:
4
5
Printing numbers:
6
Printing numbers:
```

Declarative transaction management

Declarative transaction management in Java Spring involves specifying transactional behavior in a declarative manner using configuration metadata, rather than writing code to manage transactions programmatically. This allows developers to separate transaction management concerns from business logic, resulting in cleaner, more maintainable code.

Here's how declarative transaction management works in Spring:

- Configuration Metadata: Transactional behavior is specified using configuration metadata such as XML or annotations.
- Transaction Attributes: Transactional attributes define the behavior of transactions, such as propagation behavior, **isolation level**, **rollback rules**, and **timeout**.
    - Propagation behavior refers to how a method should participate in a transaction, specifically when it is called by another method that is already part of a transaction. It defines the relationship between the existing transaction and the transaction of the method being invoked.
- Advice: Spring applies transactional behavior to methods using AOP (Aspect-Oriented Programming) advice. This advice intercepts method invocations and applies transaction management logic before and after method execution.
- Proxying: Spring creates proxies around the target beans that require transaction management. These proxies intercept method calls and delegate transaction-related logic to the underlying transaction manager.
- Transaction Manager: Spring uses a transaction manager to coordinate transactions with the underlying data source (e.g., database). The transaction manager is responsible for beginning, committing, or rolling back transactions as needed.

Declarative transaction management can be configured using two main approaches in Spring:

1. XML Configuration
2. Annotation-Based Configuration:

```
@Configuration
@EnableTransactionManagement
public class AppConfig {

    @Bean
    public DataSource dataSource() {
        // Configure and return data source
    }

    @Bean
    public PlatformTransactionManager transactionManager() {
        return new DataSourceTransactionManager(dataSource());
    }
}

@Service
@Transactional
public class UserService {
    // Service methods
}
```

Transactional behavior is applied to @Transactional (annotations). The transaction manager (DataSourceTransactionManager in this example) coordinates transactions with the underlying data source.

- Overall, declarative transaction management simplifies transaction handling by allowing developers to focus on business logic while abstracting away transaction-related concerns.

@Configuration annotation for Java-based configuration

The @Configuration annotation in Java-based configuration is used to indicate that a class contains bean definitions and configuration metadata for the Spring container. When Spring sees this annotation on a class, it knows that it should treat the class as a source of bean definitions rather than a regular Java class.

Lombok

- Its build with Java
- To avoid boilerplate code
- Getter, Setters and Constructors are easy to have with just annotations
- can be used in any Java application (Web, MVC, Android,....)

How spring handle multiple requests to endpoints concurrently

In a Spring REST application, the handling of requests to endpoints is typically handled concurrently by the server. This means that when multiple clients send requests to the same endpoint simultaneously, the server can process these requests concurrently, handling each request independently and potentially in parallel.

Here's how this typically works:

- **Concurrency Model**: Most modern web servers, including those used with Spring applications, employ a concurrency model that allows them to handle multiple requests concurrently. This means that the server can process multiple requests at the same time, utilizing multiple threads or asynchronous processing techniques.
- **Thread Pool**: Web servers often use a thread pool to manage incoming requests. When a request arrives, it is assigned to an available thread from the thread pool, allowing the server to handle multiple requests simultaneously without having to create a new thread for each request.
- **Servlet Container**: In the case of Spring MVC applications, requests are typically handled by a servlet container (e.g., Tomcat, Jetty) that manages the concurrency aspects of request handling. The servlet container is responsible for managing threads and ensuring that requests are processed concurrently.
- **Concurrency Configuratio**n: You can configure the concurrency behavior of your Spring application through settings in the servlet container or through Spring configuration. For example, you can configure the size of the thread pool, enable asynchronous processing for certain requests, or configure concurrency limits.
- **Thread Safety**: It's important to ensure that your application components, including the REST endpoints and any shared resources they access, are thread-safe. This means that they can safely handle concurrent access by multiple threads without causing data corruption or other concurrency issues.

Number of threads can one laptop handle

The number of threads a laptop can handle for a server application to process multiple requests concurrently depends on several factors:

- Hardware Resources: The laptop's CPU, memory, and disk I/O capabilities influence the number of threads it can handle effectively.
- Operating System: The operating system may impose limits on the number of threads a process can create.
- Configuration: The server application's configuration, such as thread pool size, can also affect the number of concurrent requests it can handle.
- Typically, modern laptops can handle hundreds or even thousands of threads concurrently. However, it's essential to consider the laptop's hardware specifications and the specific requirements of your server application to determine an optimal configuration.

Java Versions Comparision

Java Version	Release Date	Notable Features	Performance Improvements	Long-Term Support (LTS)

Java Version	Release Date	Notable Features	Performance Improvements	Long-Term Support (LTS)
Java 8	March 2014	- Lambda Expressions - Stream API - Date and Time API - Nashorn JavaScript Engine - Optional class	Significant improvements in JVM performance	Yes
Java 9	September 2017	- Module System (Project Jigsaw) - JShell (REPL) - Improved Javadoc - HTTP/2 Client	Performance optimizations, especially in garbage collection	No
Java 10	March 2018	- Local-Variable Type Inference (var) - Garbage-Collector Interface - Application Class-Data Sharing (AppCDS)	Further improvements in performance	No
Java 11	September 2018	- New HTTP Client API - Local-Variable Syntax for Lambda Parameters - Epsilon: A No-Op Garbage Collector	Enhanced performance with flight recorder and TLS 1.3	Yes
Java 12	March 2019	- Switch Expressions (preview) - Default CDS Archives - Shenandoah: A Low-Pause-Time Garbage Collector (experimental)	JVM Constants API for nominal descriptions	No
Java 13	September 2019	- Switch Expressions (enhanced, preview) - Text Blocks (preview) - Reimplement the Legacy Socket API	Improved start-up, CDS, and garbage collection	No
Java 14	March 2020	- Switch Expressions (standard) - Text Blocks (second preview) - Helpful NullPointerExceptions	Improved performance with NUMA-aware memory allocation	No
Java 15	September 2020	- Text Blocks (standard) - Sealed Classes (preview) - Hidden Classes	Further JVM and garbage collection optimizations	No

Java Version	Release Date	Notable Features	Performance Improvements	Long-Term Support (LTS)
Java 16	March 2021	- Records (standard) - Pattern Matching for instanceof (preview) - Sealed Classes (second preview)	Vector API (incubator) for better performance	No
Java 17	September 2021	- Sealed Classes (standard) - Pattern Matching for switch (preview) - Foreign Function & Memory API (incubator)	Significant performance improvements	Yes
Java 18	March 2022	- UTF-8 by default - Simple Web Server - Code Snippets in Javadoc	Minor performance enhancements	No
Java 19	September 2022	- Virtual Threads (preview) - Structured Concurrency (incubator) - Pattern Matching for switch (third preview)	Enhanced performance for concurrency	No
Java 20	March 2023	- Record Patterns (preview) - Foreign Function & Memory API (second incubator) - Virtual Threads (second preview)	Performance improvements for new concurrency models	No

Summary

- **Java 8**: Introduced major language features like lambdas and streams, and has long-term support.
- **Java 9-20**: Introduced incremental improvements and new features such as module system, local-variable type inference, new garbage collectors, pattern matching, and virtual threads.
- **LTS Versions**: Java 8, 11, and 17 are designated as Long-Term Support (LTS) versions, providing extended support and stability.

Choosing the right version of Java depends on the need for stability and support (LTS versions) or the desire to utilize the latest language features and improvements.

Jdk vs Jre what are they are what are the differences

JDK (Java Development Kit) and JRE (Java Runtime Environment) are both components of the Java platform, but they serve different purposes and have different capabilities.

JDK (Java Development Kit):

JDK is a software development kit used for developing Java applications. It includes tools and utilities necessary for Java development, such as the Java compiler (javac), Java runtime environment (java), Java Archive (JAR) utility, debugger (jdb), and documentation generator (javadoc). JDK also includes libraries and frameworks required for Java development, such as the Java Development Kit (JDK) libraries, JavaFX, and other development tools.

```
Developers use JDK to write, compile, debug, and package Java
applications.
```

JRE (Java Runtime Environment):

JRE is a runtime environment used for running Java applications. It includes the Java Virtual Machine (JVM), Java core libraries, and other runtime components required for executing Java applications. JRE does not include development tools like the compiler or debugger.

```
Users need to install JRE on their machines to run Java applications. It
provides the necessary runtime support for executing Java bytecode.
```

Differences

Purpose: JDK is used for Java application development, while JRE is used for running Java applications.

Capabilities: JDK allows developers to write, compile, debug, and package Java applications, while JRE only provides the runtime environment for executing Java applications.

```
If you are a developer and need to write, compile, and debug Java
applications, you should download the JDK.
If you are an end-user and only need to run Java applications, you can
download the JRE.
```

Java features

Record

- In Java, a record is a special kind of class introduced in Java 14 (JEP 359) that is designed primarily for immutable data modeling. It provides a concise syntax for declaring classes whose main purpose is to store data.
- Key Features of Java Records:
- Immutable by Default: The state of a record cannot be modified once it's been initialized, making it inherently immutable.

    - Final Fields: The fields in a record are final, which means they cannot be reassigned after the record is constructed.
    - No Setters: Records do not provide setter methods, so you cannot change the values of the fields after the object is created.

- Automatic Getter Methods: Automatically generates accessor methods (getters) for all its components (fields).
- equals() and hashCode(): Automatically implements equals() and hashCode() methods based on the record's components.
- toString(): Provides a toString() method that outputs the record's state in a readable format.
- Constructor: Automatically generates a constructor to initialize the record's components.
- Syntax Example:

```
public record Person(String name, int age) {
    // Additional methods or fields can be added here
}

// Creating a new Person record
Person person = new Person("Alice", 30);

// Accessing components (fields)
String name = person.name();
int age = person.age();

// Immutable nature ensures the state cannot be changed after
initialization
// person.name = "Bob"; // This would cause a compilation error
```

- Benefits of Java Records:
    - Simplicity.
    - Immutability by Default. Developers don't need to Implement Immutability
    - Conciseness and Readability.
    - Developers Need to Write Boilerplate Code.

SpEL

SpEL (Spring Expression Language) is a powerful expression language that supports querying and manipulating an object graph at runtime. It is part of the Spring Framework and is used to wire values into beans, typically in the Spring configuration.

Key Features of SpEL

- Accessing Properties: Allows access to properties of beans and objects.
- Method Invocation: Can call methods on objects.
- Operators: Supports various operators (arithmetic, relational, logical).

- Regular Expressions: Can use regular expressions for pattern matching.
- Collection Selection and Projection: Supports querying collections and maps.
- Templating: Can be used within Spring's @Value annotations for dynamic values.

Common Use Cases

1. Bean Configuration:

```
@Value("#{beanName.propertyName}")
private String property;
```

2. Arithmetic Operations:

```
@Value("#{beanName.property1 + beanName.property2}")
private int sum;
```

3. Logical Operations:

```
@Value("#{beanName.property1 > 5 ? 'High' : 'Low'}")
private String level;
4. Method Invocation:
```java
@Value("#{beanName.methodName()}")
private String result;
```

5. Collection Handling:

```
@Value("#{beanName.listProperty[0]}")
private String firstElement;
```

antMatchers

- In Spring Security, antMatchers is a method used to configure URL patterns for security rules. It allows you to specify which URL paths should be secured and what kind of access is required for those paths.
- The antMatchers method uses Ant-style path patterns to match URLs.

```
@Configuration
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
```
```

```
            .authorizeRequests()
            .antMatchers("/public/**").permitAll() // Allow all access to
URLs under /public/
            .antMatchers("/admin/**").hasRole("ADMIN") // Restrict access
to /admin/ URLs to users with the ADMIN role
            .antMatchers("/user/**").hasAnyRole("USER", "ADMIN") //
Restrict access to /user/ URLs to users with USER or ADMIN roles
            .anyRequest().authenticated() // Require authentication for
any other request not specified by previous antMatchers rules.
            .and()
            .formLogin() // Enable form-based login
            .and()
            .httpBasic(); // Enable basic authentication
    }
}
```

- antMatchers provides a flexible way to secure specific parts of your application based on URL patterns and roles.

C++ vs Java

Feature	Java	C++
Language Type	Object-oriented, platform-independent.	Procedural and object-oriented, platform-dependent.
Memory Management	Automatic (Garbage Collection).	Manual (`new`/`delete`).
Compilation	Compiled to bytecode, runs on JVM.	Compiled to machine code.
Multiple Inheritance	Not supported (uses interfaces).	Supported.
Pointers	No direct pointer manipulation.	Full support for pointers.
Performance	Slower due to JVM overhead.	Generally faster, direct machine code execution.
Templates/Generics	Uses generics with type erasure.	Powerful templates, resolved at compile-time.
Memory Safety	Safer due to lack of pointers, garbage collection.	Less safe, manual memory management.
Portability	"Write Once, Run Anywhere" with JVM.	Platform-dependent, may need code changes.
Concurrency	Built-in support with `java.util.concurrent`.	Lower-level support, manual synchronization.